# Gym Management Platform - Budget VPS Production

2 September 2026


System Architecture

1. Purpose and scope
2. Architecture principles
3. Logical system view
4. Application boundaries
    - Web frontend
    - Backend API
    - Mobile application
5. Multi-tenant architecture
6. PostgreSQL RLS integration
7. Identity and authorization
    - Data ownership classification
8. Authentication flow
    - Staff web
    - Mobile member
    - Authorization changes
9. Data and domain architecture
10. Financial architecture
    - Atomic financial workflows
11. File architecture
12. Deployment architecture
13. Reliability, backup, and recovery
14. Observability
15. Scalability path

Phase 0 Authoritative Database Design

1. Non-negotiable conventions
2. Scope, ownership, and RLS inventory
    - Platform-global tables (no `gym_id`, no tenant RLS)
    - Tenant-owned tables (all require `gym_id`, forced RLS)
3. Role provisioning and authorization
4. Entity definitions
    - User (global)
    - Gym (global tenant root)
    - GymSettings
    - GymMembership
    - StaffInvitation
    - Role, RolePermission, GymMembershipRole
    - Feature entitlement catalogs
    - StaffProfile
    - Member
    - MembershipPlan
    - Membership
    - CheckIn
    - Payment
    - Receipt
    - ExpenseCategory and Expense
    - SalaryPeriod and StaffSalaryLine
    - TrainerAssignment
    - PTSale and PTCommission
    - CashSession and CashTransaction

- TenantSequence
- FileAsset
- Notification
- AuditLog
- RefreshSession (global)
- PlatformAuditLog (global)
- IdempotencyRecord

5. ERD
6. Cross-tenant integrity and RLS
7. Index contract
8. Transaction and concurrency invariants
9. Deletion, immutability, and retention
10. Migration and verification rules

Phase 0 REST API Contract

1. Contract principles
2. Standard envelopes
3. HTTP behavior
4. Authentication API
   - Staff token validation sequence
   - Member token contract
5. Platform administration — Phase 1B
6. Tenant resource API
   - Gym profile and settings — Phase 1B
   - Dashboard — Phase 6
   - Staff, roles, and permissions — Phase 1B
   - Members — Phase 2
   - Membership plans and memberships — Phase 2
   - Check-ins — Phase 3
   - Payments, sales, and receipts — Phase 4
   - Expenses — Phase 4
   - Salaries — Phase 4
   - Cash book — Phase 4
   - Trainers, assignments, PT sales, and commissions — Phase 5
   - Reports and dashboard — Phase 6
   - Signed files — Phases 2-7 by owning feature
   - Audit and notifications — Phases 1B and 7
7. Member mobile API — Phase 7
8. Pagination, filtering, and concurrency
9. Idempotency
10. API security requirements
    - Permission and feature-entitlement evaluation
11. Delivery map

Security Architecture

1. Security objectives
2. Threat model
3. Layered tenant isolation
   - Layer 1: authentication
   - Layer 2: principal-to-gym relationship
   - Layer 3: Django query scoping
   - Layer 4: database constraints
   - Layer 5: PostgreSQL RLS
   - Layer 6: role and object permissions
   - Layer 7: automated tests and deployment checks
   - Two mandatory, independent controls
4. PostgreSQL context safety
5. Authentication and sessions

- Passwords and MFA
- JWT
- Browser storage
- Mobile storage
- Revocation

6. Authorization model
   - Platform permission matrix
   - Tenant permission matrix
   - Delegation and custom roles
   - Object restrictions and entitlements
7. API defenses
8. Web and mobile defenses
   - XSS
   - CSRF and CORS
   - Client data
9. File security
10. Financial security
11. Audit and privacy
12. Infrastructure and secrets
13. Rate limits and abuse controls
14. Failure behavior
15. Security test suite
    - Tenant isolation
    - Authorization
    - Authentication and API
    - Files and finance
    - Database and deployment
16. Phase 0 design status
17. Security release gate

Project Structure

1. Repository strategy
2. Backend
   - Domain app convention
3. Next.js web
   - Frontend boundaries
4. React Native mobile
5. Infrastructure
6. Tests
   - Backend
   - Frontend and mobile
7. Configuration and dependency rules
8. Naming rules
9. Approved roadmap alignment
10. Architectural decision records

Development Plan

1. Authority and delivery policy
2. Phase 0 — Approved design baseline
   - Dependencies
   - Scope and 15 required design deliverables
   - Migrations and data
   - Required reviews and checks
   - Exit criteria
3. Phase 1 — Foundation
   - Phase 1A — Security foundation
   - Phase 1B — Required Super Admin provisioning and gym prerequisites

4. Phase 2 — Members and memberships
    - Dependencies
    - Scope
    - Migrations and data
    - Required tests
    - Checks
    - Exit criteria
5. Phase 3 — Check-in and device abstraction
    - Dependencies
    - Scope
    - Migrations and data
    - Required tests
    - Checks
    - Exit criteria
6. Phase 4 — Core finance, approvals, salaries, receipts, expenses, and cash
    - Dependencies
    - Scope
    - Migrations and data
    - Required tests
    - Checks
    - Exit criteria
7. Phase 5 — Personal training
    - Dependencies
    - Scope
    - Migrations and data
    - Required tests
    - Checks
    - Exit criteria
8. Phase 6 — Dashboards and reports
    - Dependencies
    - Scope
    - Migrations and data
    - Required tests
    - Checks
    - Exit criteria
9. Phase 7 — React Native member application
    - Dependencies
    - Scope
    - Migrations and data
    - Required tests
    - Checks
    - Exit criteria
10. Phase 8 — Hardening, operations, and production deployment
    - Dependencies
    - Scope
    - Migrations and data
    - Required tests
    - Checks and deployment sequence
    - Exit criteria
11. Cross-phase test and quality strategy
    - Test pyramid and required evidence
    - CI quality gates
12. Backup, recovery, and continuity
13. Scalability path
    - Near term
    - Evidence-driven later options

14. Phase 0 deployment cost planning estimate
    Approved monthly target
    Cost and availability trade-off
    Capacity, recovery, and approval conditions
    Exclusions
15. Open decisions and approval register

ADR 0001: Single-VPS Budget Production Topology
Context
Decision
Budget
Security and recovery consequences
Alternatives considered
    Managed cloud services
    Separate application and database VPSs
    Kubernetes
Upgrade triggers
Rollback and compatibility

# System Architecture

## 1. Purpose and scope

This document defines the Phase 0 target architecture for a multi-tenant gym management SaaS. Phase 0 produces design documentation only; it does not implement application features, infrastructure, migrations, or policies.

The system has three clients:

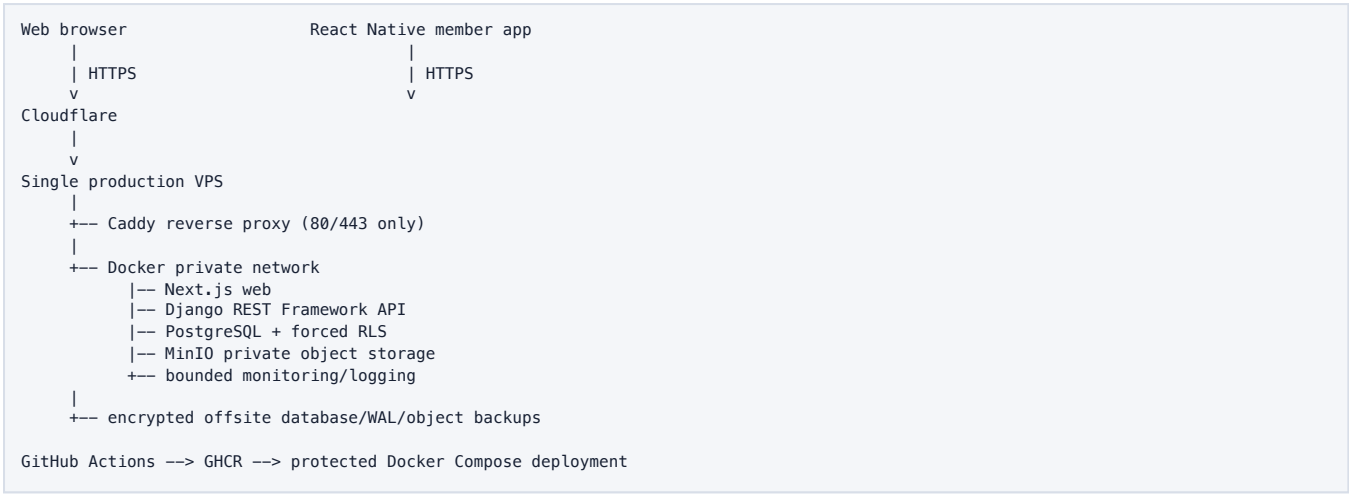
- A Next.js 15+ web application using the App Router, TypeScript, Tailwind CSS, and React Server Components (RSC) for platform and gym staff.
- A Django REST Framework (DRF) API under `/api/v1` containing all authorization and business rules.
- A separate React Native member application using the same versioned API.

The primary design goal is fail-closed tenant isolation. UI filtering is never a security control.

## 2. Architecture principles

1. Every tenant-owned record belongs to exactly one `Gym`.
2. The authenticated server context, never client input, selects the active gym.
3. Tenant isolation is enforced independently by API scoping and PostgreSQL row-level security (RLS).
4. Business rules live in backend domain services, not views or clients.
5. Financial values use decimal arithmetic and posted financial events are reversed or voided, not deleted.
6. PostgreSQL is the source of truth. Redis and Celery are introduced only for measured needs.
7. Public API contracts are versioned and do not expose Django models directly.
8. Server timestamps are UTC; business dates are interpreted in each gym's IANA timezone.
9. Currency defaults to Thai baht (`THB`), while source documents retain an explicit ISO 4217 currency.
10. Authentication, tenant selection, authorization, and financial writes fail closed when required context is missing or inconsistent.

## 3. Logical system view




Next.js may act as a thin backend-for-frontend for browser authentication cookies and server rendering. It must not duplicate authorization or financial rules. React Native calls the same API contracts directly. The single VPS is an explicitly accepted budget-production single point of failure; it is not represented as high availability.

## 4. Application boundaries

### Web frontend

- Next.js 15+ with TypeScript, Tailwind CSS, the App Router, and React Server Components by default.
- Client Components only for browser-only interaction or local transient state.
- Feature modules consume typed API contracts.
- Route layouts separate `(auth)`, tenant routes, and `super-admin`.
- Permission-aware navigation improves usability but does not grant access.
- No direct database or object-storage access.

### Backend API

- Django REST Framework provides `/api/v1/`.
- Authentication establishes user, session, and active gym.
- Tenant-aware querysets and serializer relation fields constrain all data access.
- Permission classes evaluate stable permission codes.
- Domain services own membership expiry, finance approval, payments, salary, commission, cash reconciliation, receipt sequencing, and reporting rules.
- Selectors encapsulate complex read queries and tenant-scoped aggregates.
- Database transactions define business operation boundaries.

### Mobile application

- React Native with TypeScript in a separate `/mobile` workspace.
- Access tokens are short-lived; refresh credentials use Keychain/Keystore-backed storage.
- API, authentication, feature, navigation, and UI layers are separated.
- Offline caching is limited to non-sensitive display data with explicit invalidation.
- QR payloads use opaque, rotatable identifiers rather than member details.

## 5. Multi-tenant architecture

The platform uses one PostgreSQL database and shared tables. `Gym` is the tenant root. Every tenant-owned table has `gym_id UUID NOT NULL`, including members, plans, memberships, check-ins, payments, expenses, salary periods/lines, commissions, cash records, receipts, files, notifications, staff assignments, and tenant audit events.

Tenant context is established as follows:

1. Verify JWT signature, issuer, audience, algorithm, token type, and expiry.
2. Read `sub`, `gym_id`, `gym_membership_id`, `session_id`, and `authz_version`.
3. Begin the request database transaction and set transaction-local `app.user_id` and `app.request_id`.
4. Validate the global refresh/session record, selected gym claim, active user, active gym, and subscription without querying tenant-owned tables.

5. Set transaction-local `app.gym_id` from that validated server-side selection.
6. Reload the selected `GymMembership` under forced RLS and verify active status, user/gym agreement, and current authorization version.
7. Execute only Django-scoped tenant queries and permission checks.

The client may send resource identifiers, but it cannot select ownership. Create services assign `gym=request.gym`; serializer fields never accept writable `gym_id`.

Cross-tenant identifiers return `404` for ordinary resource APIs to avoid disclosing existence.

## 6. PostgreSQL RLS integration

RLS is enabled and forced on every tenant-owned table. Policies use both `USING` and `WITH CHECK`.

```
ALTER TABLE members_member ENABLE ROW LEVEL SECURITY;
ALTER TABLE members_member FORCE ROW LEVEL SECURITY;

CREATE POLICY member_gym_isolation ON members_member
USING (
  gym_id = NULLIF(current_setting('app.gym_id', true), '')::uuid
)
WITH CHECK (
  gym_id = NULLIF(current_setting('app.gym_id', true), '')::uuid
);
```

The API runs as a non-owner, non-superuser role without BYPASSRLS. A separate migration role owns schema objects. Platform-wide operations use narrowly scoped service functions or an explicitly audited operational path; the normal tenant runtime never gains a general RLS bypass.

All tenant database work is wrapped in `transaction.atomic()`, including API requests, workers, scheduled jobs, management commands, and tenant-aware operational tools. Staff authentication installs context in two transaction-local steps: first the globally authenticated identity and request, then the gym only after global session and selected-gym validation. The first tenant query occurs after all three values are installed:

```
SELECT set_config('app.user_id', %s, true),
       set_config('app.request_id', %s, true);

-- After global session, selected gym, user, and gym lifecycle checks:
SELECT set_config('app.gym_id', %s, true);
```

The `true` argument prevents tenant context leaking through persistent or pooled connections after commit or rollback. Tenant queries must not run in middleware before the transaction context exists. Workers and management commands establish the same context for one gym per transaction.

## 7. Identity and authorization

### Data ownership classification

The shared PostgreSQL schema separates platform-global data from tenant-owned data:

- **Platform-global:** `User`, authentication credentials, refresh-session/token-family records, platform roles, platform audit events, system permission definitions, deployment metadata, and other records whose meaning is independent of a gym. These records do not acquire a nullable `gym_id` merely for convenience.
- **Tenant root:** `Gym` identifies a tenant. Its lifecycle is managed through a tightly controlled platform path.
- **Tenant-owned:** `GymMembership`, gym settings, staff profiles and assignments, members, plans, member memberships, check-ins, payments, refunds, expenses, salary periods/lines, commissions, cash sessions and transactions, receipts, tenant files, notifications, and tenant audit events. Every such table has a physical `gym_id UUID NOT NULL`, including join, history, and derived tables. A transactional outbox remains deferred; if introduced after measured need, it will also be gym-owned and RLS-protected.

Tenant ownership is explicit rather than inferred through a join. Foreign keys and uniqueness constraints include `gym_id` where necessary to prevent same-tenant relationships from crossing gyms. Platform-global operations are separated from tenant request paths and never turn a tenant table into globally readable data.

`User` is a global authentication identity and can have multiple `GymMembership` records, including multiple simultaneously active memberships for legitimate engagements at different gyms. Each `GymMembership` belongs to exactly one gym. Every gym-scoped access token selects exactly one currently active membership and carries both its `gym_membership_id` and `gym_id`; it never grants an implicit union of all of the user's memberships. A `StaffProfile` remains gym-specific.

A gym may have multiple active Owner memberships. Each Owner authenticates with an independent user/session and receives the same protected OWNER role within that gym; no shared account or password is used. Owner addition and revocation use a separate audited, MFA-protected workflow, lock the gym row, and preserve at least one active Owner. `is_owner` is an authorization marker only. Ownership percentages, capital contributions, and profit distributions are outside the current model and can later be added as separate effective-dated domain records without changing login, tenant isolation, or historical finance semantics.

Authorization combines:

1. active user and non-revoked session;

2. active gym and subscription;
3. active `GymMembership`;
4. one or more roles;
5. role permission codes;
6. object conditions, such as a trainer being assigned to a member;
7. feature entitlement where applicable.

Roles are data-driven. System role templates seed OWNER, MANAGER, RECEPTIONIST, ACCOUNTANT, and TRAINER. Gym owners may create custom roles without creating new application code. Permission codes remain stable API capabilities.

Platform `SUPER_ADMIN` is a platform role, not a tenant role. Its routes, permissions, UI, and audit trail are separate.

## 8. Authentication flow

JWT acceptance is strict: use an algorithm allowlist; verify signature, issuer (`iss`), audience (`aud`), expiry (`exp`), not-before (`nbf`) when present, issued-at sanity, token type, subject, session, and required tenant claims. Reject `alg=none`, algorithm/key confusion, unknown key IDs, malformed or duplicated claims, revoked sessions, inactive users, inactive memberships, gym/membership mismatches, and stale authorization versions. Clock skew is small and explicit. The API never accepts `gym_id` from a header, URL, cookie, or request body as a substitute for the verified token selection.

Authentication middleware performs cryptographic and syntactic token validation only and makes no tenant database queries. A transaction-wrapped API boundary first sets `app.user_id` and `app.request_id`, validates global session and selected-gym state, then sets `app.gym_id`. Only then does it reload `GymMembership` and perform tenant authorization under forced RLS and Django scoping.

```
sequenceDiagram
    autonumber
    participant C as Browser / mobile
    participant A as DRF /api/v1
    participant D as PostgreSQL + forced RLS
    C->>A: Request + gym-scoped JWT
    A->>A: Verify signature and strict claims (no DB query)
    A->>D: BEGIN transaction.atomic()
    A->>D: set_config(user_id, request_id, true)
    A->>D: Verify global session + selected active Gym
    A->>D: set_config(gym_id, true)
    A->>D: Reload selected active GymMembership under RLS
    D-->>A: Membership matches token gym and authz_version
    A->>D: Django-scoped tenant query
    D->>D: Enforce USING / WITH CHECK RLS policy
    D-->>A: Authorized rows only
    A-->>C: Response (404 for cross-tenant resource)
```

### Staff web

1. Browser submits credentials to an authentication endpoint through the web boundary.
2. API validates credentials and optional MFA.
3. If the user has one active gym membership, the API issues a gym-scoped access token; otherwise the user selects from active memberships returned only for that identity. A user may switch gyms only by exchanging or refreshing credentials for a new token that selects one different active membership.
4. Access tokens are short lived. Rotating refresh tokens are represented by hashed server-side sessions.
5. Browser tokens are held in `HttpOnly`, `Secure`, appropriate `SameSite` cookies; state-changing cookie requests require CSRF protection.
6. Refresh reuse revokes the token family.

### Mobile member

Member authentication maps a global user identity to a gym-scoped `Member` record. The token includes that gym and member link. Member endpoints enforce both tenant scope and self scope.

### Authorization changes

Role changes, suspension, password reset, and session revocation increment `authz_version` or revoke sessions. Sensitive requests compare token state with authoritative storage.

## 9. Data and domain architecture

Apps are organized by domain: tenants, users, members, memberships, check-ins, payments, expenses, salaries, trainers, commissions, cashbook, receipts, reports, notifications, files, and audit.

Views coordinate HTTP concerns. Serializers validate API input/output. Services perform commands and transactions. Selectors build optimized reads. Models enforce local invariants, while database constraints enforce durable integrity.

Cross-domain financial writes are atomic and occur only after the pre-post approval gate. For example, approving a submitted membership cash payment may post the payment, create its receipt and cash transaction, settle the membership, and append an audit event in one transaction.

External work is emitted through a transactional outbox only when background delivery becomes necessary.

The member PT badge is a selector result, not client-owned member state: `is_pt_client` is derived from a same-gym `TrainerAssignment` with `status = ACTIVE` whose effective range contains the requested gym-local date. Ending or cancelling the assignment removes the badge without rewriting the member.

Membership-expiry email runs as two bounded, idempotent Django management commands scheduled on the VPS: one creates due `Notification` rows from each gym's local date and configured warning days, and one delivers pending email rows through the approved transactional-email provider. A notification is created only when expiry email is enabled and the member currently has a nonblank email address; missing email is a normal skip with no SMS/phone fallback. The delivery command resolves the current member email again, cancels safely if it was removed or the membership is no longer eligible, records provider outcomes, and retries transient failures with a strict attempt limit. A deterministic database uniqueness key prevents duplicate warning emails across repeated scheduler runs, retries, or restarts. This limited durable notification queue does not require Celery; Celery remains deferred until measured workload justifies it.

All persisted timestamps are UTC. Gym-local business dates, reporting cutoffs, receipt dates, and cash-day boundaries are derived using the gym's validated IANA timezone. Legacy `Asia/Rangoon` input is normalized to canonical `Asia/Yangon` before storage and comparison; both names must produce identical historical offset behavior. Clients may format local time but do not define business-day boundaries.

## 10. Financial architecture

- Monetary model fields use Django `DecimalField(max_digits=18, decimal_places=2)`, PostgreSQL `NUMERIC(18,2)`, and Python `Decimal` end to end. Binary floating point is prohibited for money.
- Each financial row stores a three-letter currency copied from the gym or source document. New gyms default to `THB`.
- A transaction cannot mix currencies.
- Drafts may be edited; submission freezes their business fields. Submitted payments, expenses, PT sales, and salary periods require approval by a different authorized Manager or Owner before posting. Posted payments, expenses, salary records, commissions, receipts, cash transactions, and cash closes are immutable. Even privileged users cannot update or delete posted economic facts.
- Corrections use `VOID`, `CANCEL`, `REFUND`, or reversing records linked to the original.
- Derived totals are calculated in backend services or database aggregates, never trusted from clients.
- Idempotency keys protect retryable financial commands.
- Every financial command verifies tenant scope, permission, amount sign and scale, currency, business date, source status, and idempotency key inside one `transaction.atomic()` boundary.
- Approval also verifies that the approver is not the creator or submitter. Rejection records actor, time, and reason but creates no financial side effects.
- Receipt numbers and other tenant sequences are allocated atomically per gym; gaps are retained and explained rather than silently reused.

### Atomic financial workflows

- **Finance submission:** an authorized creator with the module's create permission creates an editable draft and submits it. Submission freezes business fields and creates no receipt, settlement, commission, expense, or cash movement.
- **Cash payment approval:** a different authorized Manager or Owner locks the submitted payment, payable source, and relevant cash session; posts the immutable payment, allocates and creates its receipt, appends the cash-in transaction, updates source settlement state, and appends a tenant audit event. Commit all records together or none.
- **Non-cash payment approval:** posts the same payment, receipt, settlement update, and audit evidence but no cashbook entry unless cash physically moves.
- **Void/refund:** never overwrite the original. Validate refundable balance, create a linked void or refund/reversal record and corrective receipt document, append the opposite cash transaction when money leaves or enters a till, update derived settlement status, and audit the reason and actor atomically.
- **Expense:** an authorized creator submits a draft; approval by a different Manager or Owner posts the immutable expense, its evidence/reference, a cash-out transaction when paid from a till, and an audit event in one transaction. Rejection remains visible and has no cash effect; post-approval correction uses reversal.
- **Salary:** Reception or finance staff enters per-period salary lines; no fixed amount is inferred from a staff profile. Submission freezes the period. Approval by a different Manager or Owner atomically posts one protected `STAFF_SALARY` expense per line and any cash-out movements. PT commission remains a separate domain and category.
- **Cash close:** lock the cash session, calculate expected cash from its immutable opening amount and posted movements, record counted cash and variance, close the session once, and append audit evidence atomically. Reopening is not an edit; it requires an explicitly authorized correcting workflow.
- **PT sale and commission:** an authorized creator submits the PT-sale draft and optional payment; approval by a different Manager or Owner atomically posts the sale/payment/receipt/cash effects and derives one commission using the captured percentage. Commission payout later creates a protected `PT_COMMISSION` expense, a cash-out transaction when applicable, and an audit event; reversal preserves the original accrual and payout.

```
sequenceDiagram
    autonumber
    participant S as Financial service
    participant D as PostgreSQL
    S->>D: BEGIN transaction.atomic() + RLS context
    S->>D: Lock pending submission and verify different approver
    S->>D: Lock payable source and open cash session
    S->>D: Insert posted Payment (Decimal, THB by default)
    S->>D: Allocate sequence and insert Receipt
    S->>D: Append cash-in CashTransaction
    S->>D: Append tenant AuditEvent
    S->>D: COMMIT
    Note over D: Any failure rolls back payment, receipt, cash, and audit together
```

An accounting ledger can be added later without changing public payment and expense contracts; source records retain stable identifiers, explicit event history, and enough immutable linkage to support that evolution.

## 11. File architecture

MinIO exposes private S3-compatible buckets only on the VPS internal Docker network. The database stores tenant-scoped metadata, ownership, object key, content type, size, checksum, scan status, and lifecycle status.

Upload flow:

1. authorize an upload intent for the active gym;
2. issue a short-lived signed URL with size and type restrictions;
3. verify the resulting object and detected type;
4. quarantine/scan untrusted files;
5. attach only verified metadata to a domain record.

Download URLs are issued only after tenant and permission checks. Object paths are not authorization controls.

```
sequenceDiagram
    autonumber
    participant C as Client
    participant A as DRF API
    participant O as Private MinIO bucket
    participant D as PostgreSQL
    C->>A: Request upload intent + metadata
    A->>A: Authorize selected gym and constraints
    A-->>C: Short-lived presigned upload URL + object key
    C->>O: Upload through constrained object endpoint
    C->>A: Complete upload intent
    A->>O: Verify object, checksum, size, detected type
    A->>D: Mark scanned/verified and attach tenant metadata
    A-->>C: File reference
    Note over C,O: Downloads require a fresh authorized presigned URL
```

## 12. Deployment architecture

- Cloudflare provides DNS, edge TLS, WAF, bot/rate controls, and origin protection; the free tier is the budget baseline.
- One production VPS provides 4–8 vCPU, 8–16 GiB RAM, and 80–160 GiB NVMe storage, subject to measured load. It runs a pinned Linux release, host firewall, automatic security patches, and Docker Compose.
- Caddy is the only public origin service and exposes ports 80/443. SSH is key-only and restricted by firewall/admin source. PostgreSQL and MinIO publish no public host ports.
- Docker Compose runs separate non-root containers for Next.js, Django/Gunicorn, PostgreSQL, MinIO, and bounded monitoring. Containers communicate only over explicit private networks and use persistent named volumes.
- Docker secrets or root-readable encrypted environment files provide database credentials, JWT signing material, storage credentials, and provider secrets. Plaintext secrets never enter Git or image layers.
- GitHub Actions runs tests/scans and publishes immutable images to GitHub Container Registry (GHCR). A protected deployment pulls digest-pinned images, runs a one-shot migration service with the migration role, performs health checks, and switches/restarts services in a controlled order.
- Encrypted PostgreSQL base/logical backups, WAL archives, and MinIO object backups leave the VPS for an independent S3-compatible backup provider. A backup stored only on the VPS is not a backup.
- Production and staging never share database volumes or credentials. Routine staging runs locally or on an ephemeral temporary VPS; the production VPS does not host a permanently running staging stack within the approximately $80 monthly budget.

```
flowchart LR
    U[Users] --> CF[Cloudflare<br/>DNS, TLS, WAF]
    CF --> RP[Caddy<br/>reverse proxy]
    subgraph VPS [Single production VPS]
        RP --> WEB[Next.js web]
        RP --> API[Django DRF API]
        API --> SQL[(PostgreSQL<br/>forced RLS)]
        API --> OBJ[(Private MinIO)]
        API --> OBS[Local structured logs<br/>bounded metrics]
    end
    GH[GitHub Actions] --> REG[GHCR]
    REG --> DEP[Protected Compose deploy]
    DEP --> WEB
    DEP --> API
    SQL --> BAK[(Encrypted offsite backup)]
    OBJ --> BAK
```

VPS database capacity must satisfy:

```
web/API worker processes × maximum database connections per worker
+ migration/operations reserve
< PostgreSQL max_connections
```

The initial worker count, connection lifetime/pool behavior, memory limits, and PostgreSQL buffers are fixed from load-test evidence. Docker memory/CPU limits reserve capacity for PostgreSQL and backup/restore operations; containers must not compete until the host swaps or exhausts disk.

## 13. Reliability, backup, and recovery

- The single VPS has no automatic failover. Host, disk, datacenter, or network failure may cause downtime until the VPS is repaired or rebuilt.
- PostgreSQL takes a daily encrypted base/logical backup and continuously archives WAL offsite. MinIO objects are versioned where practical and synchronised to independent encrypted offsite storage.
- Backup credentials are write-limited where supported and are not the same credentials used by the production application. Retention and deletion protection are approved before launch.
- Infrastructure configuration, Compose files, migration files, and restore scripts are stored in version control; secrets and backup encryption keys are stored separately.
- External monitoring alerts on HTTPS availability, certificate expiry, disk/RAM pressure, PostgreSQL health, backup age, WAL archive failure, object-sync failure, and failed deployment.
- Quarterly restore drills rebuild an isolated temporary Compose stack or replacement VPS from immutable images and offsite backups.
- The planning targets remain RPO ≤ 15 minutes through WAL archiving and RTO ≤ 4 hours through a rehearsed rebuild/restore runbook. They are targets, not an HA guarantee, and must be proven by drills before production.
- Restore procedures verify tenant isolation policies, database roles, forced RLS, financial constraints, audit evidence, and object references before recovered service is exposed.

## 14. Observability

Every request carries a validated or generated request ID, which is also installed as transaction-local `app.request_id`. Structured container logs include safe identifiers, gym ID where authorized, route, status, latency, and actor ID, but no tokens or sensitive payloads. Logs use bounded rotation and selected security/availability events are sent off-host. Lightweight metrics and external checks cover API errors, latency, authentication failures, RLS/scoping denials, database connections and saturation, VPS disk/RAM/CPU, transaction failures, backup freshness, restore-test results, presigned URL failures, and suspicious authorization denials.

Audit logs are domain evidence, distinct from diagnostic application logs.

## 15. Scalability path

1. Optimize indexes, query plans, pagination, worker counts, container limits, disk use, and database connections on the current VPS.
2. Vertically resize the VPS when measured CPU, RAM, storage, or I/O headroom falls below the approved threshold.
3. Move PostgreSQL or object storage to a separate host/managed service before adding application replicas; preserve forced RLS, roles, backups, and same-gym integrity.
4. Add an application VPS and load balancer only after the database/storage boundary and session/file behavior support multiple nodes.
5. Add Redis only after measurements establish a cache, rate-limit, or distributed-lock need and define invalidation and failure behavior.
6. Add Celery only after measured workload and latency justify asynchronous notifications, exports, scans, or scheduled expiry work.
7. Add read replicas, pre-aggregates, partitioning, or a warehouse only after tenant scope and reconciliation are proven.
8. If scale or regulation requires, move selected gyms to dedicated databases behind the same repository/service interfaces.

No scaling step may weaken tenant isolation or change financial semantics silently.

Redis and Celery are not Phase 0 baseline dependencies. Until evidence justifies them, synchronous work stays bounded, scheduled maintenance uses host timers or one-shot Compose jobs, and any future worker must establish one gym’s transaction-local RLS context per unit

of work.

# Phase 0 Authoritative Database Design

This document is the Phase 0 schema contract. PostgreSQL is authoritative; Django models, migrations, services, APIs, tests, and fixtures must conform to it. Reports and dashboards are query/service outputs, never mutable report tables in Phase 0.

## 1. Non-negotiable conventions

- Every primary key is a server-generated UUID (`UUIDField`, preferably UUIDv7 when the selected library is fixed; otherwise UUIDv4). No business identifier is a primary key.
- Every tenant-owned table contains its own immutable `gym_id UUID NOT NULL`, even when the gym is derivable through another foreign key.
- Every tenant-owned table has `UNIQUE (gym_id, id)`. Every tenant-to-tenant relationship is a composite foreign key on `(gym_id, referenced_id)` to the parent's `(gym_id, id)`.
- Every table has `created_at TIMESTAMPTZ NOT NULL` and `updated_at TIMESTAMPTZ NOT NULL`. Append-only tables set both to the insertion instant and reject later updates. `created_by` is required where a human or authenticated service initiates the record and is nullable only for documented bootstrap/system jobs.
- Instants are stored as `TIMESTAMPTZ` in UTC. Business dates are stored as `DATE`, derived in the gym's IANA timezone. Gym timezone defaults to `Asia/Yangon`; normalize legacy `Asia/Rangoon` input to `Asia/Yangon`.
- Money is `NUMERIC(18,2)` / Django `DecimalField(max_digits=18, decimal_places=2)`. Application calculations use Python `Decimal`, never binary float. Currency is an ISO 4217 `CHAR(3)`, default `THB`.
- Unless a later legal requirement overrides it, every monetary result is quantized once at the currency quantum (`Decimal("0.01")`) with `ROUND_HALF_UP`. Persisted formula inputs and rounded outputs are retained.
- Percentage snapshots are `NUMERIC(7,4)` and constrained to `0 <= value <= 100`. Rates are calculated at higher precision and only the monetary result is quantized.
- Enums are database `CHECK` constraints (or PostgreSQL enums only if migrations can safely evolve them). Normalized text is generated/stored or maintained by the service and checked nonblank where required.
- All foreign keys declare deletion behavior. Tenant data is never physically cascaded from `Gym`.
- Posted finance is immutable: business fields cannot be updated or deleted. Corrections use `VOID`, `CANCEL`, `REFUND`, or explicit reversal rows and preserve the original.

Common tenant columns, included explicitly by this contract on every tenant table, are:

```
id UUID PRIMARY KEY
gym_id UUID NOT NULL REFERENCES tenants_gym(id) ON DELETE RESTRICT
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
UNIQUE (gym_id, id)
```

Where listed below, `created_by_membership_id` forms a composite FK with `gym_id` to `GymMembership`; `updated_by_membership_id` is used on mutable master data when accountability is required.

## 2. Scope, ownership, and RLS inventory

### Platform-global tables (no `gym_id`, no tenant RLS)

| Table | Purpose and access |
|---|---|
| `User` | Global login identity; authentication and platform administration only |
| `Gym` | Tenant root; platform administration and trusted tenant-resolution path |
| `Permission` | Stable immutable authorization catalog |
| `SystemRoleTemplate` | Immutable global role definitions used only for provisioning |
| `SystemRoleTemplatePermission` | Global template-to-permission mapping |
| `Feature` | Stable global feature catalog |
| `SubscriptionPlan` | Global commercial plan catalog |
| `SubscriptionPlanFeature` | Global plan-to-feature defaults |
| `RefreshSession` | Global control-plane token/session security state for explicit staff/member principals |
| `PlatformAuditLog` | Global append-only platform administration and security audit evidence |

Global tables are not made tenant-readable merely because they lack RLS. Runtime grants expose only the rows/columns required by authentication, provisioning, or platform administration.

### Tenant-owned tables (all require gym_id, forced RLS)

GymSettings, GymMembership, StaffInvitation, StaffInvitationRole, Role, RolePermission, GymMembershipRole, GymFeatureEntitlement, StaffProfile, Member, MembershipPlan, Membership, CheckIn, Payment, Receipt, ExpenseCategory, Expense, SalaryPeriod, StaffSalaryLine, TrainerAssignment, PTSale, PTCommission, CashSession, CashTransaction, TenantSequence, FileAsset, Notification, AuditLog, and IdempotencyRecord.

No table has mixed global/tenant scope. In particular, Role.gym_id is never nullable.

## 3. Role provisioning and authorization

Permission is a stable global catalog: id, unique immutable code, module, description, risk_level (LOW, MEDIUM, HIGH, CRITICAL), is_active, and timestamps. Permission codes are data migration constants, not tenant-editable records.

SystemRoleTemplate is global and immutable after release: id, unique immutable key, name, description, version INTEGER > 0, is_active, and timestamps. SystemRoleTemplatePermission has id, system_role_template_id, permission_id, timestamps, and UNIQUE (system_role_template_id, permission_id); both FKs use RESTRICT.

At gym provisioning, each active template is copied into a tenant Role; its permission mappings are copied into tenant RolePermission rows in the same transaction. Later template changes never silently change existing gyms. An explicit, audited upgrade command may diff and apply a newer template version.

The complete canonical stable permission catalog is below. Tenant permission codes intentionally have no tenant. prefix. Feature-entitlement codes remain a separate catalog and never substitute for permissions.

```
platform.dashboard.view
platform.gyms.view
platform.gyms.create
platform.gyms.update
platform.gyms.activate
platform.gyms.suspend
platform.owners.create
platform.subscriptions.manage
platform.settings.manage
platform.audit.view

gym.view
gym.update
members.view
members.create
members.update
members.archive
memberships.view
memberships.create
memberships.update
memberships.cancel
checkins.view
checkins.create
checkins.update
payments.view
payments.create
payments.update
payments.submit
payments.approve
payments.void
payments.refund
receipts.view
receipts.void
expenses.view
expenses.create
expenses.update
expenses.submit
expenses.approve
expenses.void
cashbook.view
cashbook.open
cashbook.reconcile
cashbook.adjust
pt.assignments.view
pt.assignments.manage
pt.sales.view
pt.sales.create
pt.sales.update
pt.sales.submit
pt.sales.approve
pt.sales.void
pt.commissions.view
pt.commissions.pay
salaries.view
salaries.create
salaries.update
salaries.submit
salaries.approve
salaries.void
reports.view
reports.financial
reports.export
staff.view
staff.manage
roles.view
roles.manage
settings.view
settings.manage
files.view
files.upload
files.delete
audit.view
notifications.view
notifications.manage
```

## 4. Entity definitions

All tenant entities below include the common tenant columns from section 1.

### User (global)

Fields: `id`, `email`, `normalized_email`, `password_hash`, `first_name`, `last_name`, `phone`, `is_active`, `is_platform_admin`, `mfa_enabled`, `mfa_secret_encrypted` (nullable), `last_login_at`, `password_changed_at`, `created_at`, `updated_at`.

Constraints: `UNIQUE (normalized_email)`; normalized email is nonblank; MFA secret is present iff MFA is enabled. This is the custom Django user model from migration one. It contains no gym profile, role, or tenant status. Deletion is normally anonymization plus deactivation; referenced

users are RESTRICT/SET NULL as specified, never cascaded.

## Gym (global tenant root)

Fields: id, unique immutable code, name, legal_name, status (PENDING, ACTIVE, SUSPENDED, CLOSED), email, phone, structured address fields, timezone, currency, subscription_plan_id, subscription_status (TRIAL, ACTIVE, PAST_DUE, CANCELLED), subscription_starts_at, subscription_ends_at, created_at, updated_at.

Constraints: normalized code and name are nonblank; timezone is a supported IANA zone; currency is supported and defaults to THB; subscription end follows start. subscription_plan_id is a global FK with RESTRICT. Gyms are never physically deleted after provisioning; closing or suspension preserves all data.

## GymSettings

GymSettings is the one-to-one, tenant-owned persistence target for /api/v1/settings/. Identity, lifecycle, timezone, currency, subscription, and address remain typed fields on global tenant root Gym; operational settings are not stored in an unbounded JSON blob.

Fields: common tenant columns; week_starts_on SMALLINT (0..6); allow_manual_member_numbers BOOLEAN; member_number_prefix VARCHAR(20); receipt_number_prefix VARCHAR(20); check_in_duplicate_window_seconds INTEGER; membership_expiry_warning_days SMALLINT[]; membership_expiry_email_enabled BOOLEAN NOT NULL DEFAULT TRUE; require_separate_finance_approval BOOLEAN NOT NULL DEFAULT TRUE; created_by_membership_id; updated_by_membership_id.

Constraints: UNIQUE (gym_id); prefixes contain only approved uppercase letters, digits, and hyphen and are bounded; duplicate window is 0..86400; warning-day values are unique and each is 0..365; actor fields use same-gym composite FKs. The initial warning-day default is [0, 3, 7, 30]. When expiry email is enabled, the scheduled expiry job creates email notifications only for members whose normalized Member.email is nonblank; absence of email is a normal skip and never falls back to phone/SMS. All payments, expenses, PT sales, and salary periods require approval by a different authorized Manager or Owner in Phase 0; the approval setting records that policy but cannot be disabled without a later approved policy migration and security review. Changes are audited. New settings require typed migrations and explicit API fields rather than arbitrary client-defined keys.

## GymMembership

Fields: common tenant columns; user_id UUID NOT NULL; status (INVITED, ACTIVE, SUSPENDED, REVOKED); is_owner BOOLEAN; authz_version INTEGER NOT NULL DEFAULT 1; joined_date DATE; invited_at, activated_at, revoked_at; created_by_membership_id nullable only for the first provisioning owner.

Constraints: UNIQUE (gym_id, user_id); multiple memberships may have is_owner = TRUE in the same gym; authz_version > 0; status timestamps must agree with status. user_id references global User ON DELETE RESTRICT. Changing gym_id or user_id is forbidden. Revocation is a state transition, not deletion. An active gym must always retain at least one ACTIVE owner. Every owner add, activate, suspend, revoke, or OWNER-role change locks the Gym row, revalidates this invariant in the same transaction, and is protected by a deferred database constraint trigger so concurrent requests cannot remove the last active owner. Indexes include (gym_id, status) and a partial (gym_id, user_id) WHERE is_owner AND status = 'ACTIVE' for owner lookup; this latter index is intentionally non-unique.

is_owner represents Owner login authority only. Phase 0 stores no equity percentage, capital contribution, profit-distribution ratio, or legal-ownership claim. A future share-accounting feature must add separate effective-dated ownership-allocation records and migrations rather than changing the meaning of is_owner or rewriting historical financial records.

## StaffInvitation

Fields: common tenant columns; email, normalized_email; token_hash; inviter_membership_id; expires_at; nullable accepted_at, accepted_by_user_id, revoked_at, revoked_by_membership_id, revocation_reason; status (PENDING, ACCEPTED, REVOKED, EXPIRED).

StaffInvitationRole fields: common tenant columns; staff_invitation_id; role_id. Constraints: UNIQUE (gym_id, staff_invitation_id, role_id) and composite FKs to both tenant parents. At least one role is required before an invitation can be issued; role rows become immutable once issued, preserving the invited-role snapshot used to create normalized GymMembershipRole rows on acceptance.

StaffInvitation constraints: token_hash is globally unique, stores only a cryptographic hash of a high-entropy single-use token, and cannot change; email is nonblank; inviter and revoker use composite same-gym membership FKs; expiry follows creation; exactly the timestamps required by status are present; acceptance is allowed once, before expiry, and atomically consumes the token while creating/linking the global user and activating the selected gym membership and roles. accepted_by_user_id is a global User FK with RESTRICT. Pending invitations have partial uniqueness on (gym_id, normalized_email) so concurrent active invitations cannot be issued for the same gym/email. Revocation/expiry/acceptance are terminal; records are retained for audit.

## Role, RolePermission, GymMembershipRole

Role fields: common tenant columns; key, name, description, is_active; nullable global source_template_id; nullable source_template_version; created_by_membership_id. Constraints: UNIQUE (gym_id, key) and active normalized-name uniqueness; source ID and version are both null or both present; source template FK uses RESTRICT.

RolePermission fields: common tenant columns; role_id; global permission_id; created_by_membership_id. Constraints: UNIQUE (gym_id, role_id, permission_id); composite FK (gym_id, role_id); global permission FK RESTRICT.

GymMembershipRole fields: common tenant columns; gym_membership_id; role_id; created_by_membership_id. Constraints: UNIQUE (gym_id, gym_membership_id, role_id) and composite FKs for both tenant parents.

Tenant roles are archived/deactivated, not deleted while assigned or referenced. Assignment changes increment `GymMembership.authz_version` transactionally.

## Feature entitlement catalogs

`Feature` fields: global `id`, unique immutable `code`, `name`, `description`, `created_at`, `updated_at`.

`SubscriptionPlan` fields: global `id`, unique immutable `code`, `name`, `status` (`ACTIVE`, `RETIRED`), `price`, `currency`, `billing_interval`, timestamps. `SubscriptionPlanFeature`: global `id`, `subscription_plan_id`, `feature_id`, `enabled`, nullable `limit_value INTEGER`, timestamps, and unique plan/feature pair.

`GymFeatureEntitlement` fields: common tenant columns; global `feature_id`; `enabled`; nullable `limit_value`; `source` (`PLAN`, `OVERRIDE`); nullable `effective_from`, `effective_until`; `created_by_membership_id`. Constraints: `UNIQUE (gym_id, feature_id)`; nonnegative limit; end after start. Effective entitlement is this tenant snapshot/override, not a live mutable join to plan defaults. Changes are audited.

## StaffProfile

Fields: common tenant columns; `gym_membership_id`; `staff_code`; `display_name`; `phone`; `joined_date`; `status` (`ACTIVE`, `ON_LEAVE`, `TERMINATED`, `ARCHIVED`); nullable `profile_photo_asset_id`; `default_commission_percent`; nullable `employment_title`, `terminated_date`, `archived_at`; `created_by_membership_id`, `updated_by_membership_id`.

Constraints: `UNIQUE (gym_id, staff_code)` and `UNIQUE (gym_id, gym_membership_id)`; commission percent in `0..100`; termination date is required only for terminated status and is not before joined date; all tenant FKs are composite. Archive instead of delete after operational use.

## Member

Fields: common tenant columns; `member_number`; `first_name`, `last_name`, `normalized_full_name`; nullable normalized `phone`, `email`, `gender`, `date_of_birth`; structured address; `join_date`; emergency contact name/phone; `notes`; nullable `profile_photo_asset_id`; nullable global `linked_user_id`; `authz_version INTEGER NOT NULL DEFAULT 1`; `status` (`ACTIVE`, `EXPIRED`, `SUSPENDED`, `CANCELLED`, `ARCHIVED`); nullable `archived_at`; `created_by_membership_id`, `updated_by_membership_id`.

Constraints: `UNIQUE (gym_id, member_number)`; partial `UNIQUE (gym_id, linked_user_id) WHERE linked_user_id IS NOT NULL`; `authz_version > 0`; names nonblank; date of birth is no later than `join_date`; archived status and timestamp agree. Composite file FK; global user FK `RESTRICT`. Linking/unlinking an identity, changing self-service authorization, or suspending/archiving the member increments `authz_version` transactionally and revokes affected sessions. Indexes: `(gym_id, status, member_number)`, `(gym_id, linked_user_id) WHERE linked_user_id IS NOT NULL`, `(gym_id, normalized_full_name)`, `(gym_id, phone)`, `(gym_id, email)`, `(gym_id, join_date DESC)`.

`is_pt_client` is a read-only selector/API value, not a persisted or client-writable `Member` field. It is true only when a `TrainerAssignment` with `status = ACTIVE` exists for the member on the gym-local as-of date (`effective_from <= as_of` and `effective_until IS NULL OR effective_until >= as_of`). Historical PT sales do not keep the badge true after the active assignment ends.

## MembershipPlan

Fields: common tenant columns; `code`, `name`, `normalized_name`, `description`; `duration_value INTEGER`; `duration_unit` (`DAY`, `WEEK`, `MONTH`, `YEAR`); `price`; `currency`; `is_active`; nullable `archived_at`; `created_by_membership_id`, `updated_by_membership_id`.

Constraints: `UNIQUE (gym_id, code)`; partial unique active `(gym_id, normalized_name)`; duration positive; price nonnegative; currency equals gym currency in Phase 0. Existing plans are archived, not deleted after sale.

## Membership

Fields: common tenant columns; `member_id`, `plan_id`; immutable snapshots `plan_code`, `plan_name`, `duration_value`, `duration_unit`; `start_date`, `expiry_date`; `list_price`, `discount_amount`, `final_amount`, `currency`; `payment_status` (`UNPAID`, `PARTIAL`, `PAID`, `PARTIALLY_REFUNDED`, `REFUNDED`); lifecycle `status` (`PENDING`, `ACTIVE`, `EXPIRED`, `SUSPENDED`, `CANCELLED`); nullable `cancelled_at`, `cancelled_by_membership_id`, `cancellation_reason`; `created_by_membership_id`.

Constraints: composite member, plan, creator, and canceller FKs; `expiry_date >= start_date`; all amounts nonnegative; `discount_amount <= list_price`; exact check `final_amount = list_price - discount_amount`; status and cancellation fields agree. Expiry is calculated once with calendar-aware arithmetic in the gym timezone. Only one active overlapping membership is allowed per member in Phase 0: an exclusion constraint on `(gym_id, member_id, daterange(start_date, expiry_date, '[]'))` where status is `PENDING` or `ACTIVE`.

## CheckIn

Fields: common tenant columns; `member_id`; nullable `membership_id`; `check_in_at`; nullable `check_out_at`; `method` (`MANUAL`, `QR`, `NFC`, `BIOMETRIC`); nullable `recorded_by_membership_id`; nullable `device_id`, `source_metadata JSONB`; nullable `idempotency_key`; `created_by_membership_id` may equal recorder.

Constraints: composite FKs to all tenant parents; checkout is after check-in; `UNIQUE (gym_id, idempotency_key)` when nonnull; source JSON is size-limited and contains no biometric template. Index `(gym_id, member_id, check_in_at DESC)`. Duplicate-window policy is enforced in the service under a member lock, not by unreliable timestamp rounding.

## Payment

Fields: common tenant columns; member_id; nullable membership_id, pt_sale_id; purpose (MEMBERSHIP, PT, PRODUCT, OTHER); entry_type (CHARGE, REFUND, REVERSAL); description; positive amount; currency; method (CASH, BANK_TRANSFER, CARD, MOBILE_PAYMENT, OTHER); business_date; received_at; nullable external reference_number; status (DRAFT, PENDING_APPROVAL, POSTED, REJECTED, VOID, CANCELLED); nullable original_payment_id, reversal_payment_id, reason, submitted_at, submitted_by_membership_id, approved_at, approved_by_membership_id, rejected_at, rejected_by_membership_id, rejection_reason, posted_at, voided_at, cancelled_at, cancelled_by_membership_id; created_by_membership_id, updated_by_membership_id; nullable posted_by_membership_id.

Constraints: all tenant FKs are composite; amount is greater than zero; currency equals gym currency; CHARGE has no original, while REFUND/REVERSAL require an original posted charge; an entry cannot reference itself; one reversal per original; cash method requires a generated cash transaction when posted. DRAFT is creator-editable; submission freezes business fields; PENDING_APPROVAL has submitter evidence; POSTED has a different approver/poster and approval evidence with posted_by_membership_id = approved_by_membership_id; REJECTED has rejection actor/time/reason and no receipt, settlement, or cash effect; an unsubmitted draft may become CANCELLED only with cancellation actor/time/reason. The creator or submitter cannot approve or reject the same record. A posted charge may be voided only by atomically creating a posted REVERSAL; customer refunds are posted REFUND rows. Sum of posted, non-void refund amounts for an original cannot exceed its posted charge amount. Raw card/PIN data is never stored.

## Receipt

Fields: common tenant columns; payment_id; receipt_number; sequence_key, sequence_value BIGINT, sequence_year; issued_at; immutable snapshots of gym code/name/address, member number/name, description, amount, currency, method, business date, and issuer name; status (ISSUED, VOID, REPLACED); nullable voided_at, voided_by_membership_id, void_reason, replacement_receipt_id, rendered_file_asset_id; created_by_membership_id.

Constraints: UNIQUE (gym_id, receipt_number) and UNIQUE (gym_id, sequence_key, sequence_value); one partial unique issued receipt per payment; receipt requires a posted payment; amount/currency equal payment; void/replacement fields agree with state; replacement points to a receipt in the same gym. Receipt numbers are never reused, changed, or deleted.

## ExpenseCategory and Expense

ExpenseCategory fields: common tenant columns; code, name, normalized_name, nullable immutable system_key; display_order; is_active; nullable archived_at; created_by_membership_id, updated_by_membership_id. Constraints: unique (gym_id, code), unique (gym_id, normalized_name), and nonnegative display order. Seed categories are copied into each gym; they are not mixed-scope rows.

The canonical protected seed catalog, copied idempotently at provisioning and backfilled per gym, is:

| system_key / code | Display name | Posting rule |
|---|---|---|
| RENT | Rent | Manual approved expense |
| UTILITIES | Electricity / water | Manual approved expense |
| INTERNET | Internet | Manual approved expense |
| STAFF_SALARY | Staff salary | Salary service only |
| PT_COMMISSION | PT commission | Commission payout service only |
| CLEANING_SUPPLIES | Cleaning supplies | Manual approved expense |
| MAINTENANCE | Maintenance / equipment repair | Manual approved expense |
| MARKETING | Marketing | Manual approved expense |
| WATER_DRINKS | Water / drinks | Manual approved expense |
| OTHER | Other expenses | Manual approved expense with description |

Protected system_key and code values are immutable and unique per gym. Protected categories cannot be deleted; display names may be localized and unused manual categories may be archived. Clients cannot manually post into STAFF_SALARY or PT_COMMISSION.

Expense fields: common tenant columns; category_id; business_date; description; positive amount; currency; payment_method; nullable receipt_file_asset_id; status (DRAFT, PENDING_APPROVAL, POSTED, REJECTED, VOID, CANCELLED); nullable reversal_expense_id, submitted_at, submitted_by_membership_id, approved_at, approved_by_membership_id, rejected_at, rejected_by_membership_id, rejection_reason, posted_at, voided_at, cancelled_at, cancelled_by_membership_id, reason; created_by_membership_id, updated_by_membership_id; nullable posted_by_membership_id.

Constraints: composite category/file/actor FKs; amount positive; currency equals gym currency; an unsubmitted draft may become CANCELLED only with cancellation actor/time/reason; submission freezes business fields; approval requires a different actor from creator/submitter, sets posted_by_membership_id = approved_by_membership_id, and atomically posts the expense; rejection requires a different actor, records actor/time/reason, and creates no cash movement. A posted expense is voided only with an equal and opposite linked reversal expense; one reversal per original. A posted cash expense atomically creates its cash transaction.

## SalaryPeriod and StaffSalaryLine

SalaryPeriod fields: common tenant columns; period_start, period_end; business_date; description; status (DRAFT, PENDING_APPROVAL, POSTED, REJECTED, CANCELLED, VOID); nullable submitted_at, submitted_by_membership_id, approved_at, approved_by_membership_id, rejected_at, rejected_by_membership_id, rejection_reason, posted_at, cancelled_at, cancelled_by_membership_id, voided_at, reason; created_by_membership_id, updated_by_membership_id; nullable posted_by_membership_id.

StaffSalaryLine fields: common tenant columns; salary_period_id; staff_profile_id; positive manually entered amount; currency; payment_method (CASH, BANK_TRANSFER, CARD, MOBILE_PAYMENT, OTHER); nullable external_reference, cash_session_id, expense_id; description; created_by_membership_id, updated_by_membership_id.

Constraints: periods use gym-local dates with period_end >= period_start; UNIQUE (gym_id, salary_period_id, staff_profile_id); all tenant FKs are composite; amount is positive and currency equals gym currency. Lines are editable only while their period is DRAFT; no fixed salary or PT amount is inferred from StaffProfile. An unsubmitted draft may become CANCELLED with actor/time/reason. Submission freezes the period and all lines. Approval requires a different actor from creator/submitter, sets posted_by_membership_id = approved_by_membership_id, and atomically posts one immutable Expense per line under protected STAFF_SALARY, plus a cash transaction for each cash line, then links expense_id. Rejection also requires a different actor and has no expense/cash effect. Voiding a posted period reverses every linked salary expense and cash movement while preserving the original period and lines.

## TrainerAssignment

Fields: common tenant columns; trainer_staff_profile_id; member_id; effective_from, nullable effective_until; status (ACTIVE, ENDED, CANCELLED); nullable ended_at, reason; created_by_membership_id.

Constraints: composite trainer/member/creator FKs; end is not before start; status fields agree; exclusion constraint prevents overlapping active date ranges for the same (gym_id, trainer_staff_profile_id, member_id). End/cancel instead of deleting an effective assignment.

## PTSale and PTCommission

PTSale fields: common tenant columns; member_id, trainer_staff_profile_id; service_date; description; gross_fee; currency; commission_percent_snapshot; status (DRAFT, PENDING_APPROVAL, POSTED, REJECTED, CANCELLED, VOID); nullable payment_id, submitted_at, submitted_by_membership_id, approved_at, approved_by_membership_id, rejected_at, rejected_by_membership_id, rejection_reason, posted_at, cancelled_at, cancelled_by_membership_id, reversal_pt_sale_id, reason; created_by_membership_id, updated_by_membership_id.

PTCommission fields: common tenant columns; pt_sale_id; trainer_staff_profile_id; gross_fee_snapshot, commission_percent_snapshot, commission_amount, gym_share, paid_amount, outstanding_amount, currency; status (ACCRUED, PARTIALLY_PAID, PAID, VOID); nullable paid_at, commission_expense_id, reversal_commission_id; created_by_membership_id.

Constraints: one commission per sale via UNIQUE (gym_id, pt_sale_id); all tenant FKs composite; gross fee positive; percent in 0..100; all money nonnegative; exact checks commission_amount = ROUND_HALF_UP(gross_fee_snapshot * percent / 100), gym_share = gross_fee_snapshot − commission_amount, and outstanding_amount = commission_amount − paid_amount; paid amount cannot exceed commission. An unsubmitted draft may become CANCELLED with actor/time/reason. Submission freezes sale and optional payment inputs. Approval requires a different actor from creator/submitter and atomically posts the sale, optional payment/receipt/cash effects, and exactly one commission. Rejection also requires a different actor and creates none of those effects. Paying commission locks the commission, posts the linked PT_COMMISSION expense, and updates paid/outstanding in one transaction. Voiding a posted sale requires reversal of payment and commission; paid commission cannot be voided until its payout expense is reversed.

## CashSession and CashTransaction

CashSession fields: common tenant columns; business_date; register_code; opening_cash; derived cash_sales, cash_expenses, manual_net, expected_closing; nullable actual_closing, difference, difference_reason, closed_at, closed_by_membership_id; status (OPEN, CLOSED, VOID); opened_by_membership_id; created_by_membership_id.

Constraints: money nonnegative except signed manual_net and difference; one partial unique open session per (gym_id, register_code); exact expected_closing = opening_cash + cash_sales − cash_expenses + manual_net; when closed, actual and difference are required and difference = actual_closing − expected_closing; nonzero difference requires a reason.

CashTransaction fields: common tenant columns; cash_session_id; direction (IN, OUT); source_type (PAYMENT, EXPENSE, COMMISSION, MANUAL, REVERSAL); nullable source_id; positive amount; currency; occurred_at; business_date; description; nullable reversal_transaction_id, reason; created_by_membership_id.

Constraints: composite session/actor/reversal FKs; amount positive; currency equals gym/session currency; source uniqueness (gym_id, source_type, source_id) when source is nonnull; one reversal per transaction. Cash transactions are append-only. Manual entries require a reason and elevated permission. Closed sessions accept no transactions; correction is a transaction in a new open session referencing the original, never reopening/editing a closed session.

## TenantSequence

Fields: common tenant columns; sequence_key (for example RECEIPT:2026); prefix; next_value BIGINT; padding SMALLINT; created_by_membership_id nullable only for provisioning.

Constraints: UNIQUE (gym_id, sequence_key); next value positive; padding in 1..18; prefix format validated. Receipt allocation locks this row with SELECT ... FOR UPDATE, consumes next_value, increments it, and inserts the receipt in the same transaction:

```
{gym.code}-{year}-{sequence:06d}
W1-2026-000001
```

Transaction rollback does not consume a number; once committed, a number is never reused, including after void. Do not use `MAX(receipt_number) + 1`.

## FileAsset

Fields: common tenant columns; opaque `object_key`; `original_name`; detected `content_type`; `byte_size BIGINT`; `sha256`; `purpose`; `scan_status` (`PENDING`, `CLEAN`, `REJECTED`, `ERROR`); nullable `owner_type`, `owner_id`; `uploaded_by_membership_id`; `retention_status` (`ACTIVE`, `QUARANTINED`, `DELETE_PENDING`, `DELETED`); nullable `deleted_at`.

Constraints: `UNIQUE (gym_id, object_key)`; byte size nonnegative; SHA-256 format valid; composite uploader FK; owner is an opaque association and must be service-validated under the same gym. Object keys include the gym UUID plus a random UUID so they remain globally collision-resistant without a non-tenant-leading database index. The bucket is private. Physical deletion follows retention policy; metadata remains when needed for audit/finance.

## Notification

Fields: common tenant columns; nullable global `recipient_user_id`; nullable tenant `recipient_member_id`; `type`; `channel` (`IN_APP`, `EMAIL`, `SMS`, `PUSH`); `template_key`, `template_version`; size-limited safe `payload JSONB`; `scheduled_at`; nullable `sent_at`, `failed_at`; `status` (`PENDING`, `SENT`, `FAILED`, `CANCELLED`); nullable `provider_id`; `deduplication_key`; `attempt_count`; `created_by_membership_id` nullable for system-generated notices.

Constraints: exactly one recipient is present; composite member FK; `UNIQUE (gym_id, channel, deduplication_key)`; nonnegative attempts; terminal timestamps agree with state. No secrets or unnecessary personal data in payload.

Membership-expiry email uses `type = MEMBERSHIP_EXPIRY`, `channel = EMAIL`, a versioned template, and the member as recipient. Before creation and again before delivery, the service verifies that the membership/member/gym are active as applicable, the configured warning day matches the gym-local date, expiry email is enabled, and `Member.email` remains nonblank. The destination address is resolved from the current normalized member email at send time; it is not accepted from a scheduled payload. The payload contains only member display name, snapshotted plan display name, expiry date, and gym contact/display fields required by the template. The deduplication key is deterministic per `(membership_id, expiry_date, warning_day, template_version)`, so retries or repeated scheduler runs cannot send the same warning twice. A member without email produces no `Notification` row and no failure alert.

## AuditLog

Fields: common tenant columns; nullable global `actor_user_id`; nullable tenant `actor_membership_id`, `impersonator_membership_id`; `action`, `module`, `target_type`, `target_id UUID`; redacted `old_values JSONB`, `new_values JSONB`; `request_id`; nullable `ip_address`, `user_agent`; `outcome` (`SUCCESS`, `DENIED`, `ERROR`); `occurred_at`.

Constraints: composite membership FKs; append-only trigger rejects `UPDATE` and `DELETE`; runtime grants permit insert/select only as authorized. Platform-only events belong in `PlatformAuditLog`, never in this tenant table with a nullable `gym_id`.

## RefreshSession (global)

Fields: `id`; `principal_type` (`STAFF`, `MEMBER`); `user_id`; immutable `selected_gym_id`; nullable `gym_membership_id`; nullable `member_id`; unique `token_hash`; `family_id UUID`; `issued_at`, `expires_at`; nullable `rotated_at`, `revoked_at`, `replaced_by_session_id`; `revocation_reason`; `ip_address`, `user_agent`, `device_id`; `created_at`, `updated_at`.

Constraints: exactly one selected principal context is present: `STAFF` requires `gym_membership_id IS NOT NULL AND member_id IS NULL`, while `MEMBER` requires `member_id IS NOT NULL AND gym_membership_id IS NULL`; expiry follows issue; revoked/rotated fields agree; replacement remains in the same family, user, principal type, and selected context; token plaintext is never stored. Composite FKs `(selected_gym_id, gym_membership_id)` and `(selected_gym_id, member_id)` protect physical gym selection. Creation, refresh, and validation additionally verify authoritatively that a staff membership's `user_id`, or a member's `linked_user_id`, equals session `user_id`, that the selected principal is active, and that its current authorization version matches the token. Indexes include `(user_id, revoked_at, expires_at)`, `(family_id)`, `(selected_gym_id, gym_membership_id)` where present, and `(selected_gym_id, member_id)` where present.

`RefreshSession` is global control-plane security state, not tenant business data. It is never queried through tenant repositories, reports, exports, or ordinary tenant APIs; only the authentication/session service receives narrow access. Sessions are revoked and then retention-purged; user deletion is `RESTRICT` until purge/anonymization.

## PlatformAuditLog (global)

Fields: `id`; `actor_user_id`; `action`; `module`; nullable `target_gym_id`, `target_type`, `target_id`; nullable `effective_user_id`, `effective_gym_id`; redacted `old_values JSONB`, `new_values JSONB`; `request_id`; nullable `ip_address`, `user_agent`; `outcome` (`SUCCESS`, `DENIED`, `ERROR`); `occurred_at`; `created_at`, `updated_at`.

Constraints: append-only trigger rejects `UPDATE` and `DELETE`; runtime grants allow insertion only through the platform audit service and bounded reads only with `platform.audit.view`; actor/effective user and gym FKs use `RESTRICT`; redaction and payload-size limits apply. Required indexes include `(occurred_at DESC)`, `(actor_user_id, occurred_at DESC)`, `(target_gym_id, occurred_at DESC)`, and `(request_id)`. This table has no tenant RLS and is separate from tenant `AuditLog`.

### IdempotencyRecord

Fields: common tenant columns; nullable `actor_membership_id`; `http_method`; canonical `route`; `idempotency_key`; `request_hash`; `state` (`IN_PROGRESS`, `COMPLETED`, `FAILED`); nullable `response_status`, encrypted/size-limited `response_body`, `resource_type`, `resource_id`; `locked_until`, `expires_at`; `created_by_membership_id` nullable for service actors.

Constraints: `UNIQUE (gym_id, actor_membership_id, http_method, route, idempotency_key)` with a documented sentinel actor for system requests; same key with a different request hash is rejected; expiry follows creation; completed response is replayed. Composite actor FK. Expired rows may be hard-deleted by a privileged maintenance role.

## 5. ERD

```
GLOBAL
User --< RefreshSession >-- (selected GymMembership XOR selected Member)
User --< PlatformAuditLog
Permission >--< SystemRoleTemplatePermission >-- SystemRoleTemplate
Feature >--< SubscriptionPlanFeature >-- SubscriptionPlan --< Gym

TENANT ROOT / AUTHORIZATION
Gym --< GymMembership >-- User
 |--1 GymSettings
 |          |--0..1 StaffProfile
 |          +--< GymMembershipRole >-- Role --< RolePermission >-- Permission
 |--< StaffInvitation --< StaffInvitationRole >-- Role
 +--< GymFeatureEntitlement >-- Feature

OPERATIONS
Gym --< Member --< Membership >-- MembershipPlan
 |       |         +--< Payment --< Receipt
 |       +--< CheckIn
 |       +--< TrainerAssignment >-- StaffProfile
 |       +--< PTSale >-- StaffProfile --< PTCommission
 |
 +--< Expense >-- ExpenseCategory
 +--< SalaryPeriod --< StaffSalaryLine >-- StaffProfile
 +--< CashSession --< CashTransaction
 +--< TenantSequence
 +--< FileAsset
 +--< Notification
 +--< AuditLog
 +--< IdempotencyRecord
```

Every tenant edge above is physically protected by a composite FK containing `gym_id`; the diagram omits those repeated columns for readability.

## 6. Cross-tenant integrity and RLS

Example composite protection:

```
ALTER TABLE memberships_membership
  ADD CONSTRAINT membership_member_same_gym_fk
  FOREIGN KEY (gym_id, member_id)
  REFERENCES members_member (gym_id, id)
  ON DELETE RESTRICT;
```

Explicit reversible RunSQL migrations are required where the Django version cannot express composite FKs, exclusion constraints, forced RLS, or immutability triggers.

The RLS helper is defined explicitly and reads only the canonical transaction-local setting:

```
CREATE OR REPLACE FUNCTION app_current_gym_id()
RETURNS uuid
LANGUAGE sql
STABLE
AS $$
  SELECT NULLIF(current_setting('app.gym_id', true), '')::uuid
$$;
```

The migration owner owns this function; the runtime role may execute but not replace it. A missing setting returns NULL and therefore matches no tenant row; malformed values are rejected rather than selecting a default.

For every tenant table listed in section 2, migrations execute:

```
ALTER TABLE <table> ENABLE ROW LEVEL SECURITY;
ALTER TABLE <table> FORCE ROW LEVEL SECURITY;

CREATE POLICY <table>_tenant_select ON <table>
  FOR SELECT USING (gym_id = app_current_gym_id());
CREATE POLICY <table>_tenant_insert ON <table>
  FOR INSERT WITH CHECK (gym_id = app_current_gym_id());
CREATE POLICY <table>_tenant_update ON <table>
  FOR UPDATE
  USING (gym_id = app_current_gym_id())
  WITH CHECK (gym_id = app_current_gym_id());
CREATE POLICY <table>_tenant_delete ON <table>
  FOR DELETE USING (gym_id = app_current_gym_id());
```

For staff, `app.gym_id` is established only after global user/session validation and authoritative selection of an active `GymMembership`. For members, global user/session validation first verifies the claimed selected `Member` identity through a narrowly approved bootstrap function/policy that can return only the member row whose `linked_user_id` equals the authenticated `app.user_id`; the transaction then sets `app.gym_id` before ordinary tenant queries and reloads the member under normal RLS. Absence or invalid context fails closed. Authorization and staff/member object scope still apply above RLS. Delete policies do not grant business deletion: table grants, state rules, FKs, and triggers restrict it.

RLS policy inventory:

- Standard read/write policy: mutable tenant master/operational tables (`GymSettings`, `GymMembership`, `StaffInvitation`, `StaffInvitationRole`, `Role`, joins, entitlement, staff, member, plan, membership, check-in, category, assignment, salary line while draft, file, notification, idempotency).
- Insert/select plus constrained transition procedures: `Payment`, `Receipt`, `Expense`, `SalaryPeriod`, `PTSale`, `PTCommission`, `CashSession`.
- Insert/select only for runtime: `CashTransaction`, `AuditLog`; maintenance deletion runs as a separately controlled owner role where retention permits.
- `TenantSequence` is select/update only through a `SECURITY DEFINER` allocator that validates caller gym and has a fixed `search_path`.

`RefreshSession` and `PlatformAuditLog` are global control-plane tables and are excluded from tenant RLS inventory. Their restricted grants and append-only/session invariants are checked separately in CI.

The application runtime role must not own tables, have `BYPASSRLS`, set arbitrary tenant context outside the trusted transaction wrapper, or call maintenance functions. CI compares the tenant inventory to database metadata and fails if any tenant table lacks `gym_id NOT NULL`, `UNIQUE (gym_id,id)`, enabled/forced RLS, complete `USING`/`WITH CHECK` policies, or tenant-leading indexes.

## 7. Index contract

Every FK has an index. Every tenant operational index starts with `gym_id`; global uniqueness never substitutes for tenant-leading access paths. Required initial indexes include:

- `(gym_id, status, created_at DESC)` on stateful entities;
- `(gym_id, normalized_email, status)` and `(gym_id, expires_at)` on staff invitations;
- `(gym_id, member_id, status)` on memberships/payments and `(gym_id, member_id, check_in_at DESC)` on check-ins;
- `(gym_id, business_date, method)` on payments and `(gym_id, business_date, status)` on expenses/cash sessions;
- `(gym_id, trainer_staff_profile_id, service_date)` on PT sales;
- `(gym_id, member_id, status, effective_from, effective_until)` on trainer assignments for the derived PT badge;
- `(gym_id, status, outstanding_amount)` on PT commissions;
- `(gym_id, status, period_start, period_end)` on salary periods and `(gym_id, salary_period_id, staff_profile_id)` on salary lines;
- `(gym_id, scheduled_at, status)` on notifications;
- `(gym_id, target_type, target_id, occurred_at DESC)` on audit logs;
- partial indexes for active/non-archived records and open cash sessions.

Additional trigram/full-text indexes require measured query behavior and `EXPLAIN (ANALYZE, BUFFERS)` evidence.

## 8. Transaction and concurrency invariants

- **Tenant context:** each request uses `transaction.atomic()` and transaction-local `app.gym_id`, `app.user_id`, and `app.request_id`; pooled connections must not retain tenant state.
- **Receipt allocation:** lock exactly one `TenantSequence` row, increment, insert receipt, then commit. A unique-constraint retry handles only serialization/deadlock errors, never generates via `MAX`.
- **Payments/refunds:** lock the original payment and affected membership/PT sale. Posting, refund totals, membership payment status, receipt, and cash transaction are one transaction. Idempotency is claimed before side effects. Concurrent refunds serialize on the original.
- **Finance approval:** submitting a draft freezes its business fields. Approval locks the pending record, revalidates source and cash-session state, proves the approver differs from creator/submitter, and performs all posting side effects once. Rejection records a reason and produces no receipt, settlement, commission, expense, or cash movement.

- **Cash:** lock the open `CashSession` before appending a transaction or closing. Recompute totals from non-void append-only transactions while locked; do not trust client totals. Closing and final totals commit together.
- **Memberships:** lock the member before create/cancel/renew. The exclusion constraint is the final guard against concurrent overlap. Derive payment status from posted charges minus posted refunds; do not accept it from clients.
- **PT commission:** lock PT sale and commission. Sale posting creates exactly one commission; payout posts its expense/cash transaction and updates totals atomically. Unique constraints make retries safe.
- **Salary:** lock the pending salary period and all lines. Approval creates exactly one `STAFF_SALARY` expense per line and corresponding cash movement where required, links every line, and posts the period atomically; any failure rolls back the whole period.
- **Authorization:** role/permission changes lock the membership and increment `authz_version` in the same transaction so existing tokens can be invalidated.
- **Member authorization:** identity linking, member suspension/archive, and member self-service authorization changes lock the member and increment `Member.authz_version` in the same transaction so existing member tokens can be invalidated.
- **Deadlocks:** acquire locks in stable order: `Gym`/sequence, member or original financial record, session, then dependent rows. Retry boundedly only for PostgreSQL serialization failures/deadlocks.

## 9. Deletion, immutability, and retention

| Record class | Rule |
|---|---|
| Gym | Never hard-delete after provisioning; suspend/close |
| User | Deactivate/anonymize eligible PII; preserve referenced evidence |
| Memberships, roles, staff, members, plans, categories, assignments | Archive/end/revoke after use; hard-delete only unused drafts/configuration with no references |
| Draft/pending finance and salary | Draft may be cancelled; pending may be rejected with reason; hard deletion is limited to privileged cleanup before submission and before any dependent record exists |
| Posted payment/expense/salary/PT/cash/receipt | No update/delete of business fields; void/refund/reverse with linked records, actor, reason, and time |
| Audit log | Append-only; no runtime update/delete |
| Staff invitation | Retain terminal acceptance/revocation/expiry evidence; plaintext token is never stored |
| Refresh/idempotency/notification | Retention purge after expiry and legal/operational window |
| File asset | Private object lifecycle; preserve metadata/checksum while referenced or legally required |

FK defaults are `ON DELETE RESTRICT`. `SET NULL` is allowed only for optional global actor attribution after approved anonymization. `CASCADE` is limited to unreferenced global provisioning-template joins and is not used for tenant operational or financial records. Immutability triggers protect posted/closed/issued rows from bulk SQL paths as well as application code.

Retention periods, tax receipt rules, privacy erasure, and statutory rounding exceptions must be confirmed for the operating jurisdiction before production; until then the more conservative retention and `ROUND_HALF_UP` rules above apply.

## 10. Migration and verification rules

- A dedicated migration owner applies schema, RLS, policies, triggers, and constraints before runtime rollout; the runtime role never owns objects.
- Provisioning creates Gym, owner membership, copied roles/permissions, feature entitlements, the canonical protected expense-category catalog, approval-policy default, and sequences in one idempotent transaction.
- Migrations containing composite FKs or RLS include reversible SQL and database-level tests.
- Large backfills are resumable, tenant-batched, and leave forced RLS intact or run through a separately audited maintenance role.
- Destructive changes use expand/migrate/contract deployment with lock-impact, rollback, and data-verification plans.
- Phase 0 tests exercise cross-gym FK rejection, RLS read/write isolation, missing-context fail-closed behavior, posted-row immutability, sequence concurrency, refund limits, cash close races, membership overlap, and commission payout races.

An outbox is intentionally deferred until measured asynchronous delivery/reliability need justifies it. If introduced, every outbox record carrying tenant work will be gym-owned, include `gym_id`, use composite tenant references, and be forced-RLS protected.

The exact integration credential/device persistence model is intentionally deferred to the Phase 3 approved integration design. No device credential table is authorized in Phase 0; any future integration credential or device record is gym-owned and forced-RLS protected.

# Phase 0 REST API Contract

This document is the authoritative public HTTP contract for the planned API. Phase 0 is documentation only: no endpoint or feature described here is implemented by this contract. Later implementation, OpenAPI schemas, serializers, generated client types, and contract tests must conform to it.

## 1. Contract principles

- Base path: `/api/v1/`.
- JSON field names use `snake_case` consistently with the Python API.
- Resource IDs are opaque UUID strings.
- Dates use `YYYY-MM-DD`; instants use RFC 3339 UTC.
- Monetary values are strings to preserve decimal precision:

```
{
  "amount": "100000.00",
  "currency": "THB"
}
```

- Clients never submit `gym_id` as an ownership field.
- Serializers define explicit read and write fields; Django models are not exposed automatically.
- Every endpoint uses a named input serializer and a separate named output serializer with explicit fields. `fields = "__all__"`, model-derived writable fields, and passing request dictionaries directly to model constructors are prohibited.
- Collection endpoints use allowlisted filters, stable ordering, and cursor pagination.
- Resource APIs return `404` for absent or cross-tenant objects.
- Breaking changes require a new API version.

Unless a command below says otherwise, these fields are always read-only and are rejected if supplied: `gym_id`, actor/creator/updater/submitter/approver fields, lifecycle, approval, and posting `status` fields, receipt/sequence numbers, calculated dates, derived member PT indicators, totals, balances, payment state, commission snapshots/calculations, cash expected totals, and cash differences. Server timestamps, IDs, audit metadata, and authorization versions are also read-only.

## 2. Standard envelopes

Single-resource success:

```
{
  "success": true,
  "data": {
    "id": "7df4f2d6-79d3-4fd1-b850-968142b1b6dd"
  },
  "message": "Success"
}
```

Collection success:

```
{
  "success": true,
  "data": [],
  "meta": {
    "next_cursor": null,
    "has_more": false
  },
  "message": "Success"
}
```

Error:

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "You do not have permission to perform this action.",
    "fields": {}
  },
  "request_id": "req_01..."
}
```

Internal exceptions and database details are never returned.

## 3. HTTP behavior

| Status | Meaning |
|---|---|
| 200 | Successful read or command |
| 201 | Resource created |
| 204 | Successful command with no body |
| 400 | Malformed request/query |
| 401 | Missing, invalid, or expired authentication |
| 403 | Authenticated but capability denied |
| 404 | Resource absent or inaccessible in active gym |
| 409 | State, uniqueness, or idempotency conflict |
| 412 | Optimistic concurrency failure |
| 422 | Semantically invalid input |
| 429 | Rate limit exceeded |
| 5xx | Server failure with safe message and request ID |

Validation codes are stable, for example `INVALID_DATE_RANGE`, `MEMBERSHIP_EXPIRED`, `CASH_DIFFERENCE_REASON_REQUIRED`, `APPROVAL_REQUIRED`, `INVALID_APPROVAL_STATE`, `SELF_APPROVAL_FORBIDDEN`, `PROTECTED_EXPENSE_CATEGORY`, and `CROSS_TENANT_REFERENCE`.

## 4. Authentication API

**Delivery: Phase 1A — foundation.**

```
POST /api/v1/auth/login/
POST /api/v1/auth/refresh/
POST /api/v1/auth/logout/
POST /api/v1/auth/logout-all/
GET  /api/v1/auth/me/
GET  /api/v1/auth/gyms/
POST /api/v1/auth/select-gym/
POST /api/v1/auth/password/forgot/
POST /api/v1/auth/password/reset/
POST /api/v1/auth/mfa/challenge/
```

Staff access tokens are short-lived JWTs. Refresh credentials are opaque, rotate on every use, and are stored only as hashes in server-side sessions. A staff access token selects exactly one `GymMembership`; it never represents the union of the user's gym memberships.

Required JOSE header:

```
{
  "typ": "JWT",
  "alg": "RS256",
  "kid": "signing-key-id"
}
```

`alg` is an example from the deployment allowlist, not a client choice. `alg=none`, an algorithm outside the configured allowlist, a missing/unknown `kid`, and key/algorithm confusion are rejected.

Required staff access claims:

```
{
  "iss": "gym-platform-api",
  "aud": "gym-platform",
  "sub": "user-uuid",
  "gym_id": "gym-uuid",
  "gym_membership_id": "membership-uuid",
  "session_id": "session-uuid",
  "authz_version": 4,
  "token_type": "access",
  "iat": 1786791000,
  "nbf": 1786791000,
  "exp": 1786791600,
  "jti": "token-uuid"
}
```

The configured issuer and audience are deployment values; the examples above are placeholders until Phase 1A configuration is approved. Permission lists are not embedded as authority. The backend evaluates the current selected membership, roles, permissions, object rules, and

entitlements.

Staff authentication field contracts:

- `POST /auth/login/` request: `email`, `password`, optional `mfa_code`, optional `device_name`. Response: `access_token` only for non-cookie/mobile-style clients or an established secure browser session, `access_expires_at`, rotating `refresh_token` only at the approved token boundary, `refresh_expires_at`, `requires_gym_selection`, and `available_gym_memberships[]` containing read-only `gym_membership_id`, `gym_id`, gym display fields, and membership `status`. Passwords and MFA secrets are never echoed.
- `POST /auth/refresh/` request: rotating `refresh_token` or the approved secure cookie. Response: a new access credential and replacement refresh credential/expiry. Reuse revokes the entire token family.
- `POST /auth/logout/` request: current refresh credential/session; response `204`. `logout-all` revokes all of the user's refresh sessions and increments/revokes authorization state as required.
- `GET /auth/me/` response: `user` (`id`, `email`, `first_name`, `last_name`), selected `gym` display fields, selected `gym_membership` (`id`, `status`, `is_owner`, `roles`), effective permission codes, effective feature codes/limits, and `authz_version`; all are read-only.
- `GET /auth/gyms/` returns only active memberships belonging to the authenticated user.
- `POST /auth/select-gym/` request: `gym_membership_id` only. Response: newly scoped access/refresh credentials and the selected context.

Gym selection accepts a membership identifier from the user's own allowed membership list, verifies it server-side, and issues a new scoped token. It does not trust an arbitrary gym ID.

## Staff token validation sequence

Protected staff requests fail closed in this order:

1. parse one canonical JWT and reject malformed tokens, duplicate claims, or unsupported critical JOSE headers;
2. require the configured explicit `alg` allowlist and `kid`, resolve the expected key, and verify the signature;
3. verify exact `iss` and allowed `aud`;
4. require `token_type=access`, nonblank `sub`, UUID `jti`, and sane `iat`;
5. verify `exp` and `nbf` using only the small configured clock-skew allowance;
6. require UUID `session_id`, `gym_id`, and `gym_membership_id`, plus positive integer `authz_version`;
7. begin the request database transaction and install transaction-local user ID and request ID;
8. load the global non-revoked, non-expired STAFF refresh session and active user identified by `session_id`/`sub`, and verify its selected gym-membership context matches the token;
9. verify the active gym and subscription;
10. install transaction-local gym context, then load exactly the selected active `GymMembership` under RLS and verify its `id`, `user_id`, and physical `gym_id` match every token/session claim;
11. compare the current membership `authz_version` exactly with the token claim;
12. resolve current roles and permission codes, then enforce endpoint permission, object conditions, and feature entitlement.

Failure through step 6 or invalid/revoked session/user returns `401`. An authenticated identity lacking current gym membership, permission, or entitlement returns `403`, except resource identifiers outside the selected tenant return `404`. No request header, URL parameter, cookie value, or body `gym_id` may replace token-selected context.

## Member token contract

Member authentication uses separate routes, audience policy, serializers, and token type. It maps a global user to exactly one gym-scoped `Member` record.

```
{
  "iss": "gym-platform-api",
  "aud": "gym-platform-member",
  "sub": "user-uuid",
  "gym_id": "gym-uuid",
  "member_id": "member-uuid",
  "session_id": "session-uuid",
  "authz_version": 3,
  "token_type": "member_access",
  "iat": 1786791000,
  "nbf": 1786791000,
  "exp": 1786791600,
  "jti": "token-uuid"
}
```

Member tokens use the same strict JOSE header/signature, explicit algorithm/`kid`, issuer, audience, time, session, user, gym, and authorization-version checks as staff tokens. They additionally require `token_type=member_access`, verify the selected member principal through the approved bootstrap path, then load the claimed `member_id` under the established gym RLS context and verify `Member.linked_user_id=sub`, active member status, exact gym match, and current `Member.authz_version`. Every member endpoint adds immutable self scope (`member_id` from the token); a path, query, or body identifier can never select another member. Staff and member sessions share the global `RefreshSession` control-plane store but have explicit, mutually exclusive `principal_type` and selected-context fields; their audiences and validation paths remain separate.

Member validation therefore runs the full sequence explicitly: parse canonical JWT; require allowlisted `alg` plus known `kid`; verify signature; verify exact member issuer/audience and `token_type`; enforce `exp`, `nbf`, `iat`, `jti`, and small clock skew; begin a transaction and install

transaction-local user/request context; verify the global active MEMBER session and user; use only the narrowly approved bootstrap function/policy to prove that the selected Member is linked to that user and derive its gym; verify the active gym; install transaction-local gym context; reload that one member under ordinary RLS; compare current Member.authz_version; and finally enforce gym plus member-self scope and endpoint entitlement. Member tokens do not select a staff GymMembership or inherit staff roles. Staff tokens always select exactly one GymMembership.

## 5. Platform administration — Phase 1B

All routes require platform permissions and produce audit events.

Canonical requirements are: dashboard platform.dashboard.view; gym list/detail platform.gyms.view; gym update platform.gyms.update; activate platform.gyms.activate; suspend platform.gyms.suspend; initial or additional owner creation through the protected owner workflow platform.owners.create; subscription reads/updates platform.subscriptions.manage; and platform audit reads platform.audit.view.

```
GET  /api/v1/admin/dashboard/
GET  /api/v1/admin/gyms/
POST /api/v1/admin/gyms/
GET  /api/v1/admin/gyms/{gym_id}/
PATCH /api/v1/admin/gyms/{gym_id}/
POST /api/v1/admin/gyms/{gym_id}/activate/
POST /api/v1/admin/gyms/{gym_id}/suspend/
POST /api/v1/admin/gyms/{gym_id}/owners/
GET  /api/v1/admin/subscriptions/
PATCH /api/v1/admin/subscriptions/{id}/
GET  /api/v1/admin/audit-logs/
```

Gym creation is a single transactional service operation that:

1. validates gym code, timezone, currency, and subscription;
2. creates the gym;
3. creates or safely links the owner identity;
4. creates the owner gym membership and role assignment;
5. seeds protected default roles, permissions, and the canonical expense categories (RENT, UTILITIES, INTERNET, STAFF_SALARY, PT_COMMISSION, CLEANING_SUPPLIES, MAINTENANCE, MARKETING, WATER_DRINKS, OTHER);
6. records an audit event.

No outbox is introduced now. If owner invitation delivery later becomes asynchronous, an outbox remains deferred until measured delivery/reliability need; if introduced, its tenant work records are gym-owned and RLS-protected as specified by the database contract.

POST /admin/gyms/ is the single gym-and-owner provisioning command.

- Request fields: gym (code, name, optional legal_name, email, phone, structured address, timezone, currency, subscription_plan_id, subscription_status, optional subscription dates) and owner (email, first_name, last_name, optional phone, optional send_invitation).
- Read-only/rejected: all IDs, both gym_id occurrences, gym/subscription lifecycle timestamps, gym status, owner user/membership IDs, membership status, is_owner, role IDs, permissions, feature entitlements, categories, sequences, actors, and audit fields.
- 201 response: explicit gym output, explicit safe owner user, owner gym_membership, seeded roles, effective feature_entitlements, and seeded expense_categories[] containing id, code, system_key, name, and is_active; no password, invitation secret, refresh credential, or internal provisioning state.
- Authorization: platform.gyms.create and platform.owners.create; both are required because this command provisions the gym and its initial owner, and neither is a normal tenant permission. The service validates owner identity/link policy and subscription compatibility, then atomically creates or links the identity, creates the required initial owner membership, assigns the OWNER role, seeds tenant configuration, and audits the actor. Idempotency-Key is required because provisioning is externally retryable. Additional owners may be added later without replacing the initial owner.

## 6. Tenant resource API

### Gym profile and settings — Phase 1B

```
GET   /api/v1/gym/
PATCH /api/v1/gym/
GET   /api/v1/settings/
PATCH /api/v1/settings/
```

Gym profile read/update requires gym.view/gym.update; settings read/update requires settings.view/settings.manage.

PATCH /settings/ accepts only typed operational settings: week_starts_on, allow_manual_member_numbers, member_number_prefix, receipt_number_prefix, check_in_duplicate_window_seconds, membership_expiry_warning_days, membership_expiry_email_enabled, and require_separate_finance_approval. The approval flag is read/write only for OWNER policy administration and defaults to true; Phase 0 requires every Payment, Expense, PT Sale, and Salary record to be approved by a different authorized Manager or Owner and does not permit disabling this control through the API. The response returns those fields plus read-only

gym_id, actors, and timestamps from the tenant-owned one-to-one GymSettings record. Gym identity, address, timezone, currency, lifecycle, and subscription remain fields of /gym/, not /settings/. Arbitrary client-defined keys and unbounded JSON settings are rejected.

## Dashboard — Phase 6

```
GET /api/v1/dashboard/?range=this_month
GET /api/v1/dashboard/?date_from=2026-08-01&date_to=2026-08-15
```

Dashboard totals are computed for the active gym and its timezone. Supported preset ranges are today, this_week, this_month, last_month, and this_year. Custom ranges are bounded.

Tenant dashboard access requires reports.view; financial measures additionally require reports.financial.

The response has stable operational and financial blocks. Users lacking reports.financial receive financial: null; totals never silently downgrade to partial financial data.


```
{
  "range": {"date_from": "2026-08-01", "date_to": "2026-08-31"},
  "operational": {
    "active_members": 128,
    "new_members": 14,
    "expired_memberships": 9,
    "check_ins": 642,
    "pending_approvals": 6
  },
  "financial": {
    "total_revenue": "850000.00",
    "membership_revenue": "600000.00",
    "pt_revenue": "200000.00",
    "other_revenue": "50000.00",
    "total_expenses": "310000.00",
    "expenses_by_category": [
      {"system_key": "RENT", "amount": "30000.00"},
      {"system_key": "UTILITIES", "amount": "15000.00"},
      {"system_key": "INTERNET", "amount": "5000.00"},
      {"system_key": "STAFF_SALARY", "amount": "160000.00"},
      {"system_key": "PT_COMMISSION", "amount": "80000.00"},
      {"system_key": "CLEANING_SUPPLIES", "amount": "5000.00"},
      {"system_key": "MAINTENANCE", "amount": "5000.00"},
      {"system_key": "MARKETING", "amount": "5000.00"},
      {"system_key": "WATER_DRINKS", "amount": "3000.00"},
      {"system_key": "OTHER", "amount": "2000.00"}
    ],
    "staff_salary_expense": "160000.00",
    "pt_commission_expense": "80000.00",
    "net_profit": "540000.00"
  }
}
```


All financial values use posted, non-void source records for the selected gym and business-date range. total_revenue is posted charges minus posted refunds/reversals; its membership, PT, and other components are mutually exclusive and reconcile to the total. total_expenses is posted expenses net of linked reversals and includes salary and PT commission payouts. expenses_by_category always returns all ten protected system_key values in canonical display order, using "0.00" for a category with no activity. net_profit = total_revenue − total_expenses. active_members is the end-of-range count with an eligible active membership; new_members uses Member.join_date; expired_memberships counts memberships whose expiry occurs in range; check_ins counts gym-local check-ins in range; and pending_approvals counts Payment, Expense, PT Sale, and SalaryPeriod records awaiting approval. Draft and rejected records never affect revenue, expenses, or profit. This is the tenant dashboard's owner-focused financial view; any delegated Manager or Accountant access still requires the documented report permissions.

## Staff, roles, and permissions — Phase 1B

```
GET  /api/v1/staff/
POST /api/v1/staff/invitations/
POST /api/v1/staff/invitations/accept/
GET  /api/v1/staff/{id}/
PATCH /api/v1/staff/{id}/
POST /api/v1/staff/{id}/activate/
POST /api/v1/staff/{id}/suspend/

GET  /api/v1/owners/
POST /api/v1/owners/invitations/
POST /api/v1/owners/{gym_membership_id}/revoke/

GET  /api/v1/roles/
POST /api/v1/roles/
GET  /api/v1/roles/{id}/
PATCH /api/v1/roles/{id}/
POST /api/v1/roles/{id}/archive/
GET  /api/v1/permissions/
PUT  /api/v1/roles/{id}/permissions/
GET  /api/v1/gym-memberships/{id}/roles/
PUT  /api/v1/gym-memberships/{id}/roles/
```

Protected role templates cannot be renamed or deleted. Assignment APIs validate role and staff membership belong to the current gym.

Staff listing/detail requires `staff.view`; invitation, staff lifecycle, and membership-role assignment require `staff.manage`; role and permission listing requires `roles.view`; role mutation and permission replacement require `roles.manage`.

The owner endpoints are a separate OWNER-reserved workflow and are not authorized by `staff.manage` or `roles.manage`. Any active Owner may list the gym's Owners and invite an additional Owner after recent MFA/step-up verification. The invitation accepts `email`, `first_name`, `last_name`, optional `phone`, and optional `send_invitation`; the server fixes `is_owner = true` and the protected OWNER role, so clients cannot submit role or permission IDs. Acceptance uses the single-use invitation contract and creates or activates another same-gym Owner login without replacing existing Owners. Revocation requires a different active Owner as actor, a reason, recent MFA, row locking, and proof that at least one other active Owner will remain. All owner changes increment authorization state and create audit events. Responses expose login/role status only; no share percentage, capital contribution, profit-distribution ratio, or legal-ownership value exists in Phase 0.

`POST /api/v1/admin/gyms/{gym_id}/owners/` provides the equivalent audited Super Admin assistance path for adding an Owner. It follows the same identity-link, invitation, tenant, role, last-active-owner, authorization-version, and audit rules and does not grant the Super Admin ordinary access to gym business data.

Staff invitation/acceptance and role contracts:

- Invite request: `email`, `first_name`, `last_name`, optional `phone`, `staff_code`, `display_name`, `joined_date`, optional employment fields, and nonempty `role_ids`. Read-only/rejected: `gym_id`, user/membership/profile IDs, invitation token, all statuses/timestamps, `is_owner`, `authz_version`, actors, effective permissions, and role details. Response contains safe invitation metadata and explicit membership/staff output. Requires `staff.manage`; callers may assign only roles and permissions they are authorized to delegate.
- Accept request: `invitation_token`, `password`, `password_confirmation`, and optional profile confirmation fields. The token is single-use, hashed at rest, scoped to the invited membership/gym, and expires. Response follows login/select-gym token boundaries. The command verifies invitation state, user binding, password policy, selected membership, and gym state before atomically activating the one invited `GymMembership`; clients cannot choose a gym or role.
- Role create/update request: `key`, `name`, optional `description`, and `is_active` where permitted. Permission replacement request: unique `permission_codes`. Read-only: `gym_id`, source template ID/version, protected/template status, actors, timestamps, and effective assignment count.
- Membership-role assignment request: unique `role_ids`; response returns the selected `gym_membership_id`, assigned explicit role outputs, and incremented read-only `authz_version`. It requires `staff.manage`, same-gym membership/roles, anti-escalation/delegation checks, owner-protection invariants, and atomically increments authorization version.

## Members — Phase 2

```
GET  /api/v1/members/
POST /api/v1/members/
GET  /api/v1/members/{id}/
PATCH /api/v1/members/{id}/
POST /api/v1/members/{id}/archive/
GET  /api/v1/members/{id}/summary/
GET  /api/v1/members/{id}/payments/
GET  /api/v1/members/{id}/check-ins/
```

Filters include allowlisted `status`, `search`, `joined_from`, `joined_to`, and `is_pt_client=true|false`. The API assigns member ownership from request context. `member_id` is unique only within the active gym.

Member create request fields: optional client-visible `member_number` only if policy allows manual numbering, `first_name`, `last_name`, optional `phone`, `email`, `gender`, `date_of_birth`, structured address, `join_date`, emergency contact fields, `notes`, and tenant-scoped `profile_photo_asset_id`. Read-only/rejected: `id`, `gym_id`, normalized fields, `linked_user_id`, `is_pt_client`, `status`, `archived_at`, actors, and timestamps. Response uses an explicit member serializer and includes those read-only server fields. `is_pt_client` is server-derived from a same-gym trainer assignment with `status = ACTIVE` whose effective date range contains the gym-local as-of date; it is returned by list/detail/summary and cannot be set directly. The service requires `members.create`, any required member-management feature, same-gym clean file asset, date validity, and per-gym uniqueness.

## Membership plans and memberships — Phase 2

```
GET  /api/v1/membership-plans/
POST /api/v1/membership-plans/
GET  /api/v1/membership-plans/{id}/
PATCH /api/v1/membership-plans/{id}/
POST /api/v1/membership-plans/{id}/archive/

GET  /api/v1/memberships/
POST /api/v1/memberships/
GET  /api/v1/memberships/{id}/
PATCH /api/v1/memberships/{id}/
POST /api/v1/memberships/{id}/cancel/
POST /api/v1/memberships/{id}/suspend/
POST /api/v1/memberships/{id}/resume/
GET  /api/v1/memberships/expiry/?bucket=within_7_days
```

The create request supplies member, plan, start date, and discount—not expiry or trusted totals. The backend calculates expiry and final amount and returns them. Membership-plan reads use `memberships.view`; plan creation uses `memberships.create`; plan update and archive use

memberships.update.

Membership create request fields: member_id, plan_id, start_date, optional discount_amount, and optional non-financial note. Read-only/rejected: gym_id, plan snapshots, expiry_date, list_price, final_amount, currency, payment_status, lifecycle status, cancellation data, and actors. Response explicitly returns IDs, immutable plan/duration snapshots, start/expiry dates, list/discount/final totals, currency, payment/lifecycle statuses, creator, and timestamps. The service requires memberships.create, membership feature entitlement, active same-gym member/plan, discount authority, calendar-aware expiry calculation in gym timezone, exact decimal totals, and no overlapping pending/active membership.

Expiry buckets are expired, today, within_3_days, within_7_days, and within_30_days.

Membership-expiry email is system-generated from the gym-local expiry schedule; there is no client endpoint that supplies a destination address or directly sends the warning. For each configured warning day, an active eligible membership with a nonblank current Member.email may create one deduplicated EMAIL notification. The email contains the member display name, plan name, expiry date, and safe gym contact/display fields. If the member has no email, the scheduler skips the warning without creating a notification, reporting an error, or falling back to SMS/phone. Repeated scheduler runs and retries use the deterministic deduplication contract from DATABASE.md.

## Check-ins — Phase 3

```
GET  /api/v1/check-ins/
POST /api/v1/check-ins/
GET  /api/v1/check-ins/{id}/
POST /api/v1/check-ins/{id}/check-out/
```

Manual check-in is implemented first. method remains extensible. The exact integration credential/device model is deferred to the Phase 3 approved integration design; any future integration credential or device is gym-owned and RLS-protected and uses a dedicated ingestion contract rather than staff tokens.

Manual check-in request fields: member_id, optional occurred_at only for callers with checkins.update, and optional note. method is fixed server-side to MANUAL. Read-only/rejected: gym_id, selected membership_id, check_in_at when not explicitly backdated, check_out_at, method override, recorder/actors, device/source metadata, eligibility result, and status. Response returns the check-in plus an explicit eligibility snapshot. Requires checkins.create, check-in entitlement, active same-gym member, an eligible active membership selected by the service, bounded backdating, and duplicate-window validation under lock. Check-out/correction requires checkins.update; cross-tenant member/membership identifiers return 404.

## Payments, sales, and receipts — Phase 4

```
GET  /api/v1/payments/
POST /api/v1/payments/
GET  /api/v1/payments/{id}/
PATCH /api/v1/payments/{id}/
POST /api/v1/payments/{id}/submit/
POST /api/v1/payments/{id}/approve/
POST /api/v1/payments/{id}/reject/
POST /api/v1/payments/{id}/cancel/
POST /api/v1/payments/{id}/void/
POST /api/v1/payments/{id}/refund/
GET  /api/v1/sales/daily/?date=2026-08-15

GET  /api/v1/receipts/
GET  /api/v1/receipts/{id}/
POST /api/v1/receipts/{id}/void/
GET  /api/v1/receipts/{id}/download/
```

Payment creation returns a DRAFT; it creates no receipt, cash movement, or membership settlement. The creator submits the draft and a different authorized Manager or Owner approves it. Approval is the posting boundary and atomically returns the immutable posted payment, generated receipt, source settlement update, and cash transaction where applicable. Clients cannot submit a receipt number. Submit, approve, reject, cancel, void, and refund commands require an Idempotency-Key header.

Payment create/update fields while DRAFT: member_id, exactly one supported source reference such as optional membership_id or pt_sale_id, purpose, description, positive decimal-string amount, currency, method, business_date, optional received_at, and optional reference_number. Read-only/rejected: gym_id, entry_type, status, submission/approval/posting/actor fields, settlement totals, original/reversal IDs, cash transaction ID, receipt ID/number/snapshots, and all timestamps not explicitly accepted. A successful 201 returns the editable draft. Submit has no writable business fields and freezes the draft. Approve accepts optional cash_session_id when the payment method is cash; reject requires rejection_reason; cancel requires reason and is valid only from DRAFT. Approval revalidates all source, amount, currency, and cash-session state before posting.

Void request fields: required reason, optional business_date, and optional cash_session_id. Refund request fields: positive decimal-string amount, reason, method, business_date, optional reference_number, and optional cash session. Original payment amount/status, refundable balance, entry type, reversal/refund IDs, receipt number, actors, and derived settlement state are read-only. Both responses include original and corrective payment entries, receipt documents, cash effects, and updated refundable/settlement totals.

Payment create/update/submit/approve/void/refund require payments.create/payments.update/payments.submit/payments.approve/payments.void/payments.refund respectively; reject requires payments.approve and cancel requires payments.update. Every action additionally requires finance entitlement, same-gym source/member/session, exact currency/scale, valid source and cash-session state, and Idempotency-Key on commands. Approval or rejection

requires an actor different from the creator and submitter; RECEPTIONIST never receives `payments.approve`. Posted facts are never edited. Refunds lock the original and cannot exceed authoritative refundable balance.

`GET /sales/daily/?date=` requires `reports.financial` and returns posted, non-void payments for one gym-local business date grouped by purpose and method, with membership/PT/other revenue, refunds/reversals, and net total. Pending, rejected, cancelled, and draft records are excluded. Before Phase 5, PT totals are `"0.00"`; after Phase 5 they use the same posted PT payments as dashboard/report selectors.

Receipt list/detail/download requires `receipts.view`. `POST /receipts/{id}/void/` requires `receipts.void`, a reason, same-gym issued receipt, finance entitlement, and `Idempotency-Key`; it never relies on `payments.void`.

## Expenses — Phase 4

```
GET  /api/v1/expense-categories/
POST /api/v1/expense-categories/
PATCH /api/v1/expense-categories/{id}/

GET  /api/v1/expenses/
POST /api/v1/expenses/
GET  /api/v1/expenses/{id}/
PATCH /api/v1/expenses/{id}/
POST /api/v1/expenses/{id}/submit/
POST /api/v1/expenses/{id}/approve/
POST /api/v1/expenses/{id}/reject/
POST /api/v1/expenses/{id}/cancel/
POST /api/v1/expenses/{id}/void/
```

Only drafts are editable. A receipt uses the file upload flow and a tenant-scoped file asset ID. Reception may create and submit drafts but cannot approve them.

Expense draft create/update fields: `category_id`, `business_date`, `description`, positive decimal-string `amount`, `currency`, `payment_method`, optional `receipt_file_asset_id`, and optional policy note. Read-only/rejected: `gym_id`, `status`, posting/void/cancel/reversal fields, actors, cash effects, and calculated totals. Only `DRAFT` is mutable.

Submit has no writable business fields and freezes the draft. Approve accepts optional `cash_session_id` when cash; reject requires `rejection_reason` and also requires `expenses.approve`; cancel requires `reason`, `expenses.update`, and `DRAFT`. Void requires `reason`, optional `business_date`, and optional destination/open cash session for the correcting movement. Statuses, approval/reversal amount/link, actors, differences, and cash transaction are server-owned. Approval/voiding returns the original plus immutable posting/reversal and cash effects. Commands require `expenses.create/update/submit/approve/void` as applicable, expense entitlement, same-gym category/file/session, clean file status, valid business date/currency, valid state transition, different creator/approver, and `Idempotency-Key` for submit, approve, reject, cancel, and void. Manual requests selecting protected `STAFF_SALARY` or `PT_COMMISSION` categories return 422 `PROTECTED_EXPENSE_CATEGORY`.

## Salaries — Phase 4

```
GET   /api/v1/salary-periods/
POST  /api/v1/salary-periods/
GET   /api/v1/salary-periods/{id}/
PATCH /api/v1/salary-periods/{id}/
PUT   /api/v1/salary-periods/{id}/lines/
POST  /api/v1/salary-periods/{id}/submit/
POST  /api/v1/salary-periods/{id}/approve/
POST  /api/v1/salary-periods/{id}/reject/
POST  /api/v1/salary-periods/{id}/cancel/
POST  /api/v1/salary-periods/{id}/void/
```

Salary period create/update accepts `period_start`, `period_end`, `business_date`, and `description`. Line replacement while draft accepts unique lines containing `staff_profile_id`, positive decimal-string `amount`, `currency`, `payment_method`, optional external reference, and description. Salary values are entered per period; no fixed amount is read from staff or PT configuration. Submission freezes the period and lines. Approval accepts cash-session mappings for cash-paid lines, requires a different authorized Manager or Owner, and atomically posts one protected `STAFF_SALARY` expense per line plus cash movements. Reject requires `rejection_reason` and has no financial effect. Cancel requires `reason` and is valid only from `DRAFT`. Void requires `reason` and reverses all linked expenses/movements.

Salary list/detail/create/update/submit/approve/void require `salaries.view/create/update/submit/approve/void` as applicable; reject requires `salaries.approve` and cancel requires `salaries.update`. Every action additionally requires salary entitlement, same-gym staff/session references, valid period and currency, separation of duties, and `Idempotency-Key` for submit, approve, reject, cancel, and void.

## Cash book — Phase 4

```
GET  /api/v1/cash-sessions/
POST /api/v1/cash-sessions/open/
GET  /api/v1/cash-sessions/{id}/
POST /api/v1/cash-sessions/{id}/adjustments/
POST /api/v1/cash-sessions/{id}/reconcile/
GET  /api/v1/cash-transactions/
```

The reconcile request supplies actual closing cash and an optional reason. Expected closing and difference are server-calculated. A non-zero difference without a reason is rejected.

Open request fields: `register_code`, `business_date`, and nonnegative decimal-string `opening_cash`. Reconcile fields: decimal-string `actual_closing` and optional `difference_reason`. Manual adjustment fields: `direction`, positive decimal-string `amount`, `currency`, `business_date`, `description`, and required `reason`. Read-only/rejected everywhere: `gym_id`, all status/actor fields, `cash_sales`, `cash_expenses`, `manual_net`, `expected_closing`, `difference`, closing timestamps, source type/link, and reversal IDs.

Open, reconcile, and adjustment responses return the explicit cash session plus created immutable cash transaction(s). They require `cashbook.open`, `cashbook.reconcile`, or elevated `cashbook.adjust`, the cashbook entitlement, same-gym register/session, correct currency/business day, open-session state, and `Idempotency-Key`. Reconcile locks the session and recomputes every total; a nonzero server-calculated difference requires a reason.

## Trainers, assignments, PT sales, and commissions — Phase 5

```
GET  /api/v1/trainers/
GET  /api/v1/trainers/{id}/assigned-members/
GET  /api/v1/trainer-assignments/
POST /api/v1/trainer-assignments/
GET  /api/v1/trainer-assignments/{id}/
POST /api/v1/trainer-assignments/{id}/end/
POST /api/v1/trainer-assignments/{id}/cancel/
GET  /api/v1/pt-sales/
POST /api/v1/pt-sales/
GET  /api/v1/pt-sales/{id}/
PATCH /api/v1/pt-sales/{id}/
POST /api/v1/pt-sales/{id}/submit/
POST /api/v1/pt-sales/{id}/approve/
POST /api/v1/pt-sales/{id}/reject/
POST /api/v1/pt-sales/{id}/cancel/
POST /api/v1/pt-sales/{id}/void/
GET  /api/v1/pt-commissions/
GET  /api/v1/pt-commissions/{id}/
POST /api/v1/pt-commissions/{id}/record-payment/
```

Commission values are calculated from the trainer percentage snapshot. A trainer's queryset is additionally scoped to their own assignments and commission data.

Trainer and assignment list/detail routes require `pt.assignments.view`. Trainer assignment create/end/cancel requires `pt.assignments.manage`, PT entitlement, same-gym active trainer/member, valid dates, and no overlapping assignment. Create fields are `trainer_staff_profile_id`, `member_id`, `effective_from`, optional `effective_until`, and optional `reason`; end/cancel fields are `effective_until` where applicable and required `reason`. Read-only/rejected fields are `gym_id`, status, ended timestamp, actors, and assignment-derived member/trainer details.

PT sale create/update fields while draft: `member_id`, `trainer_staff_profile_id`, `service_date`, `description`, positive decimal-string `gross_fee`, `currency`, and optional payment draft using the payment-create writable fields. Read-only/rejected: `gym_id`, sale/approval status, payment/receipt IDs and numbers, all commission percentage snapshots and amounts, gym share, paid/outstanding totals, actors, and posting timestamps. Create returns a draft. Submit freezes the sale and optional payment. Approve accepts cash-session data where required and atomically returns the posted sale, payment/receipt, and generated commission accrual. Reject requires `rejection_reason` and creates no payment, receipt, commission, or cash movement. Cancel requires `reason` and is valid only from `DRAFT`.

Commission payment request fields: positive decimal-string `amount`, `business_date`, `payment_method`, optional `cash_session_id`, optional external reference, and required/optional policy note. Read-only/rejected: commission basis, percentage snapshot, `commission_amount`, `gym_share`, prior/new `paid_amount`, `outstanding_amount`, status, expense/reversal IDs, actors, and timestamps. Response returns the commission before/after summary and linked immutable expense/cash movement.

PT sale list/detail, create/update, submit, approve, and void require `pt.sales.view`, `pt.sales.create`/`pt.sales.update`, `pt.sales.submit`, `pt.sales.approve`, and `pt.sales.void` respectively; reject requires `pt.sales.approve` and cancel requires `pt.sales.update`. Approval or rejection requires a different actor from creator/submitter; RECEPTIONIST cannot approve. Commission list/detail requires `pt.commissions.view`; commission payment requires `pt.commissions.pay`. Every financial command requires `Idempotency-Key`, PT feature entitlement, same-gym references, trainer assignment/object-scope rules, valid states, exact decimal/currency checks, and authoritative calculation under lock.

## Reports and dashboard — Phase 6

```
GET /api/v1/reports/daily/?date=2026-08-15
GET /api/v1/reports/monthly/?year=2026&month=8
GET /api/v1/reports/cash-book/?date_from=...&date_to=...
GET /api/v1/reports/profit-loss/?date_from=...&date_to=...
POST /api/v1/reports/exports/
GET /api/v1/reports/exports/{id}/
```

Financial reports require `reports.financial`. Export creation/download additionally requires `reports.export`. Export jobs are introduced only when synchronous limits are exceeded. Every selector begins with active-gym scope.

Report request contracts are read-only query serializers:

- daily: required `date`;

- monthly: required `year` and `month`;
- cash book and profit/loss: required `date_from`, `date_to`, optional allowlisted grouping/filter values;
- export create: `report_type`, the corresponding validated bounded parameters, and `format` from an allowlist.

Responses contain read-only `gym` display identity, gym timezone/currency, normalized date range, generation instant, source-status policy, rows, and server-calculated totals/subtotals. No client total, formula, gym selector, actor, or source status override is accepted. Operational reports require `reports.view`; financial totals and exports require `reports.financial`; each also checks its report/export entitlement and configured row/date limits.

### Signed files — Phases 2-7 by owning feature

```
POST /api/v1/files/upload-intents/
POST /api/v1/files/{id}/complete/
GET  /api/v1/files/{id}/download-url/
DELETE /api/v1/files/{id}/
```

Signed URLs are short-lived. Complete verifies object metadata. File IDs from another gym return `404`.

Upload-intent request fields: `original_name`, declared `content_type`, `byte_size`, `sha256`, `purpose`, and optional intended `owner_type`/`owner_id`. Read-only/rejected: `gym_id`, `object_key`, signed URL fields, scan/retention status, detected metadata, uploader/actors, and timestamps. Response returns `file_id`, constrained signed upload `url`, required headers, and `expires_at`; object credentials and bucket-wide access are never returned.

Complete request fields: `sha256` and `byte_size` confirmation plus any provider upload token required by the intent. Response returns explicit file metadata and authoritative `scan_status`; it does not claim attachment eligibility until clean/verified. Download response returns a short-lived signed `url`, expiry, and safe filename/content type. Intent/complete require `files.upload`, download requires `files.view`, and delete requires `files.delete`; each also checks owning-feature entitlement, same-gym owner/object, purpose-specific size/type limits, clean/retention state, and object conditions. Completion verifies actual object size, checksum, and detected type; client metadata is never authoritative.

### Audit and notifications — Phases 1B and 7

```
GET  /api/v1/audit-logs/
GET  /api/v1/notifications/
POST /api/v1/notifications/{id}/read/
```

Audit reads require `audit.view`. Notification list/read requires `notifications.view`; future notification configuration or lifecycle commands require `notifications.manage`. Audit records have no update/delete endpoint.

## 7. Member mobile API — Phase 7

Member routes enforce gym scope and self scope:

```
POST /api/v1/member-auth/login/
POST /api/v1/member-auth/refresh/
POST /api/v1/member-auth/logout/
GET  /api/v1/member/me/
PATCH /api/v1/member/me/
GET  /api/v1/member/dashboard/
GET  /api/v1/member/membership/
GET  /api/v1/member/payments/
GET  /api/v1/member/check-ins/
GET  /api/v1/member/qr-token/
GET  /api/v1/member/notifications/
POST /api/v1/member/notifications/{id}/read/
```

The QR endpoint returns a short-lived signed or opaque token. It does not encode personal information or authoritative permissions.

Member auth request/response fields mirror the staff rotation contract but use member-specific serializers, audience, token types, sessions, and rate limits. Every response resource is selected from token `gym_id` plus token `member_id`; endpoints do not accept another member selector. Profile update accepts only the allowlisted self-service contact/profile fields and never `gym_id`, `member_id`, `linked_user_id`, status, membership/payment totals, or actors.

## 8. Pagination, filtering, and concurrency

Collection parameters:

```
limit=25            # default; maximum 100
after=<opaque>      # cursor
ordering=-created_at # allowlisted values only
```

Cursors encode a stable tuple such as `(created_at, id)`. Invalid cursors return `400`.

`after` is opaque, signed or integrity-protected, and bound to route, selected gym, normalized filters, and ordering. Clients must not construct it. The server returns deterministic tie-break ordering and never exposes raw database offsets or cross-tenant cursor state.

Mutable resources may expose a `version` and `ETag`. Sensitive updates require `If-Match`; stale writes return `412`.

Bulk endpoints are not created until their atomicity, per-item authorization, limits, and response behavior are specified.

## 9. Idempotency

`Idempotency-Key` is required for every financial command, including payment submit/approve/reject/cancel/void/refund, expense submit/approve/reject/cancel/void, salary submit/approve/reject/cancel/void, cash open/adjust/reconcile, PT sale submit/approve/reject/cancel/void, commission payment/reversal, and every receipt-affecting command. It is also required for gym provisioning.

The server keys records by gym, actor, method, route, and key. Repeating the same request returns the original result. Reusing a key with a different request hash returns `409 IDEMPOTENCY_KEY_REUSED`.

Keys are high-entropy client-generated strings with a documented length limit. The canonical request hash includes semantically relevant body and query fields. Concurrent duplicates wait for or safely report the in-progress operation; completed responses replay the original status/body. Validation that can affect side effects, idempotency claim, locks, writes, receipt allocation, and audit append occur in one transaction.

## 10. API security requirements

- Strict serializer field allowlists prevent mass assignment.
- Relationship fields use tenant-scoped querysets.
- Viewsets scope list and object retrieval; DRF object permission alone is insufficient for lists.
- Custom actions call object permission checks.
- Rate limits distinguish authentication, ordinary reads, writes, exports, and administrative actions.
- CORS uses explicit production origins.
- Cookie-based browser mutations require CSRF protection.
- Query complexity, date ranges, page sizes, uploads, and exports are bounded.
- API schemas and generated TypeScript types are versioned in CI.

### Permission and feature-entitlement evaluation

Authentication alone never grants a domain action. For every protected endpoint, the API:

1. completes the token/session/user/gym/selected-membership validation sequence;
2. resolves current active roles and stable permission codes from the selected membership;
3. checks the endpoint action permission;
4. loads relationships through gym-scoped serializer querysets/selectors and returns `404` for cross-tenant identifiers;
5. enforces object conditions, including trainer assignment and member self scope;
6. resolves the effective `GymFeatureEntitlement`, effective dates, enabled state, and applicable usage limit;
7. validates command state and business invariants inside the transaction immediately before mutation.

A valid same-tenant target with a missing permission or disabled/limited feature returns `403` with a stable safe code such as `PERMISSION_DENIED`, `FEATURE_NOT_ENTITLED`, or `FEATURE_LIMIT_REACHED`. The response must not disclose permissions, roles, subscription details, or foreign-tenant existence beyond what the caller may read.

## 11. Delivery map

- **Phase 1A — foundation:** envelopes, errors, request IDs, cursor pagination, staff JWT/refresh/session contracts, selected tenant context, strict serializers, permission/entitlement framework, and tenant-isolation behavior.
- **Phase 1B — Super Admin, gym, staff, roles, settings:** platform administration, transactional gym/owner provisioning, staff invitation/acceptance, role and permission management, membership-role assignment, gym profile/settings, audit reads, and foundational files.
- **Phase 2 — membership:** members, membership plans, memberships, expiry, derived PT-client badge/filter, member summaries/history, and membership-owned files.
- **Phase 3 — check-in:** manual check-in/check-out and eligibility/duplicate rules.
- **Phase 4 — finance:** payment and expense draft/manager-approval workflows, receipts, void/refund, salary periods/lines, cash sessions/transactions/reconciliation, and signed financial evidence files.
- **Phase 5 — PT:** trainer reads, trainer assignment writes/lifecycle, PT sale draft/manager approval, commission accruals, and commission payments.
- **Phase 6 — reporting:** explicit owner dashboard metrics, daily/monthly/cash-book/profit-loss reports, and bounded exports.
- **Phase 7 — mobile:** member authentication, strict tenant+self API, QR token, member notifications, and mobile file/profile boundaries.
- **Phase 8 — hardening and deployment:** compatibility/schema gates, attack and isolation tests, rate/load limits, observability, key rotation, file security, deployment checks, and production operational controls. Phase 8 adds no undocumented business API.

A delivery annotation never means implementation exists. Each phase remains unimplemented until its approved development work, migrations, tests, security review, and schema publication are complete.

# Security Architecture

## 1. Security objectives

The highest-priority property is tenant confidentiality and integrity: a gym must not read, modify, delete, reference, aggregate, infer, or download another gym's data.

Additional objectives:

- least privilege for platform, staff, members, services, and database roles;
- durable and accurate financial records;
- secure authentication and session lifecycle;
- private file handling;
- traceability of privileged and financial actions;
- safe failures without internal information disclosure.

The backend and database enforce security. Frontend controls are presentation only.

## 2. Threat model

Relevant threats include:

- IDOR and cross-tenant identifiers;
- forged or modified JWTs;
- stale privileges after role changes;
- privilege escalation through custom roles or mass assignment;
- unscoped list, search, aggregate, export, and relationship queries;
- pooled database connection tenant-context leakage;
- SQL injection and unsafe raw SQL;
- XSS, CSRF, credential theft, and brute force;
- signed URL reuse and cross-tenant file access;
- replayed payment or reconciliation requests;
- malicious uploads;
- accidental logging of secrets or personal data;
- excessive platform administrator access;
- compromised dependencies, containers, or CI credentials.

## 3. Layered tenant isolation

### Layer 1: authentication

JWT validation uses an explicit algorithm allowlist, issuer, audience, expiry, not-before, token type, signature key ID, and session state. Invalid context never falls back to an anonymous or default gym.

### Layer 2: principal-to-gym relationship

A staff access token identifies one active `GymMembership`; the API verifies that membership belongs to the user and gym and that the user, membership, gym, and subscription are active. A member access token identifies one `Member` linked directly to the authenticated global `User` by `Member.linked_user_id`; member principals do not derive gym context from `GymMembership`. The API verifies the selected member, linked user, member status, gym, session principal type, and authorization version.

### Layer 3: Django query scoping

Every tenant selector and view starts from `filter(gym=request.gym)`. Object lookup uses the scoped queryset. Serializer relationships are scoped to the same gym. A cross-tenant ID normally produces `404`.

A reusable `TenantQuerySet` may improve consistency, but no implicit manager may hide security-critical behavior. Explicit request scope remains visible at API boundaries.

### Layer 4: database constraints

Non-null `gym_id`, composite unique constraints, and composite foreign keys prevent a child in Gym A from referencing a parent in Gym B. The API cannot update `gym_id`.

### Layer 5: PostgreSQL RLS

All tenant-owned tables enable and force RLS. Policies have `USING` and `WITH CHECK`. Missing context matches no tenant rows. The runtime role is not a table owner and has neither superuser nor `BYPASSRLS`.

### Layer 6: role and object permissions

Stable permission codes grant actions. Object rules further constrain trainer assignments, member self-service, record states, and high-risk financial commands.

### Layer 7: automated tests and deployment checks

Every resource receives two-gym CRUD, filter, aggregate, relationship, and file tests. CI inspects RLS policy and runtime role configuration against real PostgreSQL.

### Two mandatory, independent controls

Explicit Django tenant scoping and forced PostgreSQL RLS are independent mandatory controls. Every tenant API path, command, worker, report, export, and aggregate must pass both controls. Passing one control never compensates for a missing or disabled other control, and neither may be treated as a temporary substitute for the other.

## 4. PostgreSQL context safety

All tenant work runs inside `transaction.atomic()`. After entering that transaction and before the first tenant query, the application sets parameterized, transaction-local context:

```
SELECT set_config('app.gym_id', :gym_id, true);
SELECT set_config('app.user_id', :user_id, true);
SELECT set_config('app.request_id', :request_id, true);
```

Rules:

- assert an active transaction before setting context;
- perform every tenant read and write, including authorization lookup, aggregate, file-metadata lookup, and audit write, only after `set_config(..., true)` in that same `transaction.atomic()` scope;
- never use session-level SET or `set_config(..., false)`;
- do not run tenant queries in middleware before context exists;
- do not perform tenant queries from streaming iterators after transaction completion;
- initialize context separately in workers, scripts, commands, and tests;
- process cross-gym jobs as one gym transaction at a time;
- clear behavior is verified across commit, rollback, exceptions, nested transactions, and reused pooled connections.

Staff context is derived from an authoritative active `GymMembership` after global identity/session validation. Member context is bootstrapped differently: after global identity/session validation, an approved narrowly scoped database function/policy may read only the selected `Member` whose `linked_user_id` equals the authenticated `app.user_id`, derive that member's physical `gym_id`, and return no ordinary tenant business data. The transaction then sets `app.gym_id`, reloads the member through normal forced RLS, and applies both gym scope and immutable member-self scope. No general tenant query, authorization join, report, or file lookup may use the bootstrap path.

Platform administration cannot use the normal tenant role as an unrestricted bypass. Cross-platform aggregates use reviewed, narrowly scoped database functions/views or a dedicated service path with explicit platform permission and audit.

## 5. Authentication and sessions

### Passwords and MFA

- Use Django's supported password hashing with a production cost setting.
- Enforce minimum length and compromised-password checks.
- Do not impose composition rules that encourage predictable passwords.
- Require MFA for super administrators and owners before production; support policy-based MFA for financial roles.
- Login and recovery responses do not reveal whether an account exists.

### JWT

- Prefer asymmetric signing with private keys supplied through Docker secrets or a root-readable encrypted host file (`0600`) that is never copied into an image or repository.
- Access lifetime target: 5–15 minutes.
- Refresh tokens are opaque or signed but always represented by a hashed server session.
- Refresh tokens rotate on every use; reuse revokes the family.
- Validate `iss`, `aud`, `alg`, `kid`, `exp`, `nbf`, `sub`, token type, session, gym membership, and authorization version.

- Never accept `alg=none`, algorithm confusion, or user-supplied verification keys.
- Do not log tokens.

### Browser storage

Use `HttpOnly`, `Secure`, suitably scoped and `SameSite` cookies. Cookie-authenticated mutations require CSRF tokens and Origin/Referer validation. Do not put authentication tokens in `localStorage`.

### Mobile storage

Store refresh credentials in iOS Keychain/Android Keystore-backed secure storage. Keep access tokens in memory where practical. Clear local data on logout or session revocation.

### Revocation

Password reset, user suspension, staff gym-membership suspension, role changes, member suspension/archive or identity-link changes, suspicious refresh reuse, and explicit logout revoke sessions or increment the selected `GymMembership.authz_version` or `Member.authz_version` as applicable.

## 6. Authorization model

Permissions are durable records identified by stable codes. A released code is never renamed or reused for a different meaning; deprecation creates a replacement code and an explicit migration. Authorization checks use permission codes plus object and state rules, not role-name conditionals scattered through views.

Matrix values are defaults:

- **Allow (A):** the protected role template includes the permission, subject to tenant scope, record-state rules, and normal validation.
- **Conditional (C):** allowed only under the object, workflow, delegation, or step-up conditions stated below.
- **Deny (D):** absent by default and denied unless an OWNER explicitly adds a permission that is in the role's delegable set. A hard-reserved permission remains denied and cannot be delegated.

### Platform permission matrix

Platform permissions use the `platform.*` namespace and are evaluated outside ordinary tenant-role permissions.

| Stable permission code | SUPER_ADMIN | Scope and condition |
|---|---|---|
| `platform.dashboard.view` | A | View bounded platform operational summaries; not tenant business records. |
| `platform.gyms.view` | A | View gym identity, lifecycle, and subscription metadata; not tenant business records. |
| `platform.gyms.create` | A | Create a gym through the audited provisioning workflow. |
| `platform.gyms.update` | A | Update gym identity and approved lifecycle metadata. |
| `platform.gyms.activate` | A | Activate an eligible gym with reason and policy checks. |
| `platform.gyms.suspend` | A | Suspend a gym with reason, impact checks, and audit. |
| `platform.owners.create` | C | Create/link the initial or an additional owner only within the audited gym-provisioning or approved owner workflow. |
| `platform.subscriptions.manage` | A | Manage plans and tenant entitlements. |
| `platform.settings.manage` | A | Manage platform-wide configuration, never tenant-owned settings through a tenant bypass. |
| `platform.audit.view` | A | View platform audit events and support-access events. |

Platform status does not grant any tenant permission. Time-limited support and break-glass access are governed workflows, not additional stable permission codes: a support session receives only explicitly approved canonical tenant permissions, while refunds, voids, cash adjustments, commission payments, role changes, and OWNER-reserved controls remain denied unless incident policy separately approves each exact action.

### Tenant permission matrix

Tenant permissions use canonical `module.action` codes with no `tenant.` prefix. This matrix defines protected template defaults for tenant staff roles.

| Stable permission code | OWNER | MANAGER | RECEPTIONIST | ACCOUNTANT | TRAINER |
|---|---|---|---|---|---|
| gym.view | A | A | A | C | C |
| gym.update | A | C | D | D | D |
| members.view | A | A | A | C | C |
| members.create | A | A | A | D | D |
| members.update | A | A | A | D | C |
| members.archive | A | C | D | D | D |
| memberships.view | A | A | A | A | C |
| memberships.create | A | A | A | D | D |
| memberships.update | A | A | A | D | D |
| memberships.cancel | A | C | C | D | D |
| checkins.view | A | A | A | D | C |
| checkins.create | A | A | A | D | C |
| checkins.update | A | C | C | D | D |
| payments.view | A | A | A | A | D |
| payments.create | A | A | A | A | D |
| payments.update | A | A | A | A | D |
| payments.submit | A | A | A | A | D |
| payments.approve | A | A | D | D | D |
| payments.void | A | C | D | C | D |
| payments.refund | A | C | D | C | D |
| receipts.view | A | A | A | A | D |
| receipts.void | A | C | D | C | D |
| expenses.view | A | A | C | A | D |
| expenses.create | A | A | C | A | D |
| expenses.update | A | A | C | A | D |
| expenses.submit | A | A | C | A | D |
| expenses.approve | A | A | D | D | D |
| expenses.void | A | C | D | C | D |
| cashbook.view | A | A | D | A | D |
| cashbook.open | A | C | D | A | D |
| cashbook.reconcile | A | C | D | A | D |
| cashbook.adjust | A | C | D | C | D |
| pt.assignments.view | A | A | C | D | C |
| pt.assignments.manage | A | A | D | D | C |
| pt.sales.view | A | A | C | A | C |
| pt.sales.create | A | A | C | A | C |
| pt.sales.update | A | A | C | A | C |
| pt.sales.submit | A | A | C | A | C |
| pt.sales.approve | A | A | D | D | D |
| pt.sales.void | A | C | D | C | D |
| pt.commissions.view | A | A | D | A | C |
| pt.commissions.pay | A | C | D | A | D |
| salaries.view | A | A | C | A | D |
| salaries.create | A | A | C | A | D |

| Stable permission code | OWNER | MANAGER | RECEPTIONIST | ACCOUNTANT | TRAINER |
|---|---|---|---|---|---|
| `salaries.update` | A | A | C | A | D |
| `salaries.submit` | A | A | C | A | D |
| `salaries.approve` | A | A | D | D | D |
| `salaries.void` | A | C | D | C | D |
| `reports.view` | A | A | C | C | C |
| `reports.financial` | A | C | D | A | D |
| `reports.export` | A | C | D | C | D |
| `staff.view` | A | A | D | D | D |
| `staff.manage` | A | C | D | D | D |
| `roles.view` | A | A | D | C | D |
| `roles.manage` | A | C | D | D | D |
| `settings.view` | A | A | D | C | D |
| `settings.manage` | A | D | D | D | D |
| `files.view` | A | A | C | C | C |
| `files.upload` | A | A | A | A | C |
| `files.delete` | A | C | D | D | D |
| `audit.view` | A | C | D | C | D |
| `notifications.view` | A | A | A | A | A |
| `notifications.manage` | A | C | D | D | D |

High-risk conditional actions require the relevant permission plus valid record state, a reason, idempotency protection where applicable, and an audit event. Payment, expense, PT-sale, and salary approval is mandatory for every record before posting. The approver must be a different active Manager or Owner gym membership from both creator and submitter; permissions cannot authorize self-approval. A permission never overrides immutable posted records, tenant scope, object scope, or database constraints.

Role-specific conditional defaults are:

- MANAGER owns the standard approval queue for submitted payments, expenses, PT sales, and salary periods. Approval is allowed only for records the manager did not create or submit, after revalidation of amounts, evidence, source state, and any cash session. Other conditional financial, cancellation, correction, archive, export, deletion, staff, audit, and commission actions remain limited to OWNER-enabled thresholds. Staff management is limited to non-OWNER accounts and roles within the manager's authority.
- RECEPTIONIST may create, edit, and submit front-desk payment, expense, PT-sale, and salary drafts but cannot approve or post them. Conditional expense and salary reads are limited to the receptionist's own drafts/submissions and minimum staff identity fields; posted salary totals, other staff payroll history, and protected category internals are hidden. PT views/sales are limited to the member transaction being served; operational reports exclude financial measures; file downloads require access to the related member or transaction.
- ACCOUNTANT conditional member access exposes only identity and contact fields needed for a financial workflow. The accountant may prepare and submit finance records but cannot approve them under the protected template; cash adjustments and corrections follow separation-of-duties rules. Operational reports, settings, files, and audit events are limited to finance-relevant scope.
- TRAINER conditional permissions are governed by the assignment and own-commission restrictions below. Trainer operational reports aggregate only records the trainer is authorized to see.

The four `*.approve` permissions for payments, expenses, PT sales, and salaries are hard-reserved to the protected MANAGER and OWNER roles in Phase 0. They are not in the delegable set and cannot be added to ACCOUNTANT, RECEPTIONIST, TRAINER, or a custom role.

## Delegation and custom roles

The OWNER controls the tenant's delegable set and may add or remove delegable permissions from MANAGER and other custom roles. The MANAGER row is the standard operational template; `C` entries require their stated workflow conditions, while `D` entries are not available unless the OWNER has placed that exact permission in the MANAGER role's delegable set. Removing a permission takes effect through the authorization-version mechanism.

A caller using `roles.manage` may create or edit non-protected custom roles and assign them only when all of these are true:

1. the caller currently possesses every permission being granted;
2. every granted permission is within the tenant's OWNER-defined delegable set;
3. the caller is allowed to manage the target staff account and role;
4. the change does not modify a protected template, OWNER account, or reserved control.

The effective grant is therefore the intersection of the caller's current permissions and the delegable set; mass assignment, direct identifiers, stale tokens, or custom-role composition cannot expand it. The API revalidates the complete resulting role on every change, including removals

and replacements, rather than checking only submitted additions.

`roles.manage` does not by itself authorize OWNER appointment/removal, protected template changes, security and billing settings, delegation-policy changes, or approval of platform support access; those remain OWNER-reserved object/workflow controls. A MANAGER cannot delegate these controls, create an OWNER-equivalent role, manage any OWNER, or grant any permission the MANAGER lacks.

A gym may have multiple active Owners with equal Owner login authority. Adding an Owner requires an active Owner or authorized Super Admin, recent MFA/step-up verification, the protected invitation workflow, and audit. Revoking or suspending an Owner requires a different active Owner or authorized platform workflow, a reason, row locking, authorization-version invalidation, and proof that another active Owner remains. The service locks the gym before counting active Owners, and a deferred database invariant prevents concurrent requests from leaving an active gym ownerless. General staff/role APIs cannot set `is_owner`, assign the protected OWNER role, or bypass these controls. Owner status is authorization only and must not be presented as evidence of equity, capital contribution, profit entitlement, or legal ownership.

### Object restrictions and entitlements

- A TRAINER with a conditional member, membership, check-in, PT, or file permission can access only members currently assigned to that trainer and records belonging to those assignments. A trainer may update only explicitly trainer-editable fields.
- `pt.commissions.view` gives a TRAINER access only to that trainer's own commission records; it never permits viewing another trainer's rates, commissions, or payroll. `pt.commissions.pay` is always denied to TRAINER.
- A RECEPTIONIST's conditional salary scope permits only draft preparation/submission and the minimum staff identity needed to select a payee. Salary approval, posted salary history, aggregate salary reporting, and other creators' salary drafts are denied.
- Member self-service endpoints are a separate principal type, not a tenant staff role. The verified `Member` linked to the authenticated global `User` supplies the authoritative gym and self identity; after approved bootstrap and normal RLS reload, every query must satisfy both `gym_id = authenticated_member.gym_id` and `member_id = authenticated_member.id`. A member principal never derives gym from `GymMembership`, and possession of another member or gym ID has no effect.
- A normal cross-tenant object lookup returns `404`, including nested resources, relationships, files, reports, and custom actions. Authentication failures remain `401`, and same-tenant permission failures may be `403`.
- Product-plan feature entitlements and RBAC answer different questions. An entitlement may disable a feature that RBAC would otherwise allow, but it cannot create, imply, or grant a permission. Access requires both an active entitlement and an authorization result of allow.

Authorization state is authoritative in PostgreSQL. Tokens carry an `authz_version` that is compared with current server state for protected requests. Redis is not assumed or required; any future cache is an optimization only, uses bounded lifetime/invalidation, and failure or staleness cannot grant access.

## 7. API defenses

- Explicit serializer fields prevent mass assignment of gym, actor, status, calculated values, sequence numbers, and permissions.
- DRF querysets are tenant-scoped for list, retrieve, update, delete, and custom actions.
- Object permissions are always called by custom actions.
- Filters and ordering fields are allowlisted.
- Cursor pagination, date range, upload, and export limits prevent unbounded work.
- Financial commands use idempotency keys and database transactions.
- SQL uses ORM or parameterized statements. Raw SQL receives security review.
- Error responses contain safe stable codes and request IDs, never traces or database messages.
- Cross-tenant probes can be rate-limited and alerted without confirming object existence.

## 8. Web and mobile defenses

### XSS

- React output escaping remains enabled.
- Avoid `dangerouslySetInnerHTML`; sanitize with a reviewed library only for an approved rich-text feature.
- Apply Content Security Policy, frame restrictions, MIME sniffing protection, Referrer Policy, and Permissions Policy.
- Uploaded HTML/SVG is not rendered inline from the application origin.

### CSRF and CORS

- Cookie authentication uses CSRF protection.
- CORS is a strict allowlist for production web origins and required headers/methods.
- CORS is not an authorization control.

### Client data

- Cache only data required for the current screen.
- Do not persist sensitive reports or tokens in browser storage.
- Navigation hiding mirrors permissions for usability only.
- QR codes contain short-lived opaque or signed references, not member personal data.

## 9. File security

MinIO object storage is private, uses explicit S3-compatible bucket policies, and is reachable only through the internal Docker network and authorized presigned-URL path.

Upload controls:

- create a tenant-scoped upload intent after permission checks;
- use random object identifiers;
- restrict signed URL expiry, method, maximum size, and expected content type;
- verify actual size, checksum, and detected content type;
- quarantine and malware-scan untrusted content before use;
- strip sensitive image metadata where required;
- disallow executable/active content unless a business case and safe delivery path exist.

Download controls:

- look up RLS-protected file metadata;
- verify tenant, actor permission, and relationship to the domain record;
- issue a short-lived URL with safe `Content-Disposition`;
- audit sensitive downloads and exports.

An object key containing a gym prefix is organization, not authorization.

## 10. Financial security

- Use `Decimal`/`DecimalField`, never binary floats.
- Backend services calculate totals, expiry, commission, expected cash, and differences.
- Posted records are immutable; correction actions require reason and linkage.
- Receipt sequences are allocated atomically and never client supplied.
- Raw card data and CVV are never stored.
- Payment provider callbacks require signature verification, timestamp tolerance, event deduplication, and idempotent processing.
- Reconciliation, refunds, voids, and manual cash adjustments require elevated capabilities and audit.
- Payments, expenses, PT sales, and salary periods create no receipt, settlement, commission, expense, or cash effect until a different authorized Manager or Owner approves them. Approval is the immutable posting boundary; rejected submissions retain audit evidence and no economic side effects.
- Database constraints assert valid ranges and tenant-consistent relationships.

## 11. Audit and privacy

Audit events include actor, gym, action, module, target ID, safely redacted field changes, result, request ID, source IP, user agent, and timestamp. Platform support actions also record the effective tenant and impersonator.

Never log:

- passwords, refresh/access tokens, MFA secrets;
- raw card or bank credentials;
- signed URLs;
- full unbounded request/response bodies;
- sensitive notes unless a specific redacted audit representation is approved.

Audit storage is append-only to the application. Critical events are exported to encrypted offsite retention with alerting so loss of the VPS does not erase all security evidence. Access to audit records itself is audited.

Privacy requirements include purpose limitation, retention schedules, controlled exports, deletion/anonymization workflows, and backup expiry. Applicable jurisdiction and retention periods must be confirmed before production.

Membership-expiry email is sent only to the member's current normalized email when the gym setting is enabled. The scheduler and sender re-check tenant, membership eligibility, warning date, and the current address; they do not trust an address copied into a queued payload. A missing or removed address is a normal skip/cancellation and never falls back to phone or another member's contact details. Templates expose only the member display name, plan display name, expiry date, and necessary gym contact/display fields. Deterministic database deduplication, bounded retries, redacted provider errors, and audited configuration changes prevent duplicate delivery and accidental personal-data leakage.

## 12. Infrastructure and secrets

- Docker secrets or root-readable encrypted environment files store database credentials, JWT private keys, provider secrets, backup keys, and MinIO signing credentials. Secrets never enter source control, container images, Compose files, logs, or shell history.
- The VPS firewall exposes only 80/443 publicly. SSH is key-only, rate-limited, restricted to approved administrators/source ranges where practical, and protected by fail2ban or equivalent controls.

- PostgreSQL and MinIO expose no public host ports and accept traffic only on explicit private Docker networks. Application, migration, backup, and administrative paths use separate least-privilege credentials.
- Production, staging, preview, and development data use separate credentials, databases, object buckets, and volumes. Permanent staging does not run on the production VPS in the budget baseline.
- Runtime, migration, reporting, and break-glass database roles are separate.
- Containers run as non-root, use minimal pinned base images, read-only filesystems where practical, dropped capabilities, health checks, resource limits, and only explicitly required private/public ports.
- The host uses a pinned supported Linux release, unattended security updates, disk encryption where the provider supports it, bounded log rotation, a monthly maintenance window, and monitored disk/RAM/CPU pressure.
- CI uses short-lived GHCR credentials and protected deployment environments. Production deploy access is narrowly scoped, audited, and cannot retrieve backup decryption material unnecessarily.
- Dependency, image, secret, and migration scans block severe findings.
- Database/WAL/object backups are encrypted before leaving the VPS, stored with an independent provider/credential boundary, monitored for freshness, retention-protected where supported, and subject to the same access expectations as primary data.
- Continuous offsite WAL archiving targets RPO ≤ 15 minutes; the replacement-VPS rebuild and restore runbook targets RTO ≤ 4 hours. These are planning targets until demonstrated by a pre-production drill and quarterly restore drills.

## 13. Rate limits and abuse controls

Separate throttles cover:

- login, refresh, password reset, and MFA;
- ordinary authenticated reads;
- mutations;
- file upload/download URL generation;
- searches and report queries;
- exports and platform administration.

Cloudflare edge controls and Caddy request limits complement application-level user/gym/session throttles. The API trusts forwarding headers only from the private reverse-proxy path. Resource limits and graceful overload protect PostgreSQL and the shared VPS from one endpoint exhausting the host.

## 14. Failure behavior

- Missing tenant context: reject or return no tenant rows.
- Invalid or conflicting token gym: `401`/`403`, never choose a default.
- Cross-tenant object: `404`.
- Permission/cache failure: deny sensitive operations.
- Disabled RLS or privileged runtime role: deployment health/security check fails.
- Background job without tenant context: fail without processing.
- Signed URL failure: do not expose a bucket URL.
- Audit pipeline failure: persist in the authoritative local append-only audit table; high-risk actions follow a documented fail-closed policy. An outbox remains deferred until measured asynchronous need.
- Secret or key loading failure: service remains unhealthy.

The application never silently switches to a privileged database credential.

## 15. Security test suite

### Tenant isolation

For every tenant resource and HTTP action:

- Gym A cannot access Gym B by path ID.
- Cross-gym IDs in request bodies and serializer relationships fail.
- Lists, filters, search, annotations, joins, counts, sums, dashboard cards, reports, and exports contain only active-gym data, including empty and adversarial filter cases.
- Bulk operations, if introduced, authorize every item.
- Insert/update cannot set or change `gym_id`.
- Two sequential gym requests on one reused pooled connection remain isolated after success, rollback, exception, and nested transaction paths.
- Concurrent requests for different gyms on pooled connections never exchange context.
- Every tenant query fails closed when attempted before `set_config(..., true)` or outside its `transaction.atomic()` scope.
- Missing/malformed RLS settings expose no rows.
- Cross-tenant probes return the documented `404` without timing or aggregate-count disclosure.

### Authorization

- Every matrix cell has allowed, conditional, and denied endpoint tests using stable permission codes.
- Receptionist cannot access owner financial reports by default.
- Receptionist can submit only authorized front-desk finance drafts, cannot approve any finance record, and cannot view salary records outside the restricted own-draft scope.
- Manager approval fails for a record created or submitted by that same manager membership, including concurrent role or actor changes.
- Trainer sees only assigned members, assignment-related files, and the trainer's own private commission.
- Member self endpoints require both the authenticated gym and authenticated member; changing either path/body identifier fails.
- Custom role create, update, replacement, and assignment cannot grant permissions absent from the caller or outside the delegable set.
- MANAGER cannot modify protected templates, OWNER accounts, OWNER-reserved controls, or construct an equivalent custom role.
- Multiple Owners can authenticate independently within the same gym; duplicate invitation/link attempts are idempotent, and no ordinary staff or role mutation can create an Owner.
- Concurrent Owner revoke/suspend/role-removal requests cannot leave an active gym without at least one active Owner; self-revocation and attempts by non-Owners fail.
- Suspended memberships, role removals, delegation changes, and stale `authz_version` fail without relying on Redis.
- Feature entitlement changes cannot add RBAC permissions, and permissions cannot bypass a disabled entitlement.
- Object permissions execute on custom actions, nested routes, bulk actions, exports, and files.

### Authentication and API

- JWT signature, issuer, audience, algorithm, expiry, type, and modified claim tests.
- Refresh rotation, replay, revocation, and logout tests.
- Unauthenticated access and rate limiting.
- CSRF tests cover every browser cookie mutation, missing/incorrect tokens, and Origin/Referer validation; strict CORS tests verify denied origins and credential behavior.
- Mass-assignment attempts cover gym, actor, owner, assignment, totals, status, permissions, roles, approval fields, commission recipient, and receipt/sequence numbers on create, update, bulk, and nested serializers.
- SQL/ORM injection payloads cover search, filters, ordering, report parameters, exports, filenames, and every reviewed raw-SQL boundary.
- Stored, reflected, and DOM XSS payloads cover displayed text, reports, exports, filenames, and uploaded active content; CSP behavior is tested.
- Privilege-escalation tests cover forged role/permission fields, stale authorization, OWNER reservation, support sessions, and platform-to-tenant boundary crossing.

### Files and finance

- Cross-gym, unassigned-trainer, and non-self-member file metadata, upload intent, download, delete, and signed URL requests are denied without object existence disclosure.
- URL expiry/replay expectations, method binding, filename disposition, checksum, content-type mismatch, oversize, malware quarantine, and active-content delivery are tested.
- Exact decimal rounding, commission split, membership expiry, expected cash, and immutable-posting rules are tested.
- Duplicate and concurrent payment, expense, PT-sale, and salary submit/approve/reject commands, callbacks, voids, refunds, cashbook open/reconcile/adjust, commission pay, and receipt allocation produce one valid financial result.
- Rejected finance submissions produce no receipt, source settlement, commission, salary expense, or cash transaction; approval after rejection or after draft mutation fails.
- Idempotency keys are actor/tenant/operation bound; replay with altered payload, tenant, object, or actor fails and concurrent identical requests return the same committed result.
- Failed and retried transactions do not double-post ledgers, reuse sequence numbers incorrectly, or leave partial audit/financial state.

### Database and deployment

Security tests run against real PostgreSQL, not SQLite. CI asserts RLS policy coverage, forced RLS, constraints, indexes, runtime grants, and absence of BYPASSRLS.

Deployment checks also assert that PostgreSQL/MinIO are not publicly reachable, containers run with approved users/capabilities/limits, production secrets are absent from images and Compose output, offsite backup/WAL age is within policy, and a temporary-stack restore reproduces roles, forced RLS, finance totals, and object references.

## 16. Phase 0 design status

Phase 0 is design and documentation only. This document defines required behavior but does not prove that endpoints, database policies, roles, or tests implement it. During Phase 0, permission codes and matrix semantics must be reviewed with product, finance, privacy, and security owners; unresolved cells or workflows are explicit design blockers.

No tenant-data feature may pass the Phase 0 security gate until its data ownership, stable permissions, object restrictions, entitlement dependency, Django scoping, forced-RLS policy, transaction context, audit events, and test cases are documented and approved. No documentation status may be represented as a runtime security control.

## 17. Security release gate

Before production:

1. Phase 0 threat model, stable permission matrix, delegation rules, and OWNER-reserved controls are approved, and implementation traceability maps every protected endpoint/action to them;
2. all critical tenant isolation tests pass against production-like pooling;
3. no critical/high unresolved dependency or image findings;
4. secret scanning clean;
5. replacement-VPS/isolated-Compose restore drill completed from encrypted offsite database/WAL/object backups within RPO ≤ 15 minutes and RTO ≤ 4 hours, with quarterly drills scheduled and owned;
6. key rotation and incident runbooks tested;
7. audit and security alerts reach responders;
8. penetration test covers IDOR, tenant isolation, privilege escalation, JWT, files, and financial workflows;
9. retention, privacy, and terms requirements approved;
10. break-glass access tested and audited;
11. CI proves both explicit Django scoping and forced PostgreSQL RLS independently fail closed;
12. all tenant work is verified to run inside `transaction.atomic()` after transaction-local `set_config(..., true)`;
13. authorization tests cover every matrix default, custom-role anti-escalation, object restriction, stale authorization, and entitlement/RBAC separation;
14. replay and concurrency tests pass for finance approval, payments, receipts, expenses, salaries, cashbook, refunds, voids, PT sales/commissions, and provider callbacks;
15. firewall, SSH, private Docker networks, container limits, disk/log rotation, external uptime, backup freshness, and offsite-restore controls pass the single-VPS production checklist.

# Project Structure

## 1. Repository strategy

A single repository coordinates API contracts, local development, documentation, and CI while keeping deployable applications independent.

```
gym-management-platform/
├── .github/
│   └── workflows/
│       ├── ci.yml
│       ├── deploy-staging.yml
│       └── deploy-production.yml
├── docs/
│   ├── ARCHITECTURE.md
│   ├── DATABASE.md
│   ├── API.md
│   ├── SECURITY.md
│   ├── PROJECT_STRUCTURE.md
│   ├── DEVELOPMENT_PLAN.md
│   ├── HOW_IT_WORKS.md
│   └── adr/
│       └── 0001-single-vps-production-topology.md
├── backend/
├── web/
├── mobile/
├── infra/
├── scripts/
├── docker-compose.yml
├── .env.example
├── .editorconfig
├── .gitignore
├── Makefile
└── README.md
```

Phase 0 is documentation-only. At the end of Phase 0, no repository scaffold, backend, web application, mobile application, infrastructure code, workflow, container, or application configuration is assumed to exist beyond the files under `/docs`. Every other path shown in this document is a target structure to be created only in its approved roadmap phase.

## 2. Backend

```
backend/
├── config/
│   ├── settings/
│   │   ├── base.py
│   │   ├── local.py
│   │   ├── test.py
│   │   └── production.py
│   ├── api_router.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── apps/
│   ├── common/
│   │   ├── api/
│   │   │   ├── exceptions.py
│   │   │   ├── pagination.py
│   │   │   └── responses.py
│   │   ├── db/
│   │   │   ├── models.py
│   │   │   └── tenant_context.py
│   │   ├── money.py
│   │   ├── time.py
│   │   └── validators.py
│   ├── tenants/
│   ├── users/
│   ├── members/
│   ├── memberships/
│   ├── checkins/
│   ├── payments/
│   ├── expenses/
│   ├── salaries/
│   ├── trainers/
│   ├── commissions/
│   ├── cashbook/
│   ├── receipts/
│   ├── reports/
│   ├── notifications/
│   ├── files/
│   └── audit/
├── requirements/
│   ├── base.txt
│   ├── development.txt
│   └── production.txt
├── tests/
│   ├── factories/
│   ├── security/
│   ├── integration/
│   └── conftest.py
├── manage.py
├── pyproject.toml
├── Dockerfile
└── .env.example
```

### Domain app convention

The backend remains domain-based. `tenants`, `users`, `members`, `memberships`, `checkins`, `payments`, `expenses`, `salaries`, `trainers`, `commissions`, `cashbook`, `receipts`, `reports`, `notifications`, `files`, and `audit` are the approved domain apps. `common` is shared infrastructure for cross-cutting API, database-context, money, time, and validation primitives; it is not a catch-all business domain.

Each substantial app follows a predictable shape:

```
apps/members/
├── api/
│   ├── serializers.py
│   ├── permissions.py
│   ├── views.py
│   └── urls.py
├── migrations/
├── admin.py
├── apps.py
├── models.py
├── selectors.py
├── services.py
├── validators.py
└── tests/
    ├── test_models.py
    ├── test_services.py
    ├── test_api.py
    ├── test_permissions.py
    └── test_tenant_isolation.py
```

Small apps need not contain empty layers. The rules are:

- `models.py`: persistence shape and local invariants;
- `serializers.py`: explicit HTTP input/output validation;
- `views.py`: HTTP orchestration only;
- `permissions.py`: reusable action/object authorization;
- `services.py`: transactional commands and business calculations;
- `selectors.py`: optimized, tenant-scoped reads;
- tests live beside the domain, with cross-domain security suites under `/tests`.

Complex financial calculations do not live in signals. Signals are limited to non-critical decoupled hooks; explicit services make transaction boundaries visible.

## 3. Next.js web

The web workspace uses Next.js 15+ with the App Router, TypeScript in strict mode, and Tailwind CSS. React Server Components (RSC) are the default; Client Components are explicit boundaries used only where browser interaction or local transient state requires them.

```
web/
├── public/
├── src/
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/
│   │   │   └── select-gym/
│   │   ├── (tenant)/
│   │   │   ├── layout.tsx
│   │   │   ├── dashboard/
│   │   │   ├── members/
│   │   │   ├── memberships/
│   │   │   ├── check-ins/
│   │   │   ├── payments/
│   │   │   ├── expenses/
│   │   │   ├── approvals/
│   │   │   ├── salaries/
│   │   │   ├── cash-book/
│   │   │   ├── pt/
│   │   │   │   ├── trainers/
│   │   │   │   ├── sales/
│   │   │   │   └── commissions/
│   │   │   ├── reports/
│   │   │   ├── staff/
│   │   │   └── settings/
│   │   ├── super-admin/
│   │   │   ├── layout.tsx
│   │   │   ├── dashboard/
│   │   │   ├── gyms/
│   │   │   ├── subscriptions/
│   │   │   └── audit-logs/
│   │   ├── api/
│   │   │   └── auth/
│   │   ├── error.tsx
│   │   ├── not-found.tsx
│   │   └── layout.tsx
│   ├── components/
│   │   ├── ui/
│   │   ├── layout/
│   │   └── feedback/
│   ├── features/
│   │   ├── auth/
│   │   ├── members/
│   │   ├── memberships/
│   │   ├── finance/
│   │   ├── approvals/
│   │   ├── salaries/
│   │   ├── checkins/
│   │   ├── trainers/
│   │   ├── pt/
│   │   ├── commissions/
│   │   ├── reports/
│   │   └── staff/
│   ├── lib/
│   │   ├── api/
│   │   │   ├── client.ts
│   │   │   ├── errors.ts
│   │   │   └── server.ts
│   │   ├── auth/
│   │   ├── env.ts
│   │   └── formatting/
│   ├── services/
│   ├── types/
│   │   └── api.generated.ts
│   ├── hooks/
│   └── utils/
├── tests/
├── next.config.ts
├── tsconfig.json
├── package.json
├── Dockerfile
└── .env.example
```

## Frontend boundaries

- Route components compose feature components.
- React Server Components are the default for routes, layouts, and initial data access where session and caching semantics allow.
- Client Components are used for forms, interactive tables, dialogs, and browser APIs.
- Payment, expense, PT-sale, and salary forms create drafts and submit them; the manager approval queue is a separate feature boundary and never performs client-side posting calculations.
- Salary period/line UI remains separate from PT commission UI. No fixed staff or PT amount is inferred in the frontend.
- PT sales, trainer workflows, and commission accrual/payout UI remain separate feature boundaries even when composed under the tenant `pt` route.
- API functions are centralized; components do not construct ad hoc URLs.

- Generated or checked API types prevent contract drift.
- Formatting utilities receive currency and timezone explicitly.
- No Redux/global store is added without a demonstrated cross-route state requirement.
- Server data caching is tenant-aware and never keyed only by route or resource ID.

## 4. React Native mobile

```
mobile/
├── src/
│   ├── app/
│   │   ├── navigation/
│   │   └── providers/
│   ├── features/
│   │   ├── auth/
│   │   ├── dashboard/
│   │   ├── profile/
│   │   ├── membership/
│   │   ├── payments/
│   │   ├── checkins/
│   │   ├── qr/
│   │   └── notifications/
│   ├── components/
│   │   └── ui/
│   ├── services/
│   │   ├── api/
│   │   ├── auth/
│   │   ├── secure-storage/
│   │   └── notifications/
│   ├── types/
│   ├── utils/
│   └── config/
├── ios/
├── android/
├── tests/
├── app.json
├── tsconfig.json
├── package.json
└── .env.example
```

The framework choice between Expo and bare React Native is deferred to Phase 7 based on notification, QR, biometric/device integration, build, and team constraints. This is an architectural decision requiring approval; API contracts remain framework-neutral.

## 5. Infrastructure

```
infra/
├── environments/
│   ├── development/
│   ├── staging/
│   └── production/
├── compose/
│   ├── docker-compose.prod.yml
│   ├── docker-compose.staging.yml
│   └── migration.yml
├── caddy/
│   └── Caddyfile
├── postgres/
│   ├── init/
│   ├── backup/
│   └── restore/
├── minio/
│   └── policies/
├── monitoring/
│   ├── healthchecks/
│   └── logrotate/
├── backup/
│   ├── database.sh
│   ├── wal-archive.sh
│   ├── objects.sh
│   └── verify.sh
├── scripts/
│   ├── deploy.sh
│   ├── rollback.sh
│   └── rebuild-vps.sh
├── cloudflare/
└── README.md
```

The Phase 8 baseline is one production VPS managed through reviewed Docker Compose and host/bootstrap scripts. Environment directories contain references and safe templates, never plaintext secrets. `staging/`, `docker-compose.staging.yml`, and `deploy-staging.yml` target local or ephemeral temporary infrastructure with separate credentials and volumes; they do not create a permanent staging stack on the production VPS. GitHub Container Registry stores immutable digest-pinned web/API images. GitHub Actions under `/.github/workflows` runs quality and security gates, publishes approved images to GHCR, then invokes a protected deployment that takes a backup checkpoint, runs the

one-shot migration service with the migration role, performs health checks, and restarts services in controlled order. Database and MinIO volumes remain local to the VPS while encrypted copies and WAL archives leave the host through `infra/backup/`.

## 6. Tests

### Backend

```
backend/tests/
├── security/
│   ├── test_rls_inventory.py
│   ├── test_connection_context.py
│   ├── test_cross_tenant_api.py
│   ├── test_jwt_security.py
│   ├── test_mass_assignment.py
│   └── test_file_isolation.py
├── integration/
│   ├── test_membership_payment_flow.py
│   ├── test_cash_reconciliation_flow.py
│   ├── test_finance_approval_flow.py
│   ├── test_salary_flow.py
│   └── test_commission_flow.py
└── factories/
```

Domain unit/API tests remain in each app. Security tests use real PostgreSQL.

### Frontend and mobile

- feature/component behavior tests;
- accessibility tests for critical forms and navigation;
- API client contract/error tests;
- a small number of end-to-end critical paths;
- no snapshots used as a substitute for behavior assertions.

## 7. Configuration and dependency rules

- `.env.example` documents names and safe placeholders only.
- Runtime configuration is validated at startup.
- Secrets come from local ignored development files or production Docker secrets/root-readable encrypted host files; they never enter Compose files, images, logs, or source control.
- Dependencies are added only for a current requirement and reviewed for maintenance/security.
- Python and Node versions are pinned in Phase 1A.
- Formatting, lint, type check, tests, migration checks, and API schema checks run in CI.

## 8. Naming rules

- Python modules and JSON fields: `snake_case`.
- TypeScript variables/functions: `camelCase`; components/types: `PascalCase`.
- URLs use plural kebab-case resources.
- Permission codes use `module.action`.
- Database indexes and constraints have stable descriptive names.
- Use `Gym` in product/domain language and `gym_id` in ownership columns; avoid mixing `tenant_id` and `gym_id`.

## 9. Approved roadmap alignment

This target structure is introduced incrementally:

1. **Phase 1A — Foundation:** repository/tooling, Django and database foundation, authentication, tenant isolation, permissions, CI, and the minimal Next.js web shells.
2. **Phase 1B — Administration:** Super Admin, gym management, staff, roles and permissions, and gym settings.
3. **Phase 2 — Membership:** members, plans, membership lifecycle, and related web features.
4. **Phase 3 — Check-in:** check-in/check-out workflows and eligibility.
5. **Phase 4 — Finance:** payment/expense/salary drafts, manager approval queue, receipts, cash book, and reconciliation.
6. **Phase 5 — PT:** trainers, submitted PT sales, manager approval, commission accruals, payouts, and their explicit web feature boundaries.
7. **Phase 6 — Reporting:** operational dashboards, explicit owner financial metrics, reports, and bounded exports.
8. **Phase 7 — Mobile:** the separate React Native member workspace and mobile delivery.
9. **Phase 8 — Hardening and production deployment:** security and quality hardening, single-VPS Docker Compose infrastructure, GHCR deployment workflows, bounded observability, encrypted offsite backups, restore validation, and production rollout.

Phase labels outside this sequence are stale and must not be used to justify creating target paths early.

## 10. Architectural decision records

Material changes are recorded under:

```
docs/adr/
├── 0001-single-vps-production-topology.md
└── ...
```

Each ADR states context, decision, alternatives, security impact, migration impact, and status. Creating ADRs begins when implementation decisions differ from or refine these Phase 0 documents.

# Development Plan

## 1. Authority and delivery policy

This is the authoritative implementation roadmap. It must remain consistent with the complete Phase 0 document set:

1. `ARCHITECTURE.md` — system boundaries, tenancy, deployment, reliability, observability, and scale;
2. `DATABASE.md` — schema, ownership, constraints, RLS, transactions, and migrations;
3. `API.md` — versioned HTTP contracts and phase delivery map;
4. `SECURITY.md` — threat model, permission matrices, isolation, and release gates;
5. `PROJECT_STRUCTURE.md` — repository, application, test, and infrastructure layout;
6. `DEVELOPMENT_PLAN.md` — sequence, dependencies, checks, and exit criteria;
7. `HOW_IT_WORKS.md` — Burmese user journeys and operational explanations derived from the contracts above;
8. accepted records under `adr/` — approved material decisions that refine the seven documents, beginning with the single-VPS production topology.

Work proceeds one approved phase at a time. Phase 1 has two ordered subphases, 1A and 1B; both must pass their own gates. A phase or subphase may not advance while any required migration, test, format check, lint check, type check, schema check, security scan, build, smoke test, or documented manual check is failing. Flaky, skipped, muted, or waived checks are errors unless the waiver, owner, risk, and expiry are explicitly approved.

Every implementation increment must:

1. inspect the repository, authoritative documents, and migration history;
2. confirm dependencies, scope, security boundaries, and acceptance criteria;
3. implement the smallest complete vertical slice;
4. add unit, database, API, tenant/RLS, permission, financial, and concurrency tests as applicable;
5. run all phase checks against real PostgreSQL where database behavior matters;
6. review tenant isolation, permissions, audit, financial integrity, privacy, and operational impact;
7. update migrations, OpenAPI/schema, generated types, runbooks, and documentation in the same change;
8. summarize evidence, unresolved issues, and rollback considerations;
9. obtain approval before starting the next phase.

Any major architecture change requires approval before implementation. The proposal must state:

```
Problem
Current approved approach
Proposed change and alternatives
Reason and evidence
Security, tenant, data, API, migration, cost, and operational impact
Rollback/compatibility plan
```

Material changes are recorded in an ADR. Safe defaults may resolve minor implementation details, but security, tenant, financial, regulatory, public-contract, migration, deployment-provider, or cross-application changes are never assumed.

Redis and Celery are explicitly deferred. They are not baseline dependencies in Phases 0–8 and may be introduced only after measured need, an approved architecture change, tenant-safe failure semantics, and operational ownership are documented.

## 2. Phase 0 — Approved design baseline

Phase 0 is documentation and approval only. It creates no application features, infrastructure, database objects, migrations, or runtime security controls.

### Dependencies

- Confirmed product scope and stakeholders for product, operations, finance, privacy, and security.
- Access to the seven Phase 0 documents and every accepted ADR listed above.
- Explicit tracking of unresolved decisions and assumptions.

### Scope and 15 required design deliverables

The seven documents together with accepted ADRs must define and cross-reference all of the following:

1. Product scope, actors, web/API/mobile boundaries, and out-of-scope behavior.
2. Repository strategy, project structure, domain boundaries, configuration, and dependency rules.
3. Multi-tenant ownership classification for every global, tenant-root, and tenant-owned domain.
4. Authoritative PostgreSQL entities, relationships, types, constraints, indexes, deletion, retention, and immutability rules.
5. Django scoping plus transaction-local tenant context, forced RLS, complete `USING`/`WITH CHECK` policies, pooled-connection safety, and separate runtime/migration roles.

6. Custom `User`, staff/member authentication, strict JWT validation, rotating refresh sessions, revocation, browser CSRF/cookie boundaries, mobile secure storage, and MFA requirements.
7. Platform and tenant RBAC, stable permission codes, role templates, custom-role anti-escalation, object restrictions, feature entitlements, and authorization-version invalidation.
8. Versioned `/api/v1/` resources, explicit serializers, safe envelopes/errors, pagination/filter limits, idempotency, concurrency, and generated-client compatibility.
9. Member, membership-plan, membership lifecycle, calendar expiry, member dashboard, and private-file behavior.
10. Check-in eligibility and checkout plus QR, NFC, and biometric/device abstraction boundaries that store no biometric templates in this system.
11. Payments, receipts, expenses, salary periods/lines, cash sessions/transactions/reconciliation, PT sales/commissions, pre-post manager approval, exact decimal rules, immutable posting, reversals, sequence allocation, and atomic workflows.
12. Web and React Native architecture, tenant-aware client data handling, accessibility, QR safety, notifications, and Expo-versus-bare decision criteria.
13. Threat model, audit/privacy/file controls, test strategy, CI/CD gates, dependency/container/secret scanning, and release security evidence.
14. Production topology, environment separation, secrets, observability, incident response, backup/PITR, restore drills, RPO/RTO approval, rollout, and rollback.
15. Performance budgets, connection capacity, bounded queries, load strategy, scaling path, deferred Redis/Celery criteria, open decisions, and deployment-cost analysis structure.

### Migrations and data

- None. Schema examples are contracts, not executed migrations.
- The model and RLS inventory in `DATABASE.md` must account for every planned tenant table.
- Seed catalogs, provisioning data, test fixtures, and migration-role behavior must be specified before implementation.

### Required reviews and checks

- Cross-document phase names, model names, endpoint delivery annotations, permission codes, and ownership rules agree.
- Every accepted ADR is referenced by the affected documents, and Phase 0 reviews prove that its decision, risks, budget, and migration path remain consistent with the core contracts.
- Every tenant resource has documented Django scope, RLS policy, permission/object rules, audit behavior, and two-gym tests.
- Financial formulas, immutable states, reversal paths, locks, idempotency, and concurrency invariants agree across database, API, architecture, and security documents.
- Backup, restore, observability, load, deployment, rollback, and scale responsibilities are documented.
- All unresolved decisions have an owner, due phase, and explicit blocking status.

### Exit criteria

- All 15 design deliverables are covered, and the seven documents plus every accepted ADR are internally consistent.
- Product, finance, privacy, security, and operational assumptions are approved or explicitly marked as blockers.
- No critical tenancy, permission, financial, API, migration, backup, or deployment question is implicit.
- Phase 1A scope is approved.

## 3. Phase 1 — Foundation

### Phase 1A — Security foundation

#### Dependencies

- Phase 0 exit criteria and Phase 1A approval.
- Supported runtime/framework versions verified for compatibility.
- Approved local PostgreSQL strategy and initial browser authentication boundary.

#### Scope

- Initialize the repository only with approval if Git is absent; add README, ignore rules, safe environment examples, and repeatable development commands.
- Create the Django/DRF project, settings split, domain skeleton, `/api/v1/`, response envelopes, exception mapping, request IDs, pagination, health checks, and environment validation.
- Create the custom `User` before the first migration.
- Implement `Gym`, `GymMembership`, `Role`, `Permission`, role/permission joins, protected role templates, `RefreshSession`, foundational `AuditLog`, and only the support entities required by their approved contracts.
- Implement staff authentication: login, refresh rotation/replay defense, logout/logout-all, current user, gym membership list, and selected-gym token exchange.
- Implement stable permission evaluation, endpoint/action permission classes, object-check hooks, entitlement boundary, and authorization-version invalidation.
- Establish one transaction per tenant unit of work; set parameterized transaction-local gym/user/request context before tenant queries.

- Enable and force RLS with complete policies. Use separate non-owner runtime and migration roles; runtime has no superuser or `BYPASSRLS`.
- Enforce explicit Django gym scoping, server-owned `gym_id`, tenant-scoped relationships, and same-gym composite constraints.
- Add CI/CD foundation for backend and web checks; deployment is not yet production release.
- Create the Next.js App Router shell with strict TypeScript, Tailwind, auth, tenant, and super-admin layouts, typed API client boundary, safe error handling, and no business pages.
- Provide Docker/local PostgreSQL and isolated test configuration. Do not add Redis or Celery.

### Migrations and data

- Initial custom-user migration must precede dependent models.
- Reversible schema migrations for foundation tables, UUID/timestamp conventions, composite constraints, indexes, RLS functions/policies, and role grants.
- Data migrations seed immutable permission catalog and versioned protected role templates.
- Migration and runtime identities are verified independently; migrations must not silently run as the runtime owner.

### Required tests

- Authentication: strict JWT claims/signature/algorithm/key, login states, refresh rotation/replay, revocation, logout, CSRF/cookie boundary, and rate limits.
- Tenant/RLS: two-gym CRUD and relationship rejection; missing/malformed context; `USING` and `WITH CHECK`; server-owned gym; commit, rollback, exception, nested transaction, pooled reuse, and concurrent different-gym requests.
- Permission: default role allow/deny baselines, stale `authz_version`, protected templates, custom-role anti-escalation primitives, and platform-versus-tenant separation.
- Financial: no financial feature is implemented; tests prove foundation APIs cannot mass-assign financial/status/calculated fields and super admin acquires no tenant financial capability.
- Concurrency: refresh replay, concurrent role changes/version increments, duplicate seed safety, and tenant-context isolation under concurrent requests.
- Frontend: strict API-client types, auth/error behavior, tenant-aware cache keys, layout guards, accessibility smoke tests, and no token persistence in browser storage.

### Checks

- Backend format, lint, type, migration drift/conflict, full PostgreSQL tests, RLS inventory, API schema compatibility, dependency and secret scans.
- Web format/lint, strict type check, tests, build, dependency scan, and environment validation.
- Clean setup from documented commands and safe failure with missing secrets or privileged database configuration.

### Exit criteria

- Authentication and gym selection work end to end through the API and web shell.
- Explicit Django scope and forced PostgreSQL RLS independently fail closed.
- Runtime role and pooled-connection isolation are proven.
- All required checks pass with no unapplied or unexpected migration.
- No Phase 1B provisioning or tenant administration feature is represented as complete.
- Phase 1A evidence and open issues are approved.

## Phase 1B — Required Super Admin provisioning and gym prerequisites

### Dependencies

- Phase 1A exit criteria.
- Approved platform permission/MFA policy, provisioning fields, subscription/entitlement defaults, invitation boundary, and owner-protection rules.

### Scope

- Implement separate Super Admin authentication/authorization surface, MFA-ready protected layout, dashboard, gym list/search/detail, lifecycle/subscription management, and platform audit views.
- Implement the required idempotent, transactional gym-and-owner provisioning command: create gym; safely create/link owner identity; create owner `GymMembership`; copy protected roles/permissions; assign OWNER; seed feature entitlements, the protected `RENT`, `UTILITIES`, `INTERNET`, `STAFF_SALARY`, `PT_COMMISSION`, `CLEANING_SUPPLIES`, `MAINTENANCE`, `MARKETING`, `WATER_DRINKS`, and `OTHER` expense categories, tenant sequences, approval policy, and required defaults; append audit evidence.
- Implement the protected co-Owner invitation, listing, login, and revocation workflow. A gym may have multiple equal-authority Owners, but owner lifecycle operations must retain at least one active Owner and must not represent shares or profit-distribution percentages.
- Implement activation/suspension behavior without granting Super Admin ordinary tenant data or financial mutation access.
- Implement gym profile/settings, validated timezone/currency display plumbing, staff invitation/acceptance, `StaffProfile`, staff lifecycle, custom roles, permission replacement, membership-role assignment, delegation checks, and authorization-version increments.
- Deliver prerequisite Super Admin, staff, role, and settings web flows.
- Keep membership plans and member business features in Phase 2.

### Migrations and data

- Add `StaffProfile`, invitation/supporting security records, tenant settings/entitlements, provisioning idempotency, platform audit records, and required composite constraints/RLS policies.
- Add idempotent seed/version migrations for permissions, protected templates, the canonical protected expense-category catalog, feature defaults, approval policy, and tenant sequences.
- Existing-gym backfills must be resumable, auditable, tenant-batched, and verified; no partial tenant is exposed.

### Required tests

- Tenant/RLS: staff, roles, settings, files, and joins cannot cross gyms; provisioning produces complete same-gym relationships; suspended gyms fail closed.
- Permission: platform-only routes; Super Admin has no implicit `tenant.*`; owner-reserved controls; non-owner anti-escalation; protected templates; delegable intersection; stale-token invalidation; custom actions call object checks.
- Financial: provisioning seeds finance prerequisites without posting economic records; currency/timezone defaults are authoritative; no platform path mutates tenant finance.
- Concurrency: duplicate provisioning/idempotency keys yield one gym and one initial Owner; duplicate co-Owner invitations/link attempts are idempotent; concurrent Owner revoke/suspend/role-removal cannot leave an active gym ownerless; concurrent role replacement/version increments are safe; invitation acceptance is single-use; partial failures roll back all provisioning rows.
- Audit/privacy: every platform, lifecycle, support, role, and settings action records redacted evidence; account-existence and invitation secrets do not leak.

### Checks

- Phase 1A checks plus provisioning rollback inspection, seed idempotency, migration/backfill verification, OpenAPI/generated-type compatibility, Super Admin and tenant-settings build/E2E smoke paths.

### Exit criteria

- An authorized Super Admin can provision exactly one complete gym with its required initial Owner atomically and safely retry the request.
- Multiple Owners can log in independently with equal Owner authority. An active Owner can add another Owner through the protected workflow, while no request or race can remove the last active Owner.
- Owners can configure gym settings, invite staff, and manage allowed roles/permissions.
- Suspended gyms and stale permissions fail closed; platform and tenant authority remain separated.
- All checks pass and Phase 2 prerequisites are approved.

## 4. Phase 2 — Members and memberships

### Dependencies

- Phase 1B exit criteria.
- Approved member numbering, membership overlap/cancellation policy, calendar-expiry rules, profile-file limits, and member-dashboard contract.

### Scope

- Implement `Member` CRUD/archive, search, profile, private files, summary, histories, and the read-only `is_pt_client` badge/filter derived from an active `TrainerAssignment`.
- Implement `MembershipPlan` CRUD/archive and immutable plan snapshots on sale.
- Implement `Membership` create, suspend/resume/cancel, exact amount calculation, overlap prevention, and calendar-aware expiry.
- Implement expiry buckets: expired, today, within 3, 7, and 30 days; expiry processing remains synchronous/bounded or platform-scheduled without Celery.
- Implement membership-expiry email at the configured warning days. Create and deliver durable, deduplicated notifications only when the feature is enabled and the member has a current nonblank email; skip members without email and never fall back to SMS/phone. Use bounded scheduled Django commands and the approved transactional-email provider without Celery.
- Implement tenant staff web pages and the API-backed member dashboard/summary for profile, current membership, days remaining, PT-client badge, payment summary, and check-in summary. Before Phase 5 assignment data exists the badge is false; React Native remains Phase 7.

### Migrations and data

- Add `Member`, `MembershipPlan`, `Membership`, member-owned `FileAsset` usage, the limited membership-expiry `Notification` queue, composite foreign keys, exclusions, tenant-leading indexes, forced RLS, notification deduplication, and immutability/snapshot constraints.
- Seed no fake production members. Backfills or imports require approved mapping, validation, tenant batching, dry run, and reconciliation.

### Required tests

- Tenant/RLS: two-gym CRUD/list/filter/search/relationship/file matrix; overlapping human identifiers; cross-tenant path/body/filter/file IDs return safe failures; the derived PT badge cannot observe another gym's assignments.

- Permission: every member/membership action and discount/cancellation condition; trainer and member-self access are not prematurely granted.
- Financial: exact `Decimal` list/discount/final amounts and currency; clients cannot set totals/payment state; no payment is posted in this phase.
- Concurrency: simultaneous membership create/renew/cancel cannot overlap or corrupt snapshots; archive/update conflicts are deterministic.
- Date/UI: month/year/leap/calendar edges in gym timezone, expiry buckets, dashboard accuracy, bounded queries, accessibility, and API contract behavior.
- Expiry email: configured warning-day boundaries, missing/removed/changed email, disabled setting, inactive/cancelled/renewed membership, cross-gym isolation, safe template data, provider failure/retry limits, and duplicate scheduler/delivery runs producing at most one email per deterministic key.

### Checks

- Standard CI gates, migration/RLS inventory, constraint tests, API schema/generated types, query-count and query-plan review for member search/dashboard, file-security tests, web build and E2E member lifecycle.

### Exit criteria

- Staff can manage members, plans, memberships, expiry, and the member dashboard within one gym only; eligible members with email receive one deduplicated expiry warning per configured day, while members without email are skipped.
- Historical plan/price snapshots and calendar expiry remain correct.
- All permission, tenant/RLS, financial-boundary, concurrency, file, and UI checks pass.

## 5. Phase 3 — Check-in and device abstraction

### Dependencies

- Phase 2 exit criteria.
- Approved eligibility, duplicate window, backdating/correction, checkout, device identity, and QR lifetime policies.

### Scope

- Implement `CheckIn`, manual check-in/check-out, eligibility snapshots, duplicate handling, corrections, search/history, and front-desk web flow.
- Implement short-lived opaque/signed QR token validation without personal data in the payload.
- Define scoped device/service credential and ingestion interfaces for NFC and biometric devices.
- Store only check-in evidence and bounded source metadata. Do not store biometric templates, raw biometric captures, or vendor secrets.
- Vendor-specific NFC/biometric integration is optional behind the approved abstraction and cannot weaken manual/QR controls.

### Migrations and data

- Add `CheckIn`, device metadata/credential records only if approved, idempotency indexes, same-gym composite FKs, tenant-leading indexes, and forced RLS.
- Device enrollment/import requires tenant ownership, key rotation, revocation, and audit records.

### Required tests

- Tenant/RLS: cross-gym member/membership/device/check-in access, list/filter/aggregate isolation, missing context, and pooled concurrent requests.
- Permission: create/view/correct/backdate/checkout actions, trainer object scope where enabled, and device credential scope.
- Financial: check-in never mutates payment state or accepts financial totals; eligibility uses authoritative membership state.
- Concurrency: simultaneous check-ins, repeated QR/NFC/device events, idempotency, duplicate-window locking, and checkout races create one valid result.
- Security/device: expired/replayed/modified QR, no PII payload, revoked device, malformed/oversized metadata, biometric-data rejection, rate limits, and audit.

### Checks

- Standard gates, real-PostgreSQL lock tests, RLS inventory, device/QR threat review, query/load baseline for peak check-in, web build, and critical E2E manual/QR flows.

### Exit criteria

- Manual and approved QR check-in paths are reliable, idempotent, tenant-safe, and auditable.
- NFC/biometric vendors can integrate only through the approved abstraction with no biometric template stored.
- Peak baseline, concurrency, permission, tenant/RLS, and security checks pass.

# 6. Phase 4 — Core finance, approvals, salaries, receipts, expenses, and cash

## Dependencies

- Phase 2 exit criteria; Phase 3 is required only where finance screens link check-in history.
- Approved tax/receipt, rounding, refund/void, manager approval, salary-period, cash-register, reconciliation, retention, and financial permission policies.

## Scope

- Implement `Payment`, `Receipt`, `ExpenseCategory`, `Expense`, `SalaryPeriod`, `StaffSalaryLine`, `CashSession`, `CashTransaction`, `TenantSequence`, and finance `IdempotencyRecord` workflows.
- Deliver membership/other non-PT payment and expense `DRAFT → PENDING_APPROVAL → POSTED/REJECTED` workflows, daily sales, atomic receipt allocation/rendering, void/refund/reversal, private expense evidence, salary periods with manually entered per-period lines, cash open/manual adjustment/automatic movement/reconciliation/close, and finance web flows.
- Reception and authorized finance staff may create/edit/submit Payment, Expense, and Salary drafts. Every record requires approval by a different Manager or Owner before posting; approval atomically creates all receipt, settlement, protected salary-expense, and cash effects. Rejection creates none of those effects.
- Posted records are immutable; corrections are linked records with actor, reason, and time.
- Every financial command uses exact `Decimal`, authoritative server calculations, one transaction, permissions, same-gym validation, idempotency, locks, audit, and cash/receipt side effects.
- PT sales, trainer assignments, and PT commissions are explicitly excluded and reserved for Phase 5.

## Migrations and data

- Add all listed models, approval actor/state constraints, append-only/immutability enforcement, sequence allocator, exact checks, same-gym composite FKs, tenant-leading indexes, forced RLS, and restricted runtime grants.
- Seed the canonical protected expense categories and tenant sequences idempotently for existing gyms. Salary posting may use only `STAFF_SALARY`; commission payout may use only `PT_COMMISSION`.
- Financial imports/backfills require reconciliation totals, immutable source references, dry run, rollback, and finance approval.

## Required tests

- Tenant/RLS: two-gym matrix for all finance resources, relationships, files, lists, totals, exports, and custom actions; runtime cannot bypass RLS.
- Permission: create/update/submit/view/approve/void/refund/reconcile/adjust separation; Reception cannot approve; creator/submitter cannot self-approve; high-risk reasons, Super Admin denial, restricted salary visibility, and mass-assignment rejection.
- Financial: exact decimal/rounding/currency; immutable posts; valid approval/rejection/void/refund/reversal links; rejected submissions have no economic effects; salary expense linkage; refundable limits; receipt snapshots; cash in/out; expected/actual/difference formulas; closed-session rules; audit completeness.
- Concurrency: duplicate idempotency keys, simultaneous submit/approve/reject, payment/refund/void/expense/salary approval, atomic receipt sequences, one open cash session, transaction-versus-close races, reconciliation races, deadlock retry bounds, rollback with no partial side effects.
- Files/UI: private receipt evidence, safe generated receipts, timezone business-day boundaries, accessibility, and end-to-end payment-to-reconciliation workflows.

## Checks

- Standard gates plus database trigger/grant inspection, sequence stress test, financial reconciliation suite, production-like pooling, bounded query plans, dependency/file scans, schema/generated types, web build, and finance E2E smoke.

## Exit criteria

- Core non-PT finance and salaries reconcile exactly from posted, non-void source records through receipts, expenses, and cash.
- No financial record can post without approval by a different Manager or Owner.
- Retry, race, rollback, refund, void, and close paths preserve one valid immutable history.
- No PT model, endpoint, calculation, or UI is represented as Phase 4 scope.
- All checks pass and finance/security evidence is approved.

# 7. Phase 5 — Personal training

## Dependencies

- Phase 4 exit criteria.
- Approved trainer eligibility/object scope, assignment overlap, PT sale/payment approval, commission snapshot/accrual/payout/reversal, and separation-of-duties policies.

### Scope

- Implement `TrainerAssignment`, trainer views, assignment lifecycle, and assigned-member restrictions.
- Implement PT-sale draft/submission with approval by a different Manager or Owner and optional atomic payment/receipt/cash effects through Phase 4 services.
- Implement `PTCommission` accrual, exact trainer/gym split, partial/full payout, linked expense/cash effects, outstanding balance, reversal, and trainer-own views.
- Deliver staff and trainer web workflows; do not duplicate finance posting logic.

### Migrations and data

- Add assignment, PT sale, and commission models with snapshots, exclusions, exact checks, same-gym composite FKs, tenant-leading indexes, forced RLS, immutable states, and restricted grants.
- Existing trainer data migration requires explicit staff-profile mapping and reconciliation of all opening commission balances.

### Required tests

- Tenant/RLS: assignments, sales, payments, receipts, commissions, expenses, and trainer views cannot cross gyms.
- Permission/object scope: trainer sees only assigned members and own commissions; payout is denied; Reception can submit but cannot approve; creator/submitter cannot self-approve; assignment/sale/pay permissions and protected financial actions follow the matrix.
- Financial: exact percentage snapshot, `ROUND_HALF_UP`, gym share, paid/outstanding totals, atomic linked payment/receipt/expense/cash records, and valid reversal ordering.
- Concurrency: overlapping assignments, duplicate PT submit/approve/reject, one commission per approved sale, concurrent partial payouts, payout-versus-void, idempotent retries, and rollback without partial finance.
- UI/API: assignment date boundaries, trainer dashboard privacy, safe errors, accessibility, and end-to-end PT sale-to-payout reconciliation.

### Checks

- Standard gates, financial reconciliation and lock tests, RLS/permission inventory, schema/generated types, query plans for assigned-member/commission lists, web build, and PT E2E.

### Exit criteria

- Assignments, PT sales, and commissions are tenant-safe, permission-safe, exactly calculated, and fully reconcilable.
- Trainers cannot access unassigned members, other trainers' private commissions, or payout controls.
- All checks pass and Phase 6 aggregation policy is approved.

## 8. Phase 6 — Dashboards and reports

### Dependencies

- Phases 2–5 exit criteria for the source domains included in each metric.
- Approved posted-only metric definitions, source-status policy, timezone cutoffs, permission split, date/row limits, export behavior, and performance budgets.

### Scope

- Implement gym operational, owner financial, and trainer-scoped dashboards. The owner contract includes active/new/expired members, check-ins, pending approvals, membership/PT/other/total revenue, expenses by protected category, staff salary expense, PT commission expense, total expenses, and net profit.
- Implement daily, monthly, cash-book, profit/loss, membership/expiry, check-in, and PT reports from authoritative records.
- Use bounded filters, stable pagination, server-calculated totals, explicit timezone/currency, and tenant-leading selectors.
- Keep exports synchronous and bounded initially. Background exports, Redis, Celery, caches, aggregates, replicas, or warehouses require measured evidence and approval.

### Migrations and data

- Prefer no mutable report tables. Add only measured indexes or approved tenant/date aggregates with reproducible rebuild and reconciliation rules.
- Any backfill is tenant-batched, resumable, forced-RLS safe, and reconciled to source records.

### Required tests

- Tenant/RLS: all counts, sums, joins, annotations, filters, empty/adversarial ranges, dashboard cards, and exports exclude other gyms.
- Permission/object scope: operational versus financial reports, receptionist denial, owner/accountant rules, trainer-assigned-only aggregates, and export permissions.
- Financial: posted/void/refund/reversal policy and all dashboard/report formulas reconcile exactly to Phase 4/5 source records; drafts, pending submissions, and rejections affect only `pending_approvals`, never revenue, expense, or profit.

- Concurrency: reports remain transactionally coherent during posting/refund/close/payout; duplicate export requests and cache/aggregate refresh, if approved, are safe.
- Performance/UI: timezone boundaries, bounded ranges, realistic volumes, query plans, N+1 limits, accessibility, loading/error states, and downloadable output safety.

### Checks

- Standard gates, formula reconciliation, RLS aggregate matrix, explain-plan review, performance-budget tests, schema/generated types, web build, and dashboard/report E2E.

### Exit criteria

- Every displayed total has an approved definition and reconciles to tenant-scoped source records.
- Permission and object scopes apply to rows, totals, exports, and cached/derived data equally.
- Performance budgets are met without unapproved Redis/Celery or weakened consistency.

## 9. Phase 7 — React Native member application

### Dependencies

- Stable member APIs from Phases 2–6.
- Approved ADR choosing Expo or bare React Native after evaluating QR, notifications, native modules, device/biometric interaction, build ownership, and store delivery.
- Approved mobile token, secure-storage, privacy, accessibility, supported-device, and release policies.

### Scope

- Create the separate TypeScript React Native application and environment/build documentation.
- Implement member-specific login/refresh/logout, secure credential storage, profile, member dashboard, current membership/days remaining, payment/check-in histories, short-lived QR display, notifications/read state, and safe offline/loading/error behavior.
- Every member API enforces both tenant and immutable member-self scope; staff roles are never inherited.
- Device biometric unlock may protect local credentials through platform APIs, but the application does not collect or store biometric templates.

### Migrations and data

- Add only approved member-auth/session, notification, device-registration, or push-token records, each with ownership, retention, revocation, RLS, and privacy controls.
- No local mobile database becomes a system of record; migrations clear or transform cached sensitive data safely.

### Required tests

- Tenant/RLS/self: changing gym/member/path/body IDs cannot expose another member; lists, files, QR, and notifications remain self-scoped.
- Permission/auth: member token audience/type, refresh/revocation, suspended member/gym, staff-token rejection, and profile field allowlists.
- Financial: payment history and totals are read-only and match authoritative server records; no client-side recalculation is trusted.
- Concurrency: refresh races, repeated notification reads, QR rotation/replay, multi-device logout/revocation, and cache invalidation.
- Mobile: Keychain/Keystore storage, logout cleanup, no sensitive logs/screenshots where controlled, offline/error/loading states, accessibility, supported devices, deep-link safety, and Android/iOS builds.

### Checks

- Standard API gates plus mobile format/lint/strict types/tests, dependency/secret scan, signed test builds, device smoke tests, store-policy review, and E2E member journeys.

### Exit criteria

- Approved Android/iOS builds provide only self-scoped member features with secure session handling.
- QR and local biometric use expose no personal or biometric template data.
- All server, mobile, privacy, accessibility, and release checks pass.

## 10. Phase 8 — Hardening, operations, and production deployment

### Dependencies

- Phases 1–7 exit criteria for all features included in the release.
- Approved production topology, domains, environments, infrastructure-as-code approach, data jurisdiction, privacy/retention, RPO/RTO, SLOs, performance budgets, incident ownership, and launch plan.
- Independent security review/penetration-test scope and production change approval.

### Scope

- Complete security hardening: IDOR/cross-tenant, privilege escalation, mass assignment, JWT/session, CSRF/CORS/XSS/injection, rate limits, files, dependencies, containers, secrets, and break-glass/support access.
- Run production-like load and soak tests for authentication, peak check-in, dashboards/reports, financial writes, receipt allocation, cash close, and PT payout.
- Implement structured logs, request/trace correlation, metrics, tracing, error reporting, audit export, dashboards, alerts, SLOs, on-call routing, and sensitive-data redaction.
- Provision the budget production topology: Cloudflare, one hardened VPS, Caddy, Docker Compose, Next.js, Django/Gunicorn, PostgreSQL with forced RLS, private MinIO, GHCR, offsite encrypted backup storage, and protected CI/CD environments.
- Configure host firewall and SSH hardening, private Docker networks, separate database roles, forced-RLS health/security checks, fixed worker/container limits within the VPS connection and memory budget, WAL-based recovery, object versioning/sync, key rotation, and least privilege.
- Implement controlled migration jobs, immutable artifacts, staged rollout, smoke/isolation tests, rollback, incident, restore, key-rotation, and break-glass runbooks.
- Complete an isolated restore drill and validate RLS, roles, constraints, financial reconciliation, and file references before launch.

### Migrations and data

- Production migrations use expand/migrate/contract where destructive or lock-sensitive, with rehearsal, duration/lock evidence, backup checkpoint, data verification, and rollback/forward-fix plan.
- Migration identity applies schema; runtime never owns objects. Seed/backfill jobs are idempotent, resumable, tenant-batched, audited, and reconciled.
- Production data is never copied to lower environments without approved minimization/anonymization.

### Required tests

- Tenant/RLS: complete two-gym resource/action/aggregate/file/export matrix under production-like pooling; metadata inventory; runtime grants; missing context; rollback/error/reuse/concurrent gym isolation.
- Permission/security: every matrix default and conditional path, custom-role escalation, stale authorization, platform/tenant boundary, member self-scope, support/break-glass audit, attack suite, and penetration-test remediation.
- Financial: full exact reconciliation, immutable-state enforcement, idempotency, receipt uniqueness, refunds/voids, cash close, PT payout, rollback, and restore integrity.
- Concurrency/load: realistic duplicate/race/deadlock tests, connection and VPS CPU/RAM/disk saturation, fixed worker/container budget, load/soak thresholds, graceful overload, deployment during traffic, and no cross-tenant context leakage.
- Operations: backup freshness alerts, point-in-time and full restore, key rotation, secret failure, alert delivery, audit retention, rollback, disaster/incident exercises, smoke tests, and mobile/web release checks.

### Checks and deployment sequence

1. All CI, security, schema, migration, build, dependency, image, and secret gates pass.
2. Provision and validate an isolated local/temporary-VPS staging Compose stack with separate credentials and volumes.
3. Rehearse migrations with the migration identity and verify runtime grants/RLS.
4. Publish digest-pinned web/API images to GHCR and deploy the release candidate through the protected Compose workflow.
5. Run smoke, isolation, financial reconciliation, load, and restore tests.
6. Resolve all critical/high findings; explicitly approve any lower-risk acceptance.
7. Approve the production change and confirmed rollback point.
8. Take the pre-deploy backup checkpoint, apply migrations, restart services in controlled order, and monitor SLOs, logs, alerts, VPS resources, database connections, and core workflows.
9. Stop or roll back on errors; do not advance rollout stages while any release gate is failing.
10. Complete launch verification and retain rollback/incident readiness.

### Exit criteria

- No unresolved critical/high security, tenant-isolation, data-integrity, financial, backup, or availability defect.
- Performance budgets and database connection limits are met under production-like load.
- Penetration/security review findings are resolved or explicitly risk-accepted by authorized owners.
- Restore, rollback, incident, key-rotation, alerting, and break-glass exercises pass.
- Production deployment is observable, reproducible, least-privileged, and approved.

## 11. Cross-phase test and quality strategy

### Test pyramid and required evidence

- Unit: calculations, validators, permission decisions, state transitions, and date logic.
- Database/model: constraints, composite relationships, RLS, immutability, sequences, locks, and migration behavior.

- API: explicit contracts, errors, serializers, filters, pagination, idempotency, authorization, and mass assignment.
- Integration: membership, check-in, payment, receipt, expense, cash, PT, commission, report, file, and audit workflows.
- Security: two-gym matrix, member self-scope, platform/tenant separation, attack cases, and production-like pooling.
- Frontend/mobile: typed clients, forms, permission-aware presentation, accessibility, offline/error behavior, and critical E2E journeys.
- Performance: realistic PostgreSQL volumes, query plans, connection budgets, load, soak, and graceful degradation.

Money tests assert exact `Decimal` values. Local-date tests set an explicit gym timezone. RLS, locking, exclusion, and PostgreSQL constraint tests never use SQLite. Factories require an explicit gym for tenant records; security fixtures use at least Gym A and Gym B with overlapping human-readable identifiers.

### CI quality gates

Every pull request must pass all applicable formatting, lint, Python/TypeScript type, migration drift/conflict, unit, API, permission, tenant/RLS, financial, concurrency, schema compatibility, generated type, build, dependency, image, and secret checks. Main-branch deployment additionally requires staging migration, smoke, isolation, reconciliation, observability, and rollback readiness.

No failed, flaky, skipped, or missing required test permits phase advancement.

## 12. Backup, recovery, and continuity

- PostgreSQL writes daily encrypted base/logical backups and continuous WAL archives to independent offsite storage. Backups kept only on the VPS do not satisfy recovery policy.
- Private MinIO objects use approved versioning/lifecycle and encrypted offsite synchronization with reference and checksum validation.
- Backup age, WAL archive, object-sync, encryption, capacity, and failure alerts are production requirements.
- Quarterly restore drills rebuild an isolated temporary Compose stack or replacement VPS from source configuration, immutable GHCR images, and offsite backups.
- A restore validates database roles, forced RLS, constraints, tenant isolation, financial reconciliation, audit evidence, and object references before exposure.
- Initial planning targets remain RPO ≤ 15 minutes and RTO ≤ 4 hours; they are restore targets rather than high-availability commitments and must be proven before launch.

## 13. Scalability path

### Near term

- Tenant-leading indexes, cursor pagination, bounded reports/exports, query-plan review, N+1 prevention, stateless API, private object storage, and strict disk/log retention.
- Explicit VPS worker/memory budget and PostgreSQL connection limit with migration, backup, and operations reserve.
- Load tests before scaling configuration changes.

### Evidence-driven later options

- Redis only for a measured cache, rate-limit, or distributed-lock requirement with tenant-safe keys, invalidation, outage behavior, and no authorization fail-open.
- Celery plus transactional outbox only for measured asynchronous notifications, scans, exports, or scheduled work with one gym context per transaction, idempotency, retries, and operational ownership.
- Reproducible tenant/date aggregates, validated read replicas, and time partitioning for high-volume check-in/audit/transaction data.
- Tenant-safe analytics warehouse, regional/data-residency deployment, or dedicated databases only behind stable domain/API contracts.
- Service extraction only around demonstrated ownership, reliability, or scaling boundaries.

No scaling change may weaken tenant isolation, permission enforcement, financial semantics, auditability, restore capability, or consistency without explicit approval.

## 14. Phase 0 deployment cost planning estimate

**Estimate date:** 2026-08-15. **Currency and period:** USD per month. **Target:** one budget-production VPS in a low-latency Southeast Asia region.

This is a planning budget, not a vendor quote. Actual prices vary by provider, region, tax, bandwidth, storage growth, exchange rate, and available promotions. Provider selection must meet the approved CPU/RAM/NVMe, backup, data-jurisdiction, and support requirements rather than choosing price alone.

### Approved monthly target

| Cost area | Planning range | Included assumption |
|---|---:|---|
| Production VPS | About $50 | 4–8 vCPU, 8–16 GiB RAM, 80–160 GiB NVMe, Linux, Docker Compose |

| Cost area | Planning range | Included assumption |
|---|---|---|
| Encrypted offsite backup/object storage | About $10 | PostgreSQL base/logical backup, WAL archive, MinIO object copy, bounded retention |
| Edge, domain, monitoring, and email allowance | About $20 | Cloudflare free baseline, domain allocation, external uptime/alerting, limited transactional email |
| **Estimated monthly production cost** | **About $80** | One production VPS; no high availability or permanent hosted staging |

The target initially supports a small rollout, planned around up to 10 gyms, approximately 1,000 monthly active users, bounded file retention, and up to roughly one million API requests per month. These are planning limits, not guarantees; production load tests and monitoring determine the actual safe capacity.

### Cost and availability trade-off

- Next.js, Django, PostgreSQL, and MinIO share one VPS. Host or datacenter failure can take the full service offline.
- The budget buys restore capability, not automatic failover. Offsite backups, WAL archiving, monitored backup freshness, a documented rebuild script, and quarterly restore drills are mandatory.
- A permanent remote staging environment, managed database, second application node, load balancer, and hot standby are outside the approximately $80 baseline.
- Capacity alerts trigger vertical VPS resizing first. Sustained resource pressure, repeated SLO failure, more than 10 active gyms, database/storage growth beyond tested headroom, or restore requirements that exceed RTO trigger review of a separate/managed PostgreSQL service or multi-node architecture.

### Capacity, recovery, and approval conditions

- Initial connection budget: `API workers × maximum connections per worker + migration/operations reserve < PostgreSQL max_connections`. Worker count and memory limits must leave explicit headroom for PostgreSQL, MinIO, backup, and the operating system.
- RPO ≤ 15 minutes requires continuous offsite WAL archiving; RTO ≤ 4 hours requires a tested replacement-VPS build and restore runbook. If drills do not meet either target, the target or budget must change before launch.
- The selected VPS region/provider, data jurisdiction, measured latency, backup-provider independence, saved monthly quote, and upgrade pricing are production approval evidence.
- Redis and Celery remain deferred and are not included. Cost alone does not authorize weakening tenant isolation, backups, financial integrity, or security gates.

### Exclusions

The target excludes taxes, premium support, high availability, a permanent hosted staging server, SMS/push/payment-provider fees, penetration tests and remediation, mobile app-store/build fees, legal/compliance services, engineering/operations staffing, temporary incident-replacement servers, and future multi-node architecture. Large email volume, file egress, or backup growth can increase the approximately $80 estimate and requires approval before purchase.

## 15. Open decisions and approval register

Track each decision with owner, due phase, options, security/data/migration/cost impact, status, and approval reference. Initial decisions include:

1. Product domains and JWT issuer/audience values.
2. Supported Python, Node, framework, PostgreSQL, and mobile versions after compatibility verification.
3. Next.js BFF versus direct secure-cookie token termination.
4. Email/SMS/push, invitation, recovery, file-scan, and notification providers.
5. MFA provider and rollout beyond mandatory Super Admin/owner requirements.
6. Local-development and single-VPS production runtime/migration/reporting database role strategy; any future managed database requires a new approved architecture decision.
7. **Resolved for Phase 0:** Every Payment, Expense, PT Sale, and Salary record uses mandatory `DRAFT → PENDING_APPROVAL → POSTED/REJECTED`; a different MANAGER or OWNER approves and approval is the posting boundary. Approval permissions are hard-reserved to those protected roles. Threshold-based auto-posting and self-approval are not allowed. Future policy changes require an approved migration and security review.
8. Jurisdiction, tax/receipt behavior, privacy, retention, RPO/RTO, and data residency.
9. Expo versus bare React Native.
10. **Resolved baseline:** Cloudflare edge plus single-VPS Docker Compose production topology with Caddy, PostgreSQL forced RLS, private MinIO, GHCR deployment, encrypted offsite backup, and bounded observability. Provider/region and final sizing remain Phase 8 evidence.
11. **Resolved baseline:** estimated production infrastructure cost is **about $80/month** with the documented single-point-of-failure trade-off and mandatory restore controls. A managed database or high-availability topology requires a new approved budget.

A blocker remains a blocker until explicitly resolved. Major architecture changes and progression to every next phase require approval.

# ADR 0001: Single-VPS Budget Production Topology

- **Status:** Accepted for Phase 0 baseline
- **Decision date:** 2026-08-15
- **Budget target:** approximately USD 80 per month

## Context

The earlier managed-cloud design used separate hosted web, API, database, object-storage, secret-management, registry, and monitoring services. Its several-hundred-dollar monthly production estimate is above the approved initial operating budget.

The initial rollout targets a small deployment of approximately ten gyms, 1,000 monthly active users, bounded file retention, and roughly one million API requests per month. Tenant isolation, PostgreSQL forced RLS, financial integrity, encrypted offsite backups, and restore testing cannot be weakened to meet the budget.

## Decision

Production will run on one hardened Linux VPS using Docker Compose:

- Cloudflare provides DNS, edge TLS, WAF, and rate controls.
- Caddy is the only public origin container and exposes ports 80/443.
- Next.js and Django/Gunicorn run as separate non-root containers.
- PostgreSQL runs on a private Docker network with persistent local storage, separate runtime/migration roles, and forced RLS.
- MinIO provides private S3-compatible object storage on the private network.
- GitHub Actions publishes digest-pinned images to GHCR and invokes a protected Compose deployment.
- PostgreSQL base/logical backups, continuous WAL archives, and MinIO object copies are encrypted and stored with an independent offsite provider.
- External monitoring covers HTTPS availability, certificates, host capacity, PostgreSQL, backup/WAL/object-copy freshness, and deployment failures.
- Staging runs locally or on an ephemeral temporary VPS with separate credentials and volumes; permanent staging does not share the production VPS.

## Budget

The monthly planning allocation is:

- VPS: approximately USD 50;
- encrypted offsite backup/object storage: approximately USD 10;
- Cloudflare/domain/monitoring/limited email allowance: approximately USD 20;
- estimated monthly total: approximately USD 80.

This is a planning estimate, not a provider quote. Provider, region, taxes, bandwidth, backup capacity, and measured load can change the actual price.

## Security and recovery consequences

Application security remains unchanged: explicit Django gym scoping and PostgreSQL forced RLS are independent mandatory controls. PostgreSQL and MinIO are not publicly exposed. Secrets do not enter images, Compose files, source control, logs, or shell history.

One VPS is a single point of failure. This topology provides restore capability rather than high availability. The initial RPO target of 15 minutes depends on healthy offsite WAL archiving; the RTO target of four hours depends on a rehearsed replacement-VPS build and restore. Both targets must be demonstrated by quarterly drills before they can be treated as operational commitments.

## Alternatives considered

### Managed cloud services

Separate managed web, API, database, object, secret, registry, and monitoring services provide stronger availability and lower operational burden but do not fit the approved initial budget.

### Separate application and database VPSs

This improves resource isolation and permits independent maintenance, but two appropriately sized hosts plus backup capacity may exceed the target. It is the preferred first structural upgrade after vertical resizing.

### Kubernetes

Kubernetes adds cost and operational complexity without evidence of a current scaling requirement. It is not approved for the baseline.

## Upgrade triggers

Architecture and budget must be reviewed when any of these occur:

- more than ten active gyms or 1,000 monthly active users;
- sustained CPU, RAM, disk, I/O, or database-connection pressure outside tested headroom;
- database/object growth or backup windows threaten the approved RPO/RTO;
- repeated availability/SLO failures caused by the shared host;
- security, data residency, contractual, or regulatory requirements demand stronger isolation or availability;
- restore drills fail the approved recovery targets.

The upgrade sequence is vertical VPS resizing, then separation or management of PostgreSQL/object storage, then multiple application nodes/load balancing. Every step must preserve tenant isolation, financial semantics, audit evidence, and recoverability.

## Rollback and compatibility

This decision changes deployment and operations only. Public API, database schema, IDs, RLS policy, and domain contracts remain provider-neutral. The system can move later to managed PostgreSQL, object storage, and multi-node application hosting by restoring the same PostgreSQL data and object keys behind the existing service interfaces.